



Universidad de Castilla-La Mancha

Escuela Superior de Ingeniería Informática de Albacete

Escuela Superior de Informática de Ciudad Real

Trabajo Fin de Máster

Máster Universitario en Big Data y Computación en la Nube

Diseño y configuración de un modelo de metadatos en OpenMetadata conforme al estándar DCAT-AP para la interoperabilidad de catálogos de datos

Alonso Marcos Muñoz

Julio, 2026

TRABAJO FIN DE MÁSTER

Máster Universitario en Big Data y Computación en la Nube

Diseño y configuración de un modelo de metadatos en OpenMetadata conforme al estándar DCAT-AP para la interoperabilidad de catálogos de datos

Autor: Alonso Marcos Muñoz

Tutor: Fernando Gualo Cejudo

Codirector: Antonio Labian Moya

Julio, 2026

*A mi familia y a quienes me han acompañado durante el máster,
por la paciencia y el apoyo que han hecho posible
llegar hasta este trabajo.*

Declaración de autoría

Yo, Alonso Marcos Muñoz , con DNI 06281718G , declaro que soy el único autor del Trabajo Fin de Máster titulado “Diseño y configuración de un modelo de metadatos en Open-Metadata conforme al estándar DCAT-AP para la interoperabilidad de catálogos de datos”. El trabajo se ha realizado de forma individual y el material procedente de terceros se cita o atribuye de acuerdo con su naturaleza documental, técnica o bibliográfica. Asimismo, declaro que el contenido aquí presentado no infringe las leyes vigentes en materia de propiedad intelectual.

Albacete, a ... de ... de 2026

Fdo.: Alonso Marcos Muñoz

Resumen

Este Trabajo Fin de Máster diseña, implementa y valida una Plataforma de Gobierno del Dato centrada en la gestión de metadatos técnicos y funcionales sobre OpenMetadata y en su exportación interoperable conforme al perfil DCAT-AP-ES. A partir del objetivo de alinear OpenMetadata con el perfil DCAT-AP, el problema se concreta en un caso de uso de validación reproducible que recorre el ciclo completo de gobierno de metadatos: desde la conexión de un sistema fuente real hasta un catálogo interoperable validado de forma automatizada.

La plataforma se construye como un flujo extremo a extremo y reproducible. Sobre Kubernetes se despliegan OpenMetadata y un PostgreSQL de referencia (esquemas *bronze*, *silver* y *gold*) como fuente técnica canónica. Sus activos técnicos (servicios, bases de datos, esquemas, tablas y columnas) se ingieren en OpenMetadata, donde se enriquecen con metadatos de gobierno. La capa *gold* se somete después a curación funcional, es decir, a la edición humana de los metadatos de negocio de cada conjunto (título, descripción, publicador y temática) en una hoja CSV versionada. Un paquete Python propio, `om_dcat_sync`, concentra toda la lógica de negocio: descubrimiento de activos, sincronización repetible (idempotente) del gobierno, exportación del catálogo a JSON-LD y validación. Una interfaz de línea de comandos y una consola web operativa reutilizan ese mismo núcleo sin duplicar reglas. La consola permite ejecutar el flujo y editar la hoja sin memorizar comandos.

La validación combina tres niveles: pruebas automatizadas unitarias y de integración sobre el núcleo Python y la consola web, una comprobación estructural del estado vivo de OpenMetadata frente al contrato esperado, a modo de prueba de aceptación técnica, y una validación semántica SHACL del perfil DCAT-AP-ES para el caso de datos de alto valor (HVD), con shapes oficiales congeladas en el repositorio. El resultado es una plataforma reproducible que transforma activos gobernados en un catálogo RDF en JSON-LD, interoperable y federable en ecosistemas como `datos.gob.es` o `data.europa.eu`, defendible mediante evidencias trazables.

Abstract

This Master's Thesis designs, implements and validates a Data Governance Platform focused on managing technical and functional metadata in OpenMetadata and on its interoperable export according to the DCAT-AP-ES profile. From the goal of aligning OpenMetadata with the DCAT-AP profile, the work frames the problem as a reproducible validation use case that spans the full metadata-governance cycle: from connecting a real source system to obtaining an interoperable catalogue validated in an automated way.

The platform is built as a reproducible end-to-end flow. OpenMetadata and a reference PostgreSQL database with bronze, silver and gold schemas, acting as the canonical technical source, are deployed on Kubernetes. Its technical assets (services, databases, schemas, tables and columns) are ingested into OpenMetadata and enriched with governance metadata. The gold layer then undergoes functional curation, that is, the human editing of each dataset's business metadata (title, description, publisher and theme) through a versioned CSV sheet. A custom Python package, `om_dcat_sync`, centralises all business logic: asset discovery, repeatable (idempotent) governance synchronization, JSON-LD catalogue export and validation. Both a command-line interface and an operational web console reuse that same core without duplicating rules. The web console makes it possible to run the flow, edit the functional sheet and inspect logs and artefacts without memorising commands.

Validation combines three levels: automated unit and integration tests over the Python core and the web console, a structural check of OpenMetadata's live state against the expected contract, as a technical acceptance test, and SHACL semantic validation of the DCAT-AP-ES profile for the High-Value Datasets (HVD) case, using official shapes frozen in the repository. The result is a simple, repeatable and reproducible platform that turns governed technical assets into an interoperable RDF catalogue serialized in JSON-LD, ready for federation with compatible ecosystems such as `datos.gob.es` or `data.europa.eu`, and defensible through traceable evidence.

Agradecimientos

Quiero expresar mi agradecimiento a las personas e instituciones que han hecho posible este trabajo.

A mi tutor, Fernando Gualo Cejudo, y a mi codirector, Antonio Labian Moya, por el acompañamiento técnico y académico a lo largo de todo el desarrollo. Su orientación constante, su criterio exigente sobre el alcance, la evidencia y la trazabilidad, y su disponibilidad para revisar y discutir cada decisión han sido determinantes para dar forma y rigor a esta memoria. Les agradezco también el marco académico proporcionado por la Universidad de Castilla-La Mancha y por el Máster Universitario en Big Data y Computación en la Nube, en el que se enmarca este Trabajo Fin de Máster.

A mi familia y a las personas cercanas, por su paciencia, comprensión y apoyo incondicional durante los meses dedicados al máster y, de manera especial, a la preparación de este trabajo. Su ánimo en los momentos de mayor carga y su confianza constante han sido un sostén imprescindible para llegar hasta aquí.

Índice general

1	Introducción	1
1.1	Contexto	1
1.2	Problema abordado	2
1.3	Solución propuesta	3
1.4	Aportación del trabajo	4
1.5	Alcance	5
1.6	Exclusiones	6
1.7	Estructura del documento	6
2	Objetivos	7
2.1	Objetivo general	7
2.2	Objetivos específicos	7
2.3	Cambio de alcance respecto a CKAN	8
3	Estado del arte	9
3.1	Gobierno del dato y gestión de metadatos	9
3.2	Catálogos de datos y metadatos técnicos	10
3.3	RDF, JSON-LD y vocabularios semánticos	11
3.3.1	<i>Qué es RDF y por qué se utiliza</i>	11
3.3.2	<i>JSON-LD como serialización elegida</i>	12
3.4	DCAT, DCAT-AP y DCAT-AP-ES	13
3.5	Datos de alto valor y caso HVD	13
3.6	Validación SHACL	13

3.7	OpenMetadata	14
3.7.1	<i>Gobierno centralizado frente a edición dataset por dataset</i>	14
3.8	Mercado de herramientas de gobierno alineadas con DCAT-AP-ES	15
3.8.1	<i>Posicionamiento de la Plataforma de Gobierno del Dato del TFM</i>	16
3.9	Reproducibilidad, idempotencia e <i>Infrastructure as Code</i>	17
3.10	Validación frente a verificación	17
3.11	Síntesis del estado del arte	18
4	Metodología de trabajo	19
4.1	Enfoque general	19
4.2	Tablero y vistas en GitHub Project	20
4.3	Fases del roadmap	20
4.4	Vista temporal del trabajo	21
4.5	Stack tecnológico	21
4.5.1	<i>PostgreSQL como fuente técnica reproducible</i>	21
4.5.2	<i>Docker, Kind, Kubernetes y Helm</i>	22
4.5.3	<i>Consola web: React, Next.js y TypeScript</i>	22
4.6	Organización del repositorio	23
5	Resultados	25
5.1	Arquitectura de la solución	25
5.1.1	<i>Actores del sistema</i>	25
5.1.2	<i>Requisitos funcionales</i>	26
5.1.3	<i>Requisitos no funcionales</i>	27
5.1.4	<i>Arquitectura lógica</i>	28
5.1.5	<i>Arquitectura física</i>	28
5.1.6	<i>Flujo extremo a extremo</i>	28
5.1.7	<i>Decisiones de diseño</i>	31
5.1.8	<i>Modelo activo DCAT-AP-ES</i>	33
5.2	Implementación	35
5.2.1	<i>Fuente técnica reproducible: PostgreSQL en tres capas</i>	35
5.2.2	<i>Doble ingesta técnica</i>	36
5.2.3	<i>Núcleo Python: paquete <code>om_dcat_sync</code></i>	37
5.2.4	<i>Hoja funcional de gobierno y defaults globales</i>	39
5.2.5	<i>Consola web operativa Next.js</i>	39
5.2.6	<i>Despliegue reproducible en Kubernetes</i>	40
5.2.7	<i>Resumen de implementación</i>	40

5.3	Validación y resultados de la validación	40
5.3.1	<i>Pruebas automatizadas</i>	40
5.3.2	<i>Validación estructural del estado vivo</i>	41
5.3.3	<i>Validación SHACL</i>	41
5.3.4	<i>Validación externa con el validador europeo (SEMIC/ITB)</i>	41
5.3.5	<i>Resultados del caso de uso de validación</i>	42
5.3.6	<i>Artefactos generados</i>	43
5.4	Discusión de los resultados	44
5.4.1	<i>Cumplimiento del problema planteado</i>	44
5.4.2	<i>Beneficios obtenidos</i>	44
5.4.3	<i>Limitaciones</i>	45
5.4.4	<i>Amenazas a la validez</i>	45
5.5	Trazabilidad entre objetivos y evidencias	46
6	Conclusiones y trabajos futuros	47
6.1	Revisión del objetivo principal	47
6.2	Cumplimiento de los objetivos	47
6.3	Justificación de competencias	48
6.4	Trabajo futuro	49
	Referencia bibliográfica	54
A	Reproducción del caso de uso de validación	55
A.1	Preparación del entorno	55
A.2	Despliegue de infraestructura	55
A.3	Ejecución del workflow canónico	56
A.4	Suite reproducible	56
B	Detalle de infraestructura	57
B.1	Versión de OpenMetadata	57
B.2	Matriz de versiones del sistema	57
B.3	Valores Helm de OpenMetadata	58
B.4	Manifiesto de PostgreSQL de referencia	58
B.5	Diagrama de despliegue	59
B.6	Notas operativas	59
B.7	Instalación portable y organismos con OpenMetadata propio	59

C	Esquema de gobierno funcional.....	61
C.1	Estructura de la hoja <code>gold_governance.csv</code>	61
C.2	Defaults globales	61
D	Mapecto DCAT-AP-ES y entregables.....	63
D.1	Correspondencia OpenMetadata → DCAT-AP-ES	63
D.2	Qué gobierna OpenMetadata y qué deriva el sistema	63
D.3	Qué aporta activar el caso HVD.....	64
D.4	Entregables del caso de uso.....	64
E	Validación y consola web	65
E.1	Comandos de validación	65
E.2	Estructura de la consola web.....	65
E.3	Pruebas de la consola web.....	65
F	Demostración guiada de la plataforma.....	67
F.1	Propósito y supuestos iniciales	67
F.2	Apertura de la consola.....	68
F.3	Estado de la infraestructura.....	68
F.4	Servicios y datasets ingeridos	69
F.5	Curación funcional en la pantalla <i>Gobierno</i>	69
F.6	Aplicación del workflow.....	70
F.7	Exportación DCAT-AP-ES.....	71
F.8	Validación SHACL y suite reproducible	72
F.9	Artefactos y descarga del catálogo RDF.....	73
F.10	Historial de ejecuciones.....	74
F.11	Federación del catálogo.....	74
F.12	Resumen del recorrido.....	75
G	Capturas y listados complementarios	77
G.1	Organización del repositorio y planificación	77
G.2	Infraestructura y despliegue	79
G.3	Ingesta y gobierno en OpenMetadata.....	80
G.4	Núcleo Python y pipeline de metadatos	82

G.5	Consola web operativa	82
G.6	Listados de código y configuración	85
G.6.1	<i>Núcleo Python</i>	85
G.6.2	<i>Exportación, configuración e infraestructura.</i>	86
G.6.3	<i>Ejemplos de RDF y SHACL</i>	88
G.7	Columnas de la hoja de gobierno	89
H	Posicionamiento Big Data y gobierno supervisado	91
H.1	Posicionamiento frente a entornos Big Data	91
H.2	Gobierno supervisado y responsabilidad del operador.	92

Índice de figuras

1.1	Visión funcional de la Plataforma de Gobierno del Dato: de la fuente técnica al catálogo interoperable validado. Elaboración propia.	3
1.2	Flujo lógico por capas y componentes de la Plataforma de Gobierno del Dato. Elaboración propia.	4
4.1	Vista Kanban del GitHub Project del TFM al cierre de la fase 05. Elaboración propia.	20
5.1	Diagrama de secuencia de la operación del responsable del catálogo, desde la curación de la hoja hasta el catálogo validado. Elaboración propia.	26
5.2	Arquitectura lógica por capas y responsabilidades: fuente técnica, catálogo técnico, gobierno funcional, núcleo Python y operación. Elaboración propia.	29
5.3	Arquitectura física de despliegue: host con Docker y clúster Kind que ejecuta OpenMetadata, sus dependencias y el PostgreSQL de referencia. Elaboración propia.	30
5.4	Flujo operativo del caso de uso de validación, paso a paso. Elaboración propia.	30
5.5	Arquitectura <i>medallion</i> : capas bronze, silver y gold de refinamiento progresivo del dato. Adaptado de la documentación de Databricks.	31
5.6	Mapeo activo entre clases DCAT-AP-ES y elementos de la plataforma. Elaboración propia.	34
5.7	Pipeline de metadatos desde OpenMetadata hasta la validación SHACL. Elaboración propia.	35
5.8	Validación externa del catálogo en el <i>SEMIC SHACL Validator</i> (ITB, Comisión Europea): perfil DCAT-AP HVD 3.0.0 Base Zero, resultado SUCCESS sin errores ni advertencias. Elaboración propia.	42
F.1	Pantalla inicial de la consola web y <i>sidebar</i> con las secciones operativas. Elaboración propia.	68
F.2	Pantalla <i>Infraestructura</i> : comprobación de prerequisites y opciones de reset limpio. Elaboración propia.	69
F.3	Pantalla <i>Ingesta</i> : servicios PostgreSQL y datasets gold descubiertos por OpenMetadata. Elaboración propia.	69

F.4	Pantalla <i>Gobierno</i> : edición de la hoja <code>gold_governance.csv</code> con listas controladas para temática y categoría High Value Datasets (Conjuntos de Datos de Alto Valor) (HVD). Elaboración propia.	70
F.5	Pantalla <i>Workflow</i> : dry-run y aplicación con resumen de cambios. Elaboración propia.	71
F.6	Pantalla <i>DCAT</i> : exportación del catálogo JavaScript Object Notation for Linked Data (JSON-LD) y previsualización. Elaboración propia.	72
F.7	Pantalla <i>Validación</i> : ejecución de la suite reproducible y resumen consolidado. Elaboración propia.	72
F.8	Pantalla <i>SHACL</i> : manifiesto de las <i>shapes</i> DCAT-AP-ES congeladas (sin descarga en tiempo de ejecución). Elaboración propia.	73
F.9	Pantalla <i>Artefactos</i> : listado de evidencias con vista previa (<i>Ver</i>) y apertura nativa (<i>Abrir</i>) para HTML renderizado y PDF en el visor del navegador. Elaboración propia.	73
F.10	Pantalla <i>Ejecuciones</i> : historial de <i>jobs</i> con estado, duración, código de salida y acceso al log y artefactos. Elaboración propia.	74
G.1	Árbol canónico del repositorio del TFM. Elaboración propia.	78
G.2	Diagrama de Gantt del roadmap del TFM, derivado del archivo declarativo <code>scripts/planning/github_project_planificacion.json</code> . Elaboración propia.	78
G.3	Despliegue Kubernetes reproducible usado como base del caso de uso de validación. Elaboración propia.	79
G.4	Contenedores Docker que materializan el clúster Kind y los nodos auxiliares. Elaboración propia.	79
G.5	Pods desplegados en el clúster Kind: OpenMetadata, MySQL, OpenSearch y PostgreSQL de referencia. Elaboración propia.	80
G.6	Esquemas <i>bronze</i> , <i>silver</i> y <i>gold</i> del PostgreSQL de referencia, vistos desde la User Interface (UI) de OpenMetadata (OM). Elaboración propia. . . .	80
G.7	Servicios de base de datos ingeridos en OpenMetadata tras la doble ingesta. Elaboración propia.	81
G.8	Propiedades personalizadas DCAT y etiquetas <code>dcat_theme.*</code> aplicadas a una tabla <i>gold</i> en OpenMetadata tras ejecutar el workflow. Elaboración propia. .	81
G.9	Estructura interna del paquete <code>om_dcat_sync</code> . Elaboración propia.	82
G.10	Pantalla <i>Gobierno</i> de la consola web: edición de la hoja <code>gold_governance.csv</code> . Elaboración propia.	83
G.11	Pantalla <i>Workflow</i> : lanzamiento de dry-run y apply con resumen de cambios. Elaboración propia.	83
G.12	Pantalla <i>DCAT</i> : exportación y previsualización del catálogo JSON-LD. Elaboración propia.	84
G.13	Pantalla <i>Validación</i> : ejecución de la suite reproducible y resumen de resultados. Elaboración propia.	84
G.14	Informe de validación HTML renderizado en el navegador desde la consola web. Elaboración propia.	85

Índice de tablas

3.1	Comparativa de capacidades de la Plataforma del TFM frente a plataformas comerciales de gobierno (p. ej. Anjana Data), plataformas open source de metadatos (OpenMetadata, DataHub) y portales de datos abiertos (CKAN). Elaboración propia.	17
4.1	Fases del roadmap del TFM con su área DAMA-DMBOK predominante. Elaboración propia, sincronizada con <code>scripts/planning/github_project_planificacion.json</code>	21
4.2	Bloques principales del repositorio. Elaboración propia.	23
5.1	Requisitos funcionales con criterio de aceptación. Elaboración propia. . . .	27
5.2	Módulos principales de <code>tfm_ingestor</code> . Elaboración propia.	37
5.3	Resumen de resultados del caso de uso de validación. Elaboración propia a partir de la documentación del repositorio.	43
5.4	Trazabilidad entre objetivos, bloques de la memoria y evidencias. Elaboración propia.	46
6.1	Cumplimiento de los objetivos del TFE y bloque donde se evidencian. Elaboración propia.	48
B.1	Matriz de versiones fijadas del sistema. Elaboración propia.	58
D.1	Correspondencia mínima OpenMetadata → DCAT-AP-ES. Elaboración propia a partir de <code>docs/dcat_mapping.md</code>	63
D.2	Entregables del caso de uso de validación y su visualización en la memoria. Elaboración propia.	64
F.1	Mapa del recorrido demo: acción del operario, artefacto producido y sección de la memoria donde se justifica formalmente. Elaboración propia.	75
G.1	Columnas de la hoja funcional <code>gold_governance.csv</code> . Elaboración propia.	89

Índice de Códigos

3.1	Dataset DCAT serializado en JSON-LD usando @context para acortar las IRIs.	12
5.1	Fragmento de <code>sql/opendata_demo_init.sql</code> : tabla publicable de la capa gold.	36
5.2	Workflow canónico end-to-end: sincroniza gobierno, exporta DCAT-AP-ES y valida SHACL HVD.	41
A.1	Instalación de dependencias Python del núcleo <code>om_dcat_sync</code> .	55
A.2	Instalación de dependencias de la consola web con <code>pnpm</code> .	55
A.3	Lanzamiento de Kind, Helm e ingesta doble.	55
A.4	Ejecución del workflow canónico end-to-end.	56
A.5	Suite reproducible documentada.	56
E.1	Validación independiente y por suite.	65
E.2	Pruebas Vitest de la consola web.	65
F.1	Comprobación rápida del entorno antes de iniciar la demostración.	67
F.2	Envío del catálogo a un harvester CKAN con extensión DCAT.	75
G.1	Configuración inmutable del workflow canónico en <code>workflow_service.py</code> .	85
G.2	Esqueleto de <code>run_workflow</code> en <code>workflow_service.py</code> .	86
G.3	Contexto JSON-LD canónico generado por <code>dcat_export.py</code> .	86
G.4	Fragmento de <code>k8s/openmetadata.values.yaml</code> : configuración mínima del release Helm.	87
G.5	Perfil operativo <code>operational_profile.yaml</code> que enlaza defaults, reglas, hoja y caso de validación.	87
G.6	Estructura de un <i>job</i> persistido en <code>state/web_jobs/</code> .	87
G.7	Tripletas RDF que describen un dataset DCAT en sintaxis Turtle (mismo grafo que el listado 3.1).	88
G.8	Fragmento ilustrativo de SHACL que exige título y descripción a cada Dataset DCAT.	88

Índice de Acrónimos

DAMA	Data Management Association International
DMBOK	Data Management Body of Knowledge
HVD	High Value Datasets (Conjuntos de Datos de Alto Valor)
FQN	Fully Qualified Name (identificador completo de activo en OpenMetadata)
CSV	Comma-Separated Values
RDF	Resource Description Framework
RDFS	Resource Description Framework Schema
OWL	Web Ontology Language
SPARQL	SPARQL Protocol and RDF Query Language
IRI	Internationalized Resource Identifier
URI	Uniform Resource Identifier
URL	Uniform Resource Locator
JSON	JavaScript Object Notation
JSON-LD	JavaScript Object Notation for Linked Data
XML	eXtensible Markup Language
TTL	Turtle (Terse RDF Triple Language)
DCAT	Data Catalog Vocabulary (W3C)
DCAT-AP	DCAT Application Profile for European Data Portals
DCAT-AP-ES	Perfil de aplicación DCAT-AP para España
SKOS	Simple Knowledge Organization System
NTI-RISP	Norma Técnica de Interoperabilidad de Reutilización de Recursos de Información del Sector Público
SHACL	Shapes Constraint Language
W3C	World Wide Web Consortium

API	Application Programming Interface
REST	Representational State Transfer
MIME	Multipurpose Internet Mail Extensions
JWT	JSON Web Token
SQL	Structured Query Language
YAML	YAML Ain't Markup Language
CI/CD	Continuous Integration / Continuous Delivery
RBAC	Role-Based Access Control
OM	OpenMetadata
CLI	Command Line Interface
UI	User Interface
PMBOK	Project Management Body of Knowledge

1. Introducción

1.1. Contexto

El dato solo genera valor cuando puede localizarse, entenderse y reutilizarse, también más allá de la organización que lo produce. En la práctica, sin embargo, los activos de información suelen permanecer dispersos en silos técnicos y descritos de forma heterogénea, lo que dificulta compartirlos. La estrategia europea de datos responde a esa problemática impulsando los *espacios de datos* (*data spaces*): entornos en los que las administraciones y organizaciones comparten datos de forma gobernada e interoperable sin perder el control sobre ellos [1]. Esa interoperabilidad no se consigue moviendo datos, sino describiéndolos con metadatos y vocabularios comunes. Por eso los portales de datos abiertos europeos y nacionales (data.europa.eu, datos.gob.es) se apoyan en perfiles estándar como DCAT-AP-ES [2], y la apertura de los conjuntos de datos de alto valor se regula a escala europea [3]. El reto, por tanto, no es almacenar más datos, sino producir catálogos descritos de forma homogénea que cualquier portal pueda agregar y reutilizar.

En ese contexto, este Trabajo Fin de Máster se enmarca en el diseño y despliegue de una configuración de metadatos en OpenMetadata alineada con DCAT-AP [4], con el objetivo de mejorar la interoperabilidad de catálogos de datos en entornos Big Data y de computación en la nube. La descripción, los objetivos y el alcance oficiales propuestos para este Trabajo Fin de Estudios (TFE) en la universidad definen explícitamente el análisis de DCAT-AP, su correspondencia con OpenMetadata, la configuración de taxonomías y metadatos personalizados, y la validación mediante exportación en RDF [5], JSON-LD [6] o formato equivalente.

El trabajo se concreta en una **Plataforma de Gobierno del Dato**. La plataforma no procesa datos de negocio con finalidad analítica, sino que gobierna metadatos: identifica activos técnicos, completa metadatos funcionales, genera un catálogo interoperable y valida su conformidad. Por catálogo interoperable se entiende, en este trabajo, un catálogo descrito con vocabularios y perfiles estándar, en particular el perfil DCAT-AP-ES, de modo que portales

de datos abiertos nacionales y europeos como `datos.gob.es` o `data.europa.eu` puedan entenderlo, agregarlo y reutilizarlo sin acuerdos previos punto a punto entre sistemas. El valor técnico del proyecto no reside en calcular métricas sobre las tablas de ejemplo, sino en demostrar un ciclo reproducible de gobierno e interoperabilidad de metadatos.

En el ámbito de los datos abiertos, DCAT [7] permite describir catálogos, conjuntos de datos, distribuciones y servicios de datos mediante RDF [5], lo que facilita el intercambio de metadatos entre sistemas. En España, DCAT-AP-ES [2] concreta este enfoque para el contexto nacional. Toma como referencia DCAT-AP [4] 2.1.1 e incorpora elementos de DCAT-AP HVD 2.2.0. Los conjuntos de datos de alto valor (HVD, del inglés *High-Value Datasets*) son las categorías de datos del sector público cuya reutilización conlleva un beneficio socioeconómico especialmente alto y cuya publicación obliga el Reglamento de Ejecución (UE) 2023/138 [3]. Sobre ellos, DCAT-AP HVD añade restricciones de descripción adicionales, como la categoría, la legislación aplicable o el servicio de acceso, para publicarlos de forma homogénea entre Estados miembros.

1.2. Problema abordado

El problema que aborda este trabajo no se limita a una carencia puntual de OpenMetadata. Es el problema, más general, de publicar catálogos de datos interoperables a partir de una solución basada en metadatos. Una organización puede disponer de un inventario técnico completo de sus activos en OpenMetadata, pero ese inventario, por sí solo, no constituye todavía un catálogo que portales externos puedan entender, agregar y reutilizar. OpenMetadata proporciona capacidades de catálogo, descubrimiento y gobierno de metadatos, pero no genera de forma nativa, en el repositorio de este TFM, un catálogo DCAT-AP-ES completo y validado a partir de esos activos técnicos gobernados. El problema consiste, por tanto, en cerrar la separación entre dos mundos:

- por un lado, el catálogo técnico gestionado en OpenMetadata, con servicios, bases de datos, esquemas, tablas, columnas, descripciones, etiquetas y metadatos personalizados;
- por otro lado, el modelo DCAT-AP-ES, que espera un grafo RDF con clases y propiedades específicas para catálogos, datasets, distribuciones, servicios de datos y agentes.

La dificultad no es únicamente transformar nombres de campos. También hay que decidir qué activos se consideran publicables, qué metadatos cura a mano una persona responsable del catálogo y cuáles se derivan por configuración (valores comunes que se declaran una sola vez y el sistema aplica a todos los datasets), y cómo validar que la salida cumple un perfil semántico externo. Los fundamentos técnicos que sostienen ese problema, como

RDF, JSON-LD, los perfiles DCAT y la validación SHACL, se presentan en el estado del arte. En conjunto, el trabajo combina infraestructura, integración con API, modelado de metadatos, exportación semántica, validación y una consola web operativa.

1.3. Solución propuesta

La solución implementada parte de un PostgreSQL de referencia desplegado junto a OpenMetadata. Ese PostgreSQL organiza los datos en tres capas según el patrón *medallion*, de menor a mayor refinamiento: *bronze*, *silver* y *gold* (el patrón y su uso aquí se detallan en la sección 5.1.7.2). Para el caso de uso de validación se realiza una doble ingesta de la misma fuente bajo dos servicios de OpenMetadata, *postgres_demo_service* y *postgres_validation_service*, lo que muestra gobierno multi-fuente sin introducir una segunda base de datos artificial. La figura 1.1 resume esa visión funcional de conjunto, de la fuente técnica al catálogo interoperable validado.

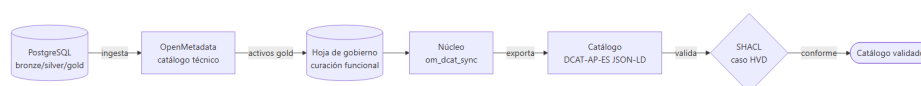


Figura 1.1: Visión funcional de la Plataforma de Gobierno del Dato: de la fuente técnica al catálogo interoperable validado. Elaboración propia.

OpenMetadata ingiere los activos técnicos de ambos servicios (servicios, esquemas, tablas y columnas), por ejemplo las tablas *gold.agenda_cultural_publica* y *gold.movilidad_resumen_municipio*. La plataforma gobierna únicamente las tablas *gold*, que son los datasets publicables. Las capas *bronze* y *silver* permanecen en el catálogo técnico de OpenMetadata pero quedan fuera del catálogo de publicación. La distinción entre catálogo técnico y catálogo de publicación, y la justificación de esa frontera, se desarrollan en el estado del arte y en la sección 5.1.7.2.

El número de datasets publicables es una consecuencia del alcance elegido y no una cifra arbitraria. Se publica la capa *gold* (no *bronze* ni *silver*) y, dentro de ella, las dos tablas de negocio definidas en la fuente de referencia. Al ingerirse esa misma fuente bajo dos servicios para demostrar el gobierno multi-fuente, resultan cuatro datasets (2 tablas por 2 servicios). Podrían ser más, añadiendo filas *gold* a la hoja de gobierno, o menos, marcando una tabla como no publicable. Cuatro es el número mínimo que permite ejercitar a la vez la publicación selectiva por capa y la diferenciación por servicio. La hoja *gold_governance.csv* identifica cada fila por su *table_fqn* (nombre completo del activo en OpenMetadata), de modo que dos tablas con el mismo nombre funcional pero distinto servicio no se mezclan.

El núcleo de la solución es el paquete Python *om_dcat_sync*, conservado internamente como *tfm_ingestor*. Este componente descubre los activos técnicos en OpenMetadata

y refresca una hoja funcional de gobierno: un archivo CSV versionado en el que cada fila corresponde a un dataset publicable y cada columna a un metadato de negocio editable (título, descripción, publicador, temática, categoría HVD y URL de acceso). Esa hoja es el punto único donde una persona responsable cura esos metadatos. A partir de ella, el núcleo aplica los metadatos sobre OpenMetadata de forma idempotente, es decir, repetible: volver a ejecutarlo no duplica ni altera lo ya aplicado. Después exporta el catálogo en JSON-LD y valida la salida con SHACL. El porqué de ambos formatos, y qué es SHACL, se detallan en el estado del arte. La consola web situada en web/ no duplica la lógica: ejecuta los mismos scripts y comandos versionados desde una interfaz operativa. La figura 1.2 detalla el flujo lógico por capas y componentes.

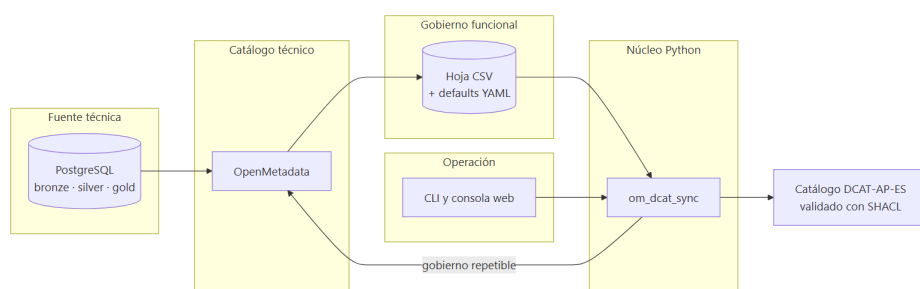


Figura 1.2: Flujo lógico por capas y componentes de la Plataforma de Gobierno del Dato. Elaboración propia.

1.4. Aportación del trabajo

La aportación principal es una capa reproducible de gobierno e interoperabilidad construida sobre OpenMetadata, cuyo fin es convertir un inventario técnico de metadatos en un catálogo DCAT-AP-ES publicable y formalmente validado, de forma repetible, trazable y operable también por perfiles no desarrolladores. En otras palabras, aporta el puente que falta entre lo que OpenMetadata descubre internamente y lo que un portal de datos abiertos puede reutilizar. Sobre esa base, el trabajo aporta:

- un modelo operativo que separa los activos técnicos ingeridos de los datasets funcionalmente publicables;
- una doble ingesta PostgreSQL que valida el gobierno multi-fuente, entendido como la capacidad de aplicar las mismas reglas de gobierno a activos de varios servicios u orígenes, identificándolos sin ambigüedad por su nombre completo (FQN) aunque compartan el nombre de tabla;
- una hoja de gobierno para curar título, descripción, publicador, temática, categoría

HVD y URL de acceso por dataset;

- un exportador a JSON-LD que materializa las clases activas del perfil DCAT-AP-ES (catálogo, conjunto de datos, distribución, servicio de datos y agente publicador), detalladas en el estado del arte;
- una validación reproducible mediante SHACL, con shapes DCAT-AP-ES locales y congeladas;
- una consola web que permite ejecutar el caso de uso de validación, revisar logs y consultar artefactos sin introducir comandos arbitrarios.

El despliegue reproducible de OpenMetadata y PostgreSQL mediante Kubernetes, Helm y scripts versionados no se contabiliza como aportación al objetivo del proyecto, sino como el soporte de infraestructura que hace posible y repetible todo lo anterior.

El valor del trabajo se aprecia mejor situándolo entre los roles que intervienen en un proyecto de datos. La infraestructura la preparan perfiles de sistemas o de ingeniería de datos. La propiedad y la responsabilidad editorial del dato corresponden a su propietario y a su *steward* o custodio. El catálogo publicado lo consumen terceros, como portales de datos abiertos, analistas u otras organizaciones. El TFM no sustituye a ninguno de esos roles. Lo que aporta es, para la persona que gobierna el catálogo (el *steward* u operario responsable), un único punto centralizado y reproducible desde el que curar los metadatos y publicar un catálogo conforme, operable indistintamente desde la hoja CSV o desde la consola web, sin tener que tocar la infraestructura ni el código. En eso reside su valor: convertir un conjunto de componentes técnicos heterogéneos en una operación de gobierno sencilla, trazable y repetible para ese responsable.

1.5. Alcance

El alcance del trabajo lo definen los objetivos general y específicos del capítulo siguiente: gobernar metadatos técnicos y funcionales sobre OpenMetadata y exportarlos como un catálogo DCAT-AP-ES validable. El caso de uso de validación no delimita ese alcance, sino que es el medio para comprobar que lo construido funciona y cubre la necesidad planteada. La plataforma gobierna metadatos generados a partir de un PostgreSQL de referencia, no datos de negocio reales. Sus tablas se organizan en capas *bronze*, *silver* y *gold*, y solo *gold* se considera publicable.

La implementación activa cubre:

- despliegue local reproducible con Kubernetes, Helm y scripts PowerShell;
- doble ingesta técnica de PostgreSQL en OpenMetadata bajo los servicios `postgres_demo_service` y `postgres_validation_service`;

-
- gobierno funcional de las tablas `gold` de ambos servicios, con cuatro datasets publicables;
 - custom properties y etiquetas DCAT en OpenMetadata;
 - exportación JSON-LD;
 - validación SHACL con el caso HVD;
 - pruebas automatizadas y validación estructural;
 - consola web operativa basada en Next.js.

1.6. Exclusiones

Quedan fuera del alcance:

- publicación automática en un portal externo CKAN o datos.gob.es;
- certificación jurídica de datasets de alto valor fuera del caso de uso de validación;
- calidad fila a fila de los datos de negocio;
- alta disponibilidad, SSO, RBAC avanzado, cifrado avanzado, backups productivos y observabilidad avanzada;
- flujos editoriales complejos propios de plataformas comerciales completas.

1.7. Estructura del documento

Siguiendo la estructura normativa del Trabajo Fin de Máster, esta memoria se organiza en seis bloques. La **introducción** (este capítulo) sitúa el contexto, el problema, la solución propuesta, sus aportaciones y el alcance y las exclusiones del trabajo. El bloque de **objetivos** fija el objetivo general y los objetivos específicos. El **estado del arte** presenta el marco técnico (gobierno del dato, catálogos de metadatos, RDF y JSON-LD, DCAT-AP-ES, SHACL y OpenMetadata) y compara las herramientas del mercado alineadas con DCAT-AP-ES. La **metodología de trabajo** describe el enfoque de gestión del proyecto (Kanban y PMBOK, tablero de seguimiento y fases) y el stack tecnológico empleado. Los **resultados** recogen la arquitectura de la solución, su implementación y la validación realizada, junto con las evidencias generadas. Por último, las **conclusiones y trabajos futuros** revisan el cumplimiento de los objetivos y plantean las líneas de evolución.

2. Objetivos

2.1. Objetivo general

El objetivo general del TFM, según los objetivos oficiales propuestos para el TFE en la universidad, es **diseñar y desplegar una configuración de metadatos en OpenMetadata alineada con el estándar DCAT-AP**, que permita **mejorar la interoperabilidad y la gestión semántica de catálogos de datos** en entornos Big Data y de computación en la nube. En este repositorio, ese perfil DCAT-AP se concreta en su aplicación nacional DCAT-AP-ES.

En términos implementados, este objetivo se materializa en una Plataforma de Gobierno del Dato capaz de descubrir activos técnicos, enriquecerlos con metadatos funcionales, exportar un catálogo JSON-LD y validar su conformidad mediante SHACL. La validación se apoya en dos servicios PostgreSQL ingeridos en OpenMetadata, lo que permite demostrar que el gobierno no depende de una única fuente ni de nombres de tabla globalmente únicos.

2.2. Objetivos específicos

Los objetivos específicos se formulan de forma verificable:

1. Analizar DCAT-AP y su concreción DCAT-AP-ES, incluyendo clases, propiedades, vocabularios controlados y validación SHACL.
2. Mapear las entidades de OpenMetadata con las clases principales de DCAT-AP-ES.
3. Configurar en OpenMetadata los metadatos mínimos necesarios para representar datasets publicables.
4. Implementar un workflow reproducible de sincronización de metadatos desde activos técnicos reales.

-
5. Ejecutar doble ingesta PostgreSQL para validar gobierno multi-fuente con tablas homónimas.
 6. Exportar el catálogo gobernado en JSON-LD compatible con DCAT-AP-ES.
 7. Validar la salida con SHACL y evaluar beneficios, limitaciones, idempotencia y mantenibilidad.
 8. Proporcionar una consola web operativa que ejecute el mismo flujo sin duplicar reglas de negocio.

2.3. Cambio de alcance respecto a CKAN

La descripción oficial del TFE planteaba implementar un pipeline de ingesta o sincronización de metadatos (*harvesting*) desde una fuente externa, y proponía CKAN como ejemplo. Durante el desarrollo se optó por no emplear CKAN como origen operativo principal. La razón es de fondo: CKAN es ante todo un catálogo de publicación e intercambio de metadatos ya elaborados, de modo que cosechar metadatos de un catálogo externo no demuestra la parte central del trabajo, que es generar, gobernar y validar metadatos a partir de activos técnicos reproducibles. La formulación literal de la descripción no resultaba, por tanto, del todo apropiada para el objetivo de fondo del TFM. El cambio de origen queda suficientemente justificado en términos de funcionalidad y de objetivos, porque permite demostrar el ciclo completo de gobierno que el trabajo persigue sin alterar lo esencial de lo planteado: alinear OpenMetadata con DCAT-AP y validar la interoperabilidad del catálogo resultante.

Conviene además separar dos conceptos que no son equivalentes. La *gestión del dato* (*data management*) abarca las funciones operativas que producen y mantienen los datos (arquitectura, modelado, almacenamiento, integración o calidad). El *gobierno del dato* (*data governance*) es, en cambio, la función de supervisión que define políticas, roles, responsabilidades y controles sobre esas funciones; en el marco DAMA-DMBOK es el núcleo que coordina y da autoridad al resto de la gestión, no una tarea operativa más [8, 9]. Este TFM se sitúa en la función de gobierno (decidir qué se publica, con qué metadatos y bajo qué reglas), apoyándose en funciones de gestión como la catalogación y la integración sin pretender cubrirlas todas. CKAN se mantiene como posible destino o punto de federación futura, no como fuente canónica del caso de uso de validación.

3. Estado del arte

Este capítulo explica primero los conceptos y tecnologías que forman el estado del arte utilizado por el TFM. La implementación propia se reserva para capítulos posteriores. La separación es deliberada: antes de justificar cómo se ha construido la plataforma conviene entender qué problema resuelven el gobierno del dato, los catálogos de metadatos, DCAT-APES, SHACL y OpenMetadata en el contexto actual. El stack tecnológico concreto (PostgreSQL, Docker, Kubernetes, Helm y la consola web) se justifica en el capítulo de metodología.

3.1. Gobierno del dato y gestión de metadatos

El gobierno del dato es el conjunto de responsabilidades, políticas, procesos, roles y controles que permite gestionar los datos como activos de una organización. En la práctica actual no se limita a almacenar información: define quién es responsable de cada activo, qué significado tiene, bajo qué reglas se publica, qué calidad mínima debe tener, cómo se traza su origen y cómo se comparte con terceros. Data Management Association International (DAMA)-Data Management Body of Knowledge (DMBOK) se utiliza habitualmente como marco de referencia para ordenar estas funciones de gestión de datos [8]. Ese marco distingue roles con responsabilidades distintas: el *propietario* (*owner*) del dato, que responde de él; el *steward* o custodio, que lo describe, lo cura y vela por su calidad; los perfiles técnicos de sistemas e ingeniería de datos, que operan la infraestructura y los pipelines; y los consumidores, que reutilizan el catálogo publicado. La Plataforma del TFM se dirige sobre todo al rol de *steward*, al que ofrece un punto único de gobierno. En el ámbito español de datos abiertos, datos.gob.es relaciona estas prácticas con principios como accesibilidad, consistencia, auditabilidad, exactitud y seguridad [9].

DMBOK agrupa once áreas funcionales: arquitectura de datos, modelado y diseño, almacenamiento y operaciones, seguridad, integración e interoperabilidad, documentación y contenido, datos maestros y de referencia, almacenamiento analítico y *business intelligence*,

metadatos, calidad y gobierno [8]. La Plataforma del TFM se sitúa de forma intencionada en la intersección de tres áreas: **metadatos** (descripción semántica de activos), **integración e interoperabilidad** (exportación a un perfil externo) y **gobierno** (políticas sobre qué se publica y cómo). El resto de áreas no se aborda porque corresponden a programas de gestión de datos de mayor envergadura, con equipos dedicados y objetivos transversales.

La gestión de metadatos es una pieza central de ese gobierno: los metadatos describen los datos y permiten entender qué existe, dónde se encuentra, quién lo mantiene y cómo puede reutilizarse. DMBOK distingue cuatro tipos: *de negocio* (significado, glosario, responsable), *técnicos* (esquema, tipo, claves), *operacionales* (frecuencia, latencia) y *de gobierno* (clasificación, sensibilidad, calidad) [8]. La Plataforma del TFM gobierna principalmente metadatos técnicos descubiertos por OM y metadatos de negocio aportados por la hoja funcional, y deja el resto fuera del alcance por no ser esenciales para demostrar conformidad Perfil de aplicación DCAT-AP para España (DCAT-AP-ES).

La Plataforma de Gobierno del Dato del TFM adopta esa visión y la acota deliberadamente a un perímetro reproducible. No persigue medir la calidad de cada valor almacenado en tablas ni sustituir una suite corporativa completa, decisiones que pertenecen a iniciativas con alcance, presupuesto y ciclo de vida distintos. Su objetivo es entregar un *ciclo de gobierno de metadatos* defendible y verificable: descubrir activos técnicos, enriquecerlos con metadatos funcionales, publicar una representación interoperable y validar formalmente que esa representación cumple un perfil externo, por ejemplo DCAT-AP-ES o, en su variante más exigente, DCAT-AP HVD.

3.2. Catálogos de datos y metadatos técnicos

Un catálogo de datos organiza descripciones de activos para que puedan encontrarse, entenderse y reutilizarse. Conviene matizar un punto importante: cuando un activo llega a un catálogo, y muy especialmente a un catálogo de publicación, se asume que ya ha sido gobernado. El catálogo no es el lugar donde empieza el gobierno, sino donde se hace visible y consultable su resultado. El gobierno (responsable, significado, reglas de publicación y calidad mínima) es condición previa, y el catálogo lo materializa y lo expone. En el estado actual de la ingeniería de datos, los catálogos suelen integrar conectores de ingesta, modelos de entidades, búsqueda, etiquetas, glosarios, owners, descripciones, linaje y APIs. La utilidad principal es que los equipos de datos dejen de depender de conocimiento informal y puedan consultar un inventario vivo.

La diferencia entre datos y metadatos es clave. Una tabla PostgreSQL contiene filas y columnas con datos. OpenMetadata registra que existe esa tabla, su nombre, esquema, columnas, descripción, etiquetas y propiedades adicionales. DCAT-AP-ES, por su parte, no espera la tabla como tal, sino una descripción semántica del dataset publicable. Por eso el

problema de este TFM no es mover datos de negocio, sino transformar metadatos técnicos y funcionales en un grafo de catálogo.

También es importante distinguir, como ya se anticipó en la introducción, el catálogo técnico del catálogo de publicación. Un catálogo técnico ayuda a operar internamente los activos. Un catálogo de publicación, como los usados para datos abiertos, exige metadatos comprensibles, vocabularios controlados y formatos interoperables. La Plataforma de Gobierno del Dato une ambos niveles: parte de activos técnicos en OpenMetadata y produce una salida DCAT-AP-ES validable.

3.3. RDF, JSON-LD y vocabularios semánticos

3.3.1. Qué es RDF y por qué se utiliza

Resource Description Framework (RDF) (*Resource Description Framework*) es la recomendación del World Wide Web Consortium (W3C) para representar información en la Web como un grafo dirigido y etiquetado [5]. La unidad mínima de RDF es la *tripleta*: un enunciado con sujeto, predicado y objeto. El sujeto identifica un recurso, el predicado nombra una relación o propiedad y el objeto puede ser otro recurso o un valor literal con tipo (cadena, fecha, número, etc.). Al encadenar muchas tripletas se obtiene un grafo en el que cualquier sistema puede navegar relaciones sin conocer el esquema interno del sistema fuente.

La ventaja decisiva de RDF respecto a otros modelos de descripción es el uso de Internationalized Resource Identifiers (IRIs)/Uniform Resource Identifiers (URIs) como identificadores estables. Si dos catálogos diferentes utilizan la misma IRI de la clase *Dataset* del vocabulario Data Catalog Vocabulary (W3C) (DCAT) (`dcat:Dataset`, expandida a la IRI oficial publicada por el W3C), ambos están hablando de la misma entidad aunque sus bases de datos internas sean distintas. Esa propiedad permite que organizaciones independientes describan datos de forma interoperable y que portales agregadores combinen catálogos heterogéneos.

RDF no impone una sintaxis concreta. Las mismas tripletas pueden serializarse en distintos formatos según el caso de uso: eXtensible Markup Language (XML)-RDF para herramientas históricas, Turtle (Turtle (Terse RDF Triple Language) (TTL)) para legibilidad humana, N-Triples para flujos línea a línea o JSON-LD para integración con tecnologías web. La elección del formato no cambia el grafo subyacente.

Sobre RDF se construyen capas habituales en gobierno de datos: Resource Description Framework Schema (RDFS) declara clases, jerarquías y dominio o rango de las propiedades, Web Ontology Language (OWL) añade lógica descriptiva para restricciones y equivalencias, Simple Knowledge Organization System (SKOS) estandariza vocabularios controlados y taxonomías (temas, categorías o licencias) y SPARQL Protocol and RDF Query Language (SPARQL)

es el lenguaje de consulta sobre grafos RDF, equivalente conceptual a Structured Query Language (SQL).

3.3.2. JSON-LD como serialización elegida

JSON-LD es una serialización de RDF basada en JavaScript Object Notation (JSON) [6]. Conserva las tripletas, pero las expresa con la sintaxis que ya manejan los entornos web actuales. Tres elementos lo distinguen: el `@context`, que asocia nombres cortos con IRIs completas; el `@id`, que identifica unívocamente cada recurso; y el `@type`, que indica la clase RDF a la que pertenece.

El listado 3.1 muestra un dataset DCAT serializado en JSON-LD. El mismo grafo en sintaxis Turtle, más compacta y legible para humanos, se recoge en el listado G.7 del Anexo G.

```
1 {
2   "@context": {
3     "dcat": "http://www.w3.org/ns/dcat#",
4     "dct": "http://purl.org/dc/terms/",
5     "ex": "https://www.uclm.es/datos/plataforma-gobierno-dato/"
6   },
7   "@id": "ex:agenda-cultural-publica",
8   "@type": "dcat:Dataset",
9   "dct:title": { "@value": "Agenda cultural pública - servicio operativo",
10    ↪ "@language": "es" },
11   "dct:description": { "@value": "Relación de eventos culturales públicos.", "
12    ↪ "@language": "es" },
13   "dct:publisher": { "@id": "ex:UCLM" },
14   "dcat:theme": { "@id": "http://datos.gob.es/kos/sector-publico/sector/
15    ↪ cultura-ocio" }
```

Código 3.1: Dataset DCAT serializado en JSON-LD usando `@context` para acortar las IRIs.

En este TFM, JSON-LD es el formato canónico del catálogo exportado por tres razones prácticas: es legible en el repositorio sin herramientas RDF especializadas, se versiona en Git como texto plano y se diferencia limpiamente entre ejecuciones, y es entrada directa para los validadores Shapes Constraint Language (SHACL) y los portales que aceptan DCAT-AP-ES. La interoperabilidad real, en todo caso, no proviene de usar JSON, sino de usar vocabularios y perfiles compartidos, idea que se retoma en la síntesis del estado del arte (sección 3.11). Por eso la memoria se apoya en DCAT, DCAT Application Profile for European Data Portals (DCAT-AP) y DCAT-AP-ES.

3.4. DCAT, DCAT-AP y DCAT-AP-ES

DCAT es el vocabulario del W3C para describir catálogos de datos y facilitar el intercambio de metadatos entre sistemas [7]. Su modelo incluye clases como `dcat:Catalog`, `dcat:Dataset`, `dcat:Distribution` y `dcat:DataService`. La idea principal es separar el recurso conceptual, sus formas de acceso y los servicios que lo hacen disponible.

DCAT-AP adapta DCAT al contexto europeo de portales de datos. Su objetivo es que los catálogos del sector público puedan intercambiar descripciones de datasets de manera homogénea y facilitar agregación por portales nacionales o europeos [4]. DCAT-AP no sustituye a DCAT: lo perfila, selecciona propiedades, cardinalidades y recomendaciones para un ámbito concreto.

DCAT-AP-ES concreta ese perfil para España. La documentación oficial de DCAT-AP-ES 1.0.0 indica que adopta DCAT-AP 2.1.1 e incorpora elementos de DCAT-AP HVD 2.2.0 para describir conjuntos de datos de alto valor [2]. Además, define relaciones, vocabularios controlados, convenciones y material de validación. Esta concreción es relevante para el TFM porque el enunciado oficial habla de DCAT-AP, pero la implementación operativa necesita un perfil verificable en el contexto español.

3.5. Datos de alto valor y caso HVD

La regulación europea sobre conjuntos de datos de alto valor ha aumentado la importancia de describir correctamente categorías, licencias, servicios de acceso y legislación aplicable. En DCAT-AP-ES, el caso HVD incorpora restricciones adicionales sobre datasets, distribuciones y servicios de datos [10], por lo que obliga a modelar más que un título y una descripción: exige relacionar dataset, distribución, servicio, categoría, legislación, licencia y contacto. En esta memoria la clasificación HVD se interpreta como hipótesis de diseño del caso de uso de validación, no como una calificación jurídica automática de datos reales. Su papel es técnico: ejercitar el perfil más exigente y demostrar que el exportador construye una salida formalmente validable.

3.6. Validación SHACL

Apoyándose en los conceptos de RDF, JSON-LD y DCAT-AP-ES ya introducidos, esta sección describe el mecanismo de validación formal del catálogo. SHACL (*Shapes Constraint Language*) es un lenguaje del W3C para validar grafos RDF mediante *shapes* [11]. Una *shape* describe restricciones que deben cumplir los nodos de un grafo: clases esperadas, propiedades obligatorias, cardinalidades, tipos de valor, severidades o relaciones entre recursos.

En un catálogo DCAT-AP-ES, SHACL permite comprobar si el JSON-LD exportado contiene las piezas exigidas por el perfil.

La validación SHACL no garantiza que el dataset sea útil o editorialmente perfecto, sino que el grafo cumple restricciones formales: un catálogo puede pasar SHACL y seguir necesitando revisión humana de redacción, licencias o pertinencia. Aun así, aporta una base objetiva y reproducible para defender que el modelo cumple un contrato externo, y una *shape* versionada permite repetir la comprobación en local, en Continuous Integration / Continuous Delivery (CI/CD) o en la consola web. Por eso el TFM vendoriza las *shapes* oficiales y evita descargarlas en tiempo de ejecución.

A modo de ejemplo, el listado G.8 del Anexo G declara en SHACL que todo recurso de tipo `dcat:Dataset` debe tener al menos un título y una descripción. Si el grafo JSON-LD exportado infringe estas restricciones, el validador devuelve una violación con severidad `sh:Violation` que indica la `sh:focusNode`, la `sh:resultPath` y el mensaje correspondiente.

3.7. OpenMetadata

OM es una plataforma abierta de descubrimiento, observabilidad y gobierno de metadatos. Su documentación la presenta como una solución unificada con repositorio central de metadatos, conectores de ingesta, búsqueda, linaje, calidad, colaboración y gobierno [12]. Para este TFM, sus funciones más relevantes son el catálogo técnico, la Application Programming Interface (API) Representational State Transfer (REST), las entidades de base de datos y la posibilidad de añadir etiquetas y *custom properties*.

OM funciona como punto de captura y gobierno técnico. Una ingesta de PostgreSQL registra servicios, bases de datos, esquemas, tablas y columnas. Sobre esas entidades se pueden mantener descripciones, nombres visibles, dominios, etiquetas y propiedades extendidas. La plataforma resultante no inventa activos manualmente: parte de lo descubierto por OM.

3.7.1. Gobierno centralizado frente a edición dataset por dataset

Sobre OM existen dos formas razonables de hacer que un catálogo cumpla DCAT-AP-ES. La primera es la que se obtiene de fábrica: declarar el conjunto mínimo de propiedades DCAT-AP-ES como *custom properties* y editar cada dataset desde la interfaz de OM, dataset por dataset, hasta que el grafo exportado contenga todos los campos requeridos. Este flujo es válido para una decena de tablas, pero presenta tres limitaciones cuando aumenta el volumen: el trabajo manual escala linealmente con el número de datasets, las reglas globales (publicador, licencia, taxonomía, legislación HVD) deben repetirse en cada activo, y la

trazabilidad de quién editó qué se diluye en eventos de UI.

La segunda forma, propuesta operativa de este TFM, externaliza la curación funcional a una hoja de gobierno versionada que se reconcilia contra OM desde un flujo repetible. Una persona responsable edita un único archivo Comma-Separated Values (CSV) en el que cada fila es un dataset publicable, mientras que los valores comunes del catálogo (publicador, licencia, idioma, contacto, taxonomía y legislación HVD) viven como *defaults* en un YAML Ain't Markup Language (YAML), sobrescribibles por fila cuando un dataset concreto lo requiere. Un servicio Python fusiona ambas fuentes, sincroniza los cambios contra la API REST de OM, exporta el catálogo DCAT-AP-ES en JSON-LD y lo valida con SHACL. Así, el mismo estado de gobierno se aplica en una sola operación, los valores comunes no se duplican y la exportación es determinista, una propiedad que se deriva de la idempotencia del workflow (sección 3.9).

La propuesta no excluye usar las *custom properties* de OM: las requiere y las gobierna. La diferencia está en quién las gestiona. En el flujo dataset por dataset las gestiona un operador humano. En el flujo centralizado las gestiona el servicio Python a partir de una fuente declarativa, y OM se reserva como sistema de captura técnica, búsqueda y autoridad de identificadores.

3.8. Mercado de herramientas de gobierno alineadas con DCAT-AP-ES

La adopción reciente de DCAT-AP-ES ha llevado a los principales fabricantes de plataformas de gobierno de datos a anunciar soporte específico para este perfil. La oferta puede agruparse en tres bloques con propiedades distintas: plataformas comerciales de gobierno con módulo DCAT-AP-ES, soluciones de portal de datos abiertos y plataformas de metadatos open source.

Plataformas comerciales de gobierno con módulo DCAT-AP-ES. Productos como Anjana Data publican configuraciones alineadas con DCAT-AP-ES [13], ofreciendo flujos editoriales, aprobaciones, jerarquías de dominios y publicación gestionada hacia portales nacionales. El valor que aportan es de producto completo: tienen UI pulida, gestión de roles, multi-tenant, ciclo de vida de activos, soporte y certificaciones. Su contrapartida es el coste de licencia, la complejidad de despliegue y, en términos académicos, la opacidad: las reglas de mapeo y validación viven dentro del producto y no son reproducibles fuera de él.

Portales de datos abiertos. Sistemas como CKAN o las extensiones de portal nacional (datos.gob.es, data.europa.eu) describen el dataset en términos de DCAT-AP y DCAT-AP-ES des-

de el lado del consumidor: federan, agregan y publican. No están diseñados para gobernar metadatos técnicos desde la fuente, sino que asumen que los catálogos productores ya entregan descripciones conformes. Por eso este TFM descarta CKAN como flujo operativo principal y lo mantiene únicamente como destino federable.

Plataformas open source de metadatos. Soluciones como OpenMetadata [12], DataHub [14] o Amundsen [15] ofrecen catálogo técnico, ingesta y gestión de *custom properties*, pero no incluyen un exportador DCAT-AP-ES oficial certificado ni una validación SHACL contra las *shapes* HVD. Para cumplir el perfil hay que extenderlas con código propio, que es precisamente la aportación de este TFM sobre OM.

3.8.1. Posicionamiento de la Plataforma de Gobierno del Dato del TFM

La Plataforma de Gobierno del Dato no pretende competir con un producto comercial. Su valor se sitúa en una franja concreta: reproducibilidad total (todo el flujo es código versionado, sin componentes propietarios), conformidad verificable (validación SHACL con las *shapes* oficiales, incluido el perfil HVD), gobierno centralizado y declarativo (una hoja CSV y defaults YAML en lugar de la edición dataset por dataset de la sección 3.7.1) y operación cerrada desde una consola web. A cambio, acepta no cubrir flujos avanzados de aprobación, búsqueda editorial sofisticada, multi-tenant ni publicación automática hacia portales externos. Ofrece, eso sí, el mismo perfil DCAT-AP-ES que prometen las herramientas comerciales, construido con piezas estándar (Python, API REST, JSON-LD, SHACL) y reproducible en cualquier entorno con Kubernetes local.

La tabla 3.1 resume, a modo de *benchmark*, dónde aporta valor la propuesta frente a las tres familias de herramientas, evitando una comparación puramente narrativa.

Capacidad	TFM	Comercial	Open source	Portal abierto
Exportación DCAT-AP-ES	✓	✓	✗	Parcial
Validación SHACL HVD reproducible	✓	Parcial	✗	✗
Gobierno centralizado declarativo (CSV + YAML)	✓	Parcial	✗	✗
Reproducibilidad total sin componentes propietarios	✓	✗	✓	Parcial
Sin coste de licencia	✓	✗	✓	✓
Multi-tenant, roles y aprobaciones editoriales	✗	✓	Parcial	Parcial
Publicación/federación automática a portales	✗	✓	✗	✓

Tabla 3.1: Comparativa de capacidades de la Plataforma del TFM frente a plataformas comerciales de gobierno (p. ej. Anjana Data), plataformas open source de metadatos (OpenMetadata, DataHub) y portales de datos abiertos (CKAN). Elaboración propia.

3.9. Reproducibilidad, idempotencia e *Infrastructure as Code*

La reproducibilidad es un requisito transversal de la ingeniería de plataformas: un entorno debe poder reconstruirse de forma determinista a partir de su descripción declarativa, sin depender del estado mutable de la máquina que lo levantó [16]. La combinación de Docker, Kind, Helm y manifiestos versionados implementa este principio, al describir el clúster como datos (YAML) y no como pasos manuales. Su complemento operativo es la idempotencia: una operación es idempotente cuando aplicarla varias veces produce el mismo resultado que aplicarla una sola vez. En gobierno de metadatos es crítica, porque un workflow no idempotente duplicaría etiquetas o sobrescribiría campos curados. La plataforma se apoya en que los PATCH de la API REST de OM [12] son idempotentes para garantizar que la segunda ejecución no aplique cambios, propiedad verificada en la sección 5.3. Las pruebas completan el marco, con pytest [17] sobre el núcleo Python y Vitest [18] sobre la consola web.

3.10. Validación frente a verificación

La memoria distingue *verificación* y *validación*. La verificación comprueba que el sistema se construye correctamente según su especificación interna (tests unitarios, tipos consistentes, operaciones idempotentes). La validación comprueba que cumple un contrato externo: el catálogo exportado satisface las *shapes* SHACL oficiales de DCAT-AP-ES [10], con las pro-

piedades obligatorias del perfil HVD y los vocabularios temáticos de la taxonomía Norma Técnica de Interoperabilidad de Reutilización de Recursos de Información del Sector Público (NTI-RISP) [19]. Ambas capas se necesitan: la verificación sin validación produce software estable pero quizá irrelevante, y la validación sin verificación produce conformidad puntual que puede romperse en la siguiente ejecución.

3.11. Síntesis del estado del arte

El estado actual combina varias capas: gobierno del dato para definir responsabilidades, catálogos de metadatos para inventariar activos, perfiles DCAT para interoperar, SHACL para validar, OM para capturar y gobernar metadatos técnicos, Kubernetes y Helm para reproducir infraestructura, y una consola web para operar el flujo. La aportación del TFM consiste en ensamblar esas piezas con un alcance reducido, reproducible y defendible: doble ingesta PostgreSQL, gobierno funcional de tablas `gold`, exportación DCAT-AP-ES y validación SHACL, con una capa centralizada de curación que diferencia la propuesta frente a la edición *custom property* dataset por dataset descrita en la sección 3.7.1.

4. Metodología de trabajo

Este capítulo describe cómo se ha organizado y ejecutado el trabajo: el enfoque de gestión del proyecto, el tablero y las fases del *roadmap*, el stack tecnológico elegido y la organización del repositorio. La descripción de la arquitectura, la implementación y la validación de la solución se reserva para el capítulo de resultados.

4.1. Enfoque general

El TFM se ha gestionado con un enfoque *lean* basado en **Kanban** [20], complementado con los procesos de planificación, seguimiento y cierre del Project Management Body of Knowledge (PMBOK) [21] adaptados a un trabajo individual. La elección no es ornamental: Kanban encaja con un trabajo de alcance acotado y entrega incremental, mientras que el PMBOK aporta vocabulario y disciplina para los hitos académicos (planificación, ejecución, validación, cierre). Esta combinación se alinea con las prácticas habituales del sector cuando un único responsable cubre los roles de analista, desarrollador y revisor.

Las reglas operativas se han fijado de manera explícita para evitar deriva:

- **Límite de trabajo en curso (WIP):** máximo 2-3 tareas simultáneas, una sola en En curso cuando incluye despliegue o cambios de infraestructura.
- **Revisión semanal:** cierre y replanificación cada lunes; revisión adicional al cierre de cada fase.
- **Definición de Hecho (DoD):** una tarea solo se mueve a Hecho cuando existe evidencia versionada en el repositorio (código, configuración, documentación o artefacto regenerable) y, en su caso, tests verdes.

4.2. Tablero y vistas en GitHub Project

La herramienta canónica de seguimiento es **GitHub Projects v2** [22], sincronizada con los *issues* del propio repositorio. El tablero combina seis estados (Backlog, Por definir, En curso, Bloqueado, En revisión, Hecho) con campos personalizados de fase, tipo de tarea, fecha de inicio y fecha de fin. Esta estructura permite obtener simultáneamente una vista Kanban, un *roadmap* cronológico (Gantt) y un listado de evidencias por tarea.

La sincronización no es manual: el archivo declarativo `scripts/planning/github_project_planificacion.json` contiene la fuente de verdad del roadmap completo. El script `scripts/planning/bootstrap_github_project.py` aplica ese archivo contra el Project remoto de forma idempotente, creando o actualizando *milestones*, *issues*, etiquetas y campos del Project sin duplicar items. Esta automatización garantiza que el tablero refleja el estado real del trabajo planificado y evita la deriva entre documentación y herramienta. La figura 4.1 muestra el tablero real al cierre de la fase 05 como evidencia de este seguimiento.

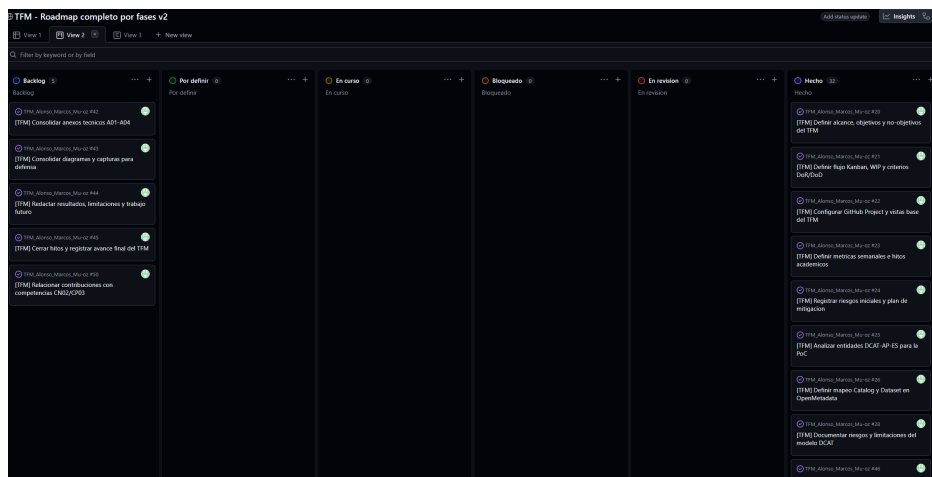


Figura 4.1: Vista Kanban del GitHub Project del TFM al cierre de la fase 05. Elaboración propia.

4.3. Fases del roadmap

El trabajo se ha organizado en seis fases secuenciales con superposición controlada. La tabla 4.1 resume cada fase con su ventana temporal real, las áreas DMBOK [8] implicadas y el estado al cierre del cuerpo de la memoria.

Fase	Ventana	Área DMBOK implicada
01 Planificación	feb–abr 2026	Gobierno, arquitectura
02 Modelo DCAT-AP-ES	mar–abr 2026	Metadatos, interoperabilidad
03 Configuración OpenMetadata	abr 2026	Metadatos, integración
04 Pipeline de ingesta	abr–may 2026	Integración, gobierno
05 Validación	may 2026	Calidad, gobierno
06 Memoria y defensa	may–jul 2026	Documentación

Tabla 4.1: Fases del roadmap del TFM con su área DAMA-DMBOK predominante. Elaboración propia, sincronizada con `scripts/planning/github_project_planificacion.json`.

4.4. Vista temporal del trabajo

El diagrama de Gantt derivado del Project (figura G.2 del Anexo G) muestra que las fases se solapan donde la naturaleza del trabajo lo permitía (por ejemplo, la fase 02 de modelo DCAT-AP-ES arrancó mientras se cerraban riesgos de la fase 01). El bloque de redacción de la memoria está reservado a las últimas semanas, con la defensa en julio de 2026 como hito final.

4.5. Stack tecnológico

El stack tecnológico de la plataforma se ha elegido priorizando reproducibilidad, portabilidad y el uso de estándares abiertos. Esta sección justifica las tecnologías de la fuente de datos, del despliegue y de la consola web; su uso concreto en la solución se detalla en el capítulo de resultados.

4.5.1. PostgreSQL como fuente técnica reproducible

PostgreSQL es una base de datos relacional ampliamente utilizada y adecuada para construir un caso de uso reproducible. En el mundo real, los catálogos técnicos suelen ingerir metadatos desde múltiples motores: PostgreSQL, MySQL, sistemas cloud, data lakes o almacenes analíticos. Para el TFM, una única tecnología de origen es suficiente siempre que permita demostrar estructura técnica real: servicio, base de datos, esquemas, tablas y columnas.

La doble ingesta usada en el caso de uso de validación no pretende simular dos organizaciones distintas, sino evidenciar que la lógica de gobierno opera por identificadores completos de OpenMetadata. Dos servicios pueden contener tablas con el mismo nombre. Si el sistema mezclara filas por `schema.table`, el resultado sería incorrecto. Por eso el FQN del

activo técnico se convierte en un identificador esencial.

4.5.2. Docker, Kind, Kubernetes y Helm

Docker proporciona contenedores: paquetes ejecutables con aplicación, dependencias y configuración de runtime [23]. Kind permite ejecutar clústeres Kubernetes locales usando nodos como contenedores Docker, lo que resulta adecuado para desarrollo, pruebas y CI [24]. Kubernetes es una plataforma portable y extensible para gestionar cargas de trabajo y servicios contenedorizados mediante configuración declarativa y automatización [16]. Helm actúa como gestor de paquetes para Kubernetes, empaquetando aplicaciones como charts y desplegándolas como releases [25].

En el estado actual de la ingeniería cloud, esta combinación es habitual para construir entornos reproducibles. No hace falta contratar un clúster gestionado para demostrar una arquitectura portable: Kind permite trabajar localmente, Kubernetes mantiene el modelo operativo y Helm aproxima el despliegue a prácticas profesionales. La limitación es evidente: no aporta alta disponibilidad ni hardening productivo. Para un TFM, esa limitación es aceptable porque el objetivo es reproducibilidad y claridad, no operación empresarial avanzada (véase la figura G.3 del Anexo G).

4.5.3. Consola web: React, Next.js y TypeScript

Una consola web operativa convierte comandos reproducibles en una experiencia de uso guiada. En proyectos de gobierno de datos, esto es relevante porque no todos los perfiles que revisan metadatos son desarrolladores. Una persona responsable de catálogo puede necesitar revisar la hoja de gobierno, lanzar una validación o consultar artefactos sin recordar comandos de PowerShell o Python.

La tecnología web actual facilita construir este tipo de consola con componentes reutilizables y separación entre cliente y servidor. React permite definir interfaces mediante componentes y estado [26]. Next.js añade estructura de aplicación, rutas, componentes de servidor y una capa backend integrada [27]. TypeScript aporta comprobación estática de tipos sobre JavaScript, ayudando a detectar errores antes de ejecutar el código [28].

En esta plataforma, la consola web representa una forma habitual de operar flujos técnicos desde una interfaz controlada. La regla arquitectónica es estricta: la UI no contiene lógica de gobierno. Solo invoca operaciones cerradas, edita la hoja funcional y muestra resultados, logs y artefactos.

4.6. Organización del repositorio

Todo el código, la configuración y la documentación del TFM están disponibles en el repositorio público del autor en GitHub.¹ El repositorio sigue una organización deliberada para que cada bloque sea identificable y reproducible. El árbol de directorios canónico se recoge en la figura G.1 del Anexo G, y la tabla 4.2 resume la responsabilidad funcional de cada bloque.

Bloque	Responsabilidad
sql/	Activo técnico fuente del caso de uso de validación.
k8s/	Despliegue declarativo de OM y dependencias.
scripts/infra/	Operaciones reproducibles de infraestructura.
tfm_ingestor/	Lógica de negocio del gobierno y la exportación.
web/	Consola operativa sobre la lógica anterior.
docs/	Fuente narrativa y de decisiones.
tmp_pytest/	Evidencias regenerables.

Tabla 4.2: Bloques principales del repositorio. Elaboración propia.

¹https://github.com/AlonsoMarcosM/TFM_Alonso_Marcos_Mu-oz

5. Resultados

Este capítulo presenta los resultados del trabajo: la arquitectura de la solución y sus decisiones de diseño, la implementación en código, la validación realizada y la discusión de beneficios, limitaciones y trazabilidad.

5.1. Arquitectura de la solución

5.1.1. Actores del sistema

El sistema tiene un actor principal y dos secundarios. El actor principal es el **responsable del catálogo** (*steward* u operario de gobierno), que cura los metadatos y ejecuta el flujo, indistintamente desde la hoja CSV o desde la consola web. Como actores secundarios intervienen el **perfil técnico** (sistemas e ingeniería de datos), que prepara y mantiene la infraestructura, y los **consumidores** del catálogo publicado (portales de datos abiertos, analistas u otras organizaciones), que reutilizan la salida DCAT-AP-ES.

El diagrama de secuencia de la figura 5.1 resume esa interacción de extremo a extremo, desde la edición de la hoja de gobierno hasta la validación SHACL.

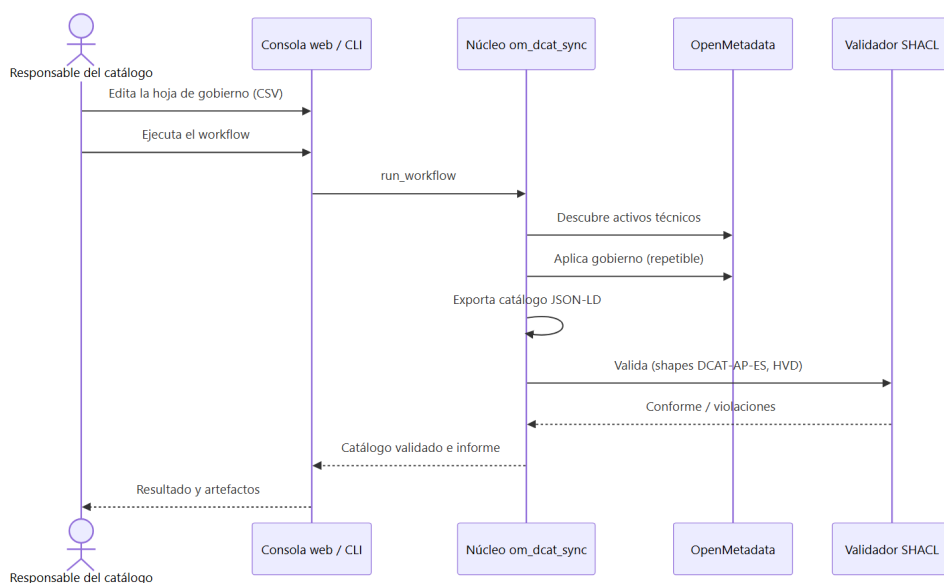


Figura 5.1: Diagrama de secuencia de la operación del responsable del catálogo, desde la curación de la hoja hasta el catálogo validado. Elaboración propia.

5.1.2. Requisitos funcionales

Los requisitos funcionales identifican capacidades observables que la plataforma debe ofrecer al usuario operador y al evaluador externo. La tabla 5.1 los enumera con identificador, descripción y criterio de aceptación verificable.

ID	Requisito	Criterio de aceptación
RF-01	Despliegue reproducible de OpenMetadata y sus dependencias.	El script <code>launch_infra.ps1</code> levanta el clúster con <code>kubect1 get pods</code> sin errores.
RF-02	PostgreSQL de referencia con capas <code>bronze/silver/gold</code> .	<code>opendata_demo_init.sql</code> aplicado; los tres esquemas son visibles.
RF-03	Doble ingesta de PostgreSQL en OpenMetadata.	Dos <code>DatabaseService</code> diferenciados con tablas <code>gold</code> .
RF-04	Selección de datasets publicables.	Solo filas con <code>publicar=si</code> aparecen en el catálogo exportado.
RF-05	Curación funcional por Fully Qualified Name (identificador completo de activo en OpenMetadata) (FQN) de tabla.	La hoja CSV identifica filas únicas por <code>table_fqn</code> .
RF-06	Sincronización idempotente con OpenMetadata.	Una segunda ejecución consecutiva aplica 0 cambios.
RF-07	Exportación a JSON-LD conforme al perfil activo.	Se genera <code>dcat_catalog.jsonld</code> con clases DCAT activas.
RF-08	Validación SHACL contra el bundle vendorizado.	Se genera <code>dcat_validation_report.ttl</code> sin violaciones bloqueantes.
RF-09	Consola web operativa para todo el flujo.	La UI ejecuta cada etapa y muestra logs y artefactos.

Tabla 5.1: Requisitos funcionales con criterio de aceptación. Elaboración propia.

5.1.3. Requisitos no funcionales

Los requisitos no funcionales recogen las propiedades de calidad transversales que la plataforma debe sostener:

- **Reproducibilidad:** toda ejecución parte de comandos versionados y un repositorio limpio reconstruye el entorno completo.
- **Idempotencia:** la sincronización contra OM no duplica etiquetas, propiedades ni recursos.
- **Simplicidad operativa:** una única capa Python concentra las reglas, mientras que la Command Line Interface (CLI), los scripts y la web son envoltorios.
- **Trazabilidad:** cada artefacto se asocia a su configuración, código y comando de origen.
- **Separación de responsabilidades:** la consola web no implementa reglas, las delega en el núcleo.
- **Portabilidad:** el despliegue es transferible entre entornos compatibles con Kubernetes y Helm.
- **Auditabilidad:** cada ejecución produce evidencia regenerable y consultable.

Se priorizan la reproducibilidad y la idempotencia por encima de capacidades habituales en sistemas productivos, como la alta disponibilidad, el Role-Based Access Control (RBAC) avanzado o el escalado, que quedan fuera del alcance funcional declarado en la introducción.

5.1.4. Arquitectura lógica

La arquitectura se organiza en cinco capas. En lugar de describirlas con una tabla, se representan mediante un diagrama (figura 5.2), que hace explícita la responsabilidad de cada capa y la dirección de las dependencias y del flujo de datos entre ellas [29]. De arriba abajo, la fuente técnica (PostgreSQL de referencia) alimenta el catálogo técnico (OpenMetadata), del que el gobierno funcional toma los activos descubiertos; el núcleo Python fusiona hoja y defaults, aplica gobierno idempotente sobre OpenMetadata y produce el catálogo DCAT-AP-ES validado; la capa de operación (CLI, scripts y consola web), en la base, invoca ese mismo núcleo sin duplicar reglas.

Cada capa se respalda con evidencia versionada en el repositorio: la fuente técnica en `sql/opendata_demo_init.sql`; el catálogo técnico en `ingest_postgres_double.ps1`; el gobierno funcional en `gold_governance.csv` y `governance_defaults.yaml`; el núcleo en `tfm_ingestor/src/tfm_ingestor/`; y la operación en `scripts/infra/` y `web/`.

5.1.5. Arquitectura física

La vista física (figura 5.3) muestra cómo se materializan esas capas en el despliegue. Todo se ejecuta sobre un único host (portátil, VPS o nube) con Docker como motor de contenedores y un clúster Kubernetes local gestionado por Kind. OpenMetadata, sus dependencias (MySQL y OpenSearch) y el PostgreSQL de referencia se ejecutan como cargas dentro del clúster. El operario interactúa a través de la consola web y del CLI, que se comunican con OpenMetadata mediante un `port-forward` sobre la API REST autenticada con JSON Web Token (JWT). El detalle de los *values* Helm y de los manifiestos se recoge en el Anexo B.

5.1.6. Flujo extremo a extremo

El flujo implementado sigue una secuencia cerrada, resumida en el diagrama de secuencia de la figura 5.1. Primero se levanta la infraestructura. Después se ingiere PostgreSQL dos veces en OpenMetadata, bajo dos servicios diferenciados. A continuación se preparan etiquetas y custom properties. El workflow refresca la hoja de gobierno, permite revisar los metadatos funcionales, aplica cambios en OpenMetadata, exporta JSON-LD y ejecuta la validación SHACL. La figura 5.4 muestra esa secuencia operativa por pasos; la transformación conceptual de los metadatos, desde los activos técnicos hasta la salida validada, se representa más adelante en la figura 5.7.

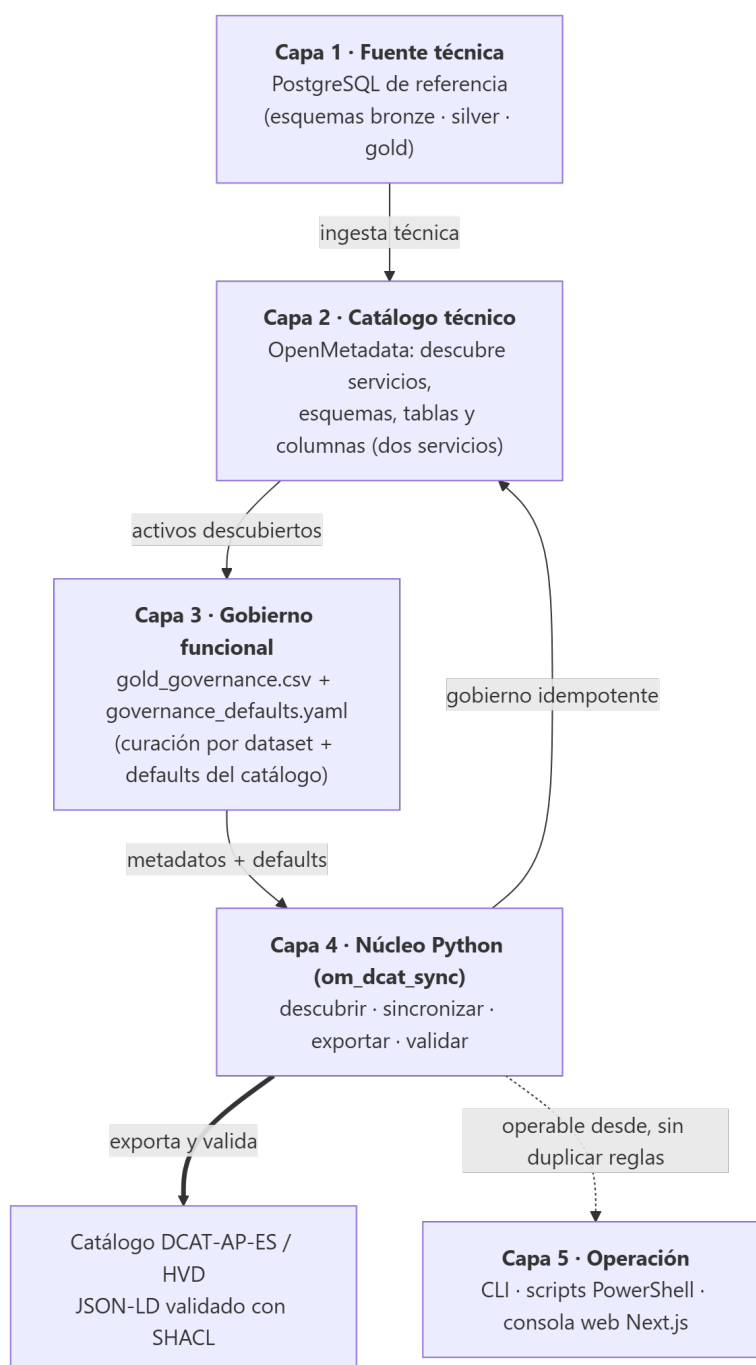


Figura 5.2: Arquitectura lógica por capas y responsabilidades: fuente técnica, catálogo técnico, gobierno funcional, núcleo Python y operación. Elaboración propia.

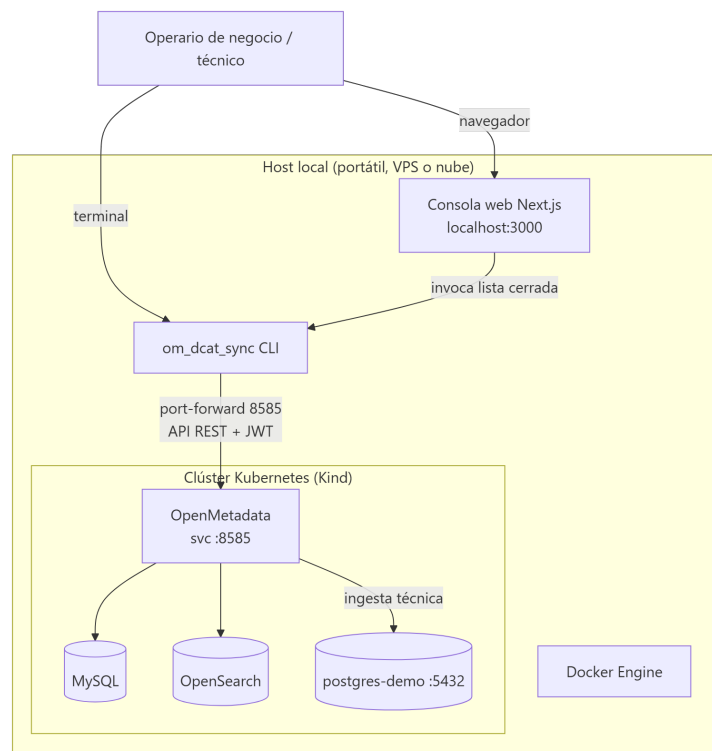


Figura 5.3: Arquitectura física de despliegue: host con Docker y clúster Kind que ejecuta OpenMetadata, sus dependencias y el PostgreSQL de referencia. Elaboración propia.



Figura 5.4: Flujo operativo del caso de uso de validación, paso a paso. Elaboración propia.

5.1.7. Decisiones de diseño

5.1.7.1. PostgreSQL como fuente canónica

El origen técnico se fija en PostgreSQL porque permite controlar la estructura, reproducir el entorno y demostrar descubrimiento real de activos técnicos. El SQL de referencia crea seis tablas distribuidas en tres capas. Las tablas `gold.movilidad_resumen_municipio` y `gold.agenda_cultural_publica` representan el catálogo publicable.

5.1.7.2. Gobierno restringido a la capa gold

La arquitectura *medallion* (bronze/silver/gold) es una convención consolidada en plataformas de datos modernas, popularizada por Databricks como patrón de zonas progresivas de refinamiento [30], que separa tres estados de madurez: datos crudos ingeridos sin transformar (bronze), datos limpios y conformados (silver) y datos refinados, agregados y listos para consumo final (gold). La plataforma gobierna *únicamente* la capa gold; las capas bronze y silver se ingieren en OM pero no se exportan como datasets DCAT-AP-ES. La figura 5.5 ilustra estas tres capas de refinamiento progresivo.

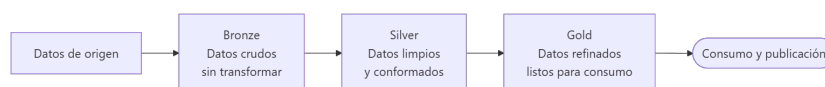


Figura 5.5: Arquitectura *medallion*: capas bronze, silver y gold de refinamiento progresivo del dato. Adaptado de la documentación de Databricks.

Esta restricción es deliberada y se apoya en cuatro razones convergentes. Por **conformidad semántica**, el perfil exige que un `dcat:Dataset` sea un conjunto «publicable» con título, descripción, publicador y temática (y, en HVD, categoría y legislación), algo que las tablas bronze y silver no satisfacen, pues contienen estructuras transitorias sin significado funcional fuera del pipeline; publicarlas falsearía el catálogo y haría fracasar la validación SHACL. Por **coherencia con DAMA-DMBOK**, que establece que el gobierno aplica sobre activos cuyo significado, propiedad y calidad son estables [8], condición que las capas transitorias no cumplen. Por **separación entre catálogo técnico y de publicación**, ya que OM mantiene visibles las tres capas como inventario interno mientras que la frontera se aplica solo en la exportación, igual que en los portales de datos abiertos, que nunca publican el inventario técnico completo. Y por **foco en el valor evaluado**, porque añadir bronze y silver multiplicaría el trabajo editorial sin añadir capacidad demostrable: el flujo, la idempotencia y las shapes son los mismos.

La decisión, por tanto, no es una limitación de implementación, sino una alineación con la semántica del perfil, con el marco de gobierno y con la práctica de los portales de datos

abiertos. La plataforma puede gobernar tablas adicionales sin cambios estructurales: basta marcar `publicar=si` en `gold_governance.csv`, lo que demuestra que la decisión es de alcance, no de capacidad técnica.

5.1.7.3. Doble ingesta como validación multi-fuente

La doble ingesta usa la misma base de referencia, pero crea dos `DatabaseService` en `OpenMetadata:postgres_demo_service` y `postgres_validation_service`. El objetivo no es duplicar datos por volumen, sino comprobar que el gobierno funciona cuando existen tablas con el mismo nombre en servicios distintos. Por eso la hoja funcional conserva `schema_name`, `table_name` y, sobre todo, `table_fqn`.

5.1.7.4. Hoja funcional para gobierno editorial

En este trabajo, *curación* designa la actividad editorial de seleccionar, describir, enriquecer y mantener los metadatos de un activo para que sean comprensibles y publicables. Es el término consolidado en gestión del dato (*data curation* [8]), frente a «edición», demasiado genérica, o a «limpieza» y «saneamiento», que aluden a la calidad del dato de negocio y no a sus metadatos. La *curación funcional* es esa misma actividad aplicada a los metadatos funcionales o de negocio de cada dataset (título, descripción, publicador, temática y categoría HVD), por oposición a los metadatos técnicos que OM captura de forma automática.

La curación funcional se concentra en `tfm_ingestor/config/gold_governance.csv`. Esta hoja permite editar los metadatos que varían por dataset: decisión de publicación, título, descripción, publicador, temática DCAT, categoría HVD y URL de acceso. Los campos técnicos (esquema, tabla y FQN) no deben editarse manualmente salvo regeneración controlada del caso de uso. Las columnas de la hoja se resumen en la tabla G.1 del Anexo G, y el detalle completo con criterios de validación, en el Anexo C.

El campo `publicar` merece una explicación de gobierno y no de mero filtrado técnico. La exportación del catálogo es idempotente y, por defecto, recorre el total de activos gobernados (datasets, distribuciones y servicios de datos). El valor `si/no` introduce un control editorial sobre qué entra realmente en el catálogo DCAT-AP-ES publicado. Esa distinción alinea la plataforma con el principio DAMA de gestionar el ciclo de vida del dato [8]: permite diferenciar los datasets publicables de aquellos que permanecen privados, en estado de borrador o catalogados internamente pero todavía no federables. De este modo, una misma hoja puede reflejar estados de publicación distintos por dataset y acompañar su evolución hacia la federación sin perder ni trazabilidad ni reproducibilidad.

5.1.7.5. Configuración global por YAML

Los metadatos globales del catálogo se derivan de `governance_defaults.yaml`: nombre del catálogo, descripción, publicador, URI de organismo, página principal, idioma, licencia, legislación HVD, contacto y bases de URLs. Estos valores operan como *defaults* a dos niveles: describen el `dcat : Catalog` y, además, sirven de valor por defecto heredable por cada `dcat : Dataset`. Esta separación evita repetir información común en cada dataset y reduce errores de consistencia. Cuando un dataset concreto requiere un publicador, una licencia, un idioma o un contacto propios, algo habitual cuando coexisten varios organismos o responsables, la hoja funcional puede sobrescribir el default global a nivel de fila sin modificar el núcleo. El perfil operativo que conecta esos defaults con la hoja CSV y fija el caso HVD como modo de validación se recoge en el listado G.5 del Anexo G.

5.1.7.6. Núcleo único de lógica

La regla arquitectónica principal es que la lógica de gobierno vive en Python: el CLI, los scripts y la web invocan ese núcleo, lo que evita reglas paralelas en la interfaz y facilita probar la solución con `pytest`. El entrypoint canónico es una `dataclass` inmutable (`frozen=True`) que fija todos los parámetros del workflow antes de ejecutarlo (su firma se detalla en la implementación). El término *dry-run* («ejecución en seco») designa una ejecución que calcula y muestra el plan de cambios sin aplicarlos sobre OM; se conserva en inglés por ser el nombre literal de la opción del CLI (`--dry-run`).

5.1.8. Modelo activo DCAT-AP-ES

El modelo activo del TFM cubre las clases `dcat : Catalog`, `dcat : Dataset`, `dcat : Distribution`, `dcat : DataService` y `foaf : Agent`. `OpenMetadata` mantiene únicamente los metadatos que conviene gobernar manualmente: `displayName`, `description`, `dcat_publisher_name`, `dcat_hvd_category`, `dcat_access_url` y `tags` `dcat_theme.*`. El resto se deriva durante la exportación.

Esta decisión prioriza el camino completo sobre la cobertura exhaustiva. El alcance funcional declarado en el capítulo 2 no incluye los metadatos opcionales que no inciden en la conformidad HVD; modelarlos exigiría duplicar trabajo editorial sin aportar valor adicional al ciclo de validación, y queda explícitamente reservado como evolución posterior. La plataforma demuestra, por tanto, un camino completo y validable para el subconjunto obligatorio y operativo del caso HVD.

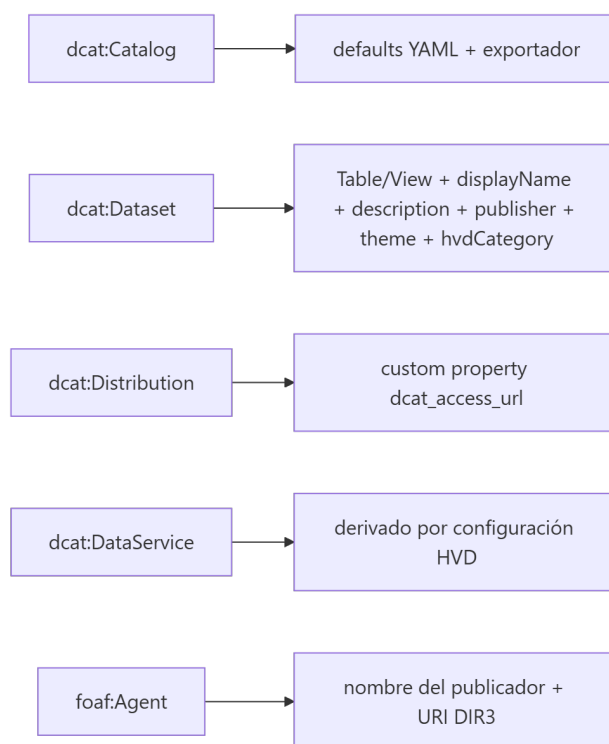


Figura 5.6: Mapeo activo entre clases DCAT-AP-ES y elementos de la plataforma. Elaboración propia.

5.2. Implementación

Esta sección describe cómo se materializan en código las decisiones de diseño de la sección 5.1. La narrativa sigue el orden lógico de la plataforma: fuente técnica, ingesta, núcleo Python, configuración funcional, consola web y, por último, el despliegue reproducible. La figura 5.7 resume esa transformación de los metadatos, desde los activos técnicos descubiertos en OpenMetadata hasta la salida validada con SHACL, y sirve de hilo conductor de la sección. La organización del repositorio se describe en la sección 4.6. Los comandos exactos para reproducir cada bloque están en el Anexo A; las capturas que evidencian cada paso y los listados de configuración extensos, en el Anexo G; y el recorrido visual completo desde la consola web, en el Anexo F.

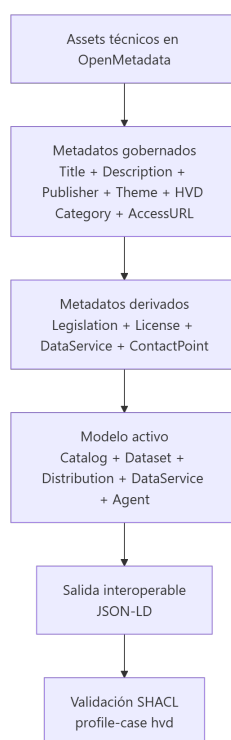


Figura 5.7: Pipeline de metadatos desde OpenMetadata hasta la validación SHACL. Elaboración propia.

5.2.1. Fuente técnica reproducible: PostgreSQL en tres capas

La fuente técnica es un PostgreSQL desplegado en el mismo clúster Kubernetes que OM, inicializado a partir de `sql/opendata_demo_init.sql` [31]. El SQL crea tres esquemas aliñados con el patrón *medallion* (bronze/silver/gold), de modo que el caso de uso de vali-

dación dispone de datos sintéticos verosímiles sin depender de descargas externas. Únicamente el esquema `gold` se considera publicable: las capas `bronze` y `silver` se ingieren para mantener visible la cadena de transformaciones, pero no se exportan como datasets DCAT-AP-ES, por las razones de conformidad semántica, alineamiento con DMBOK y separación entre catálogo técnico y de publicación que se argumentan en la sección 5.1.7.2. El listado 5.1 muestra la creación de una de las tablas `gold` publicables, con una estructura suficientemente expresiva (claves temporales, dimensiones territoriales y métricas) para que el catálogo exportado contenga campos significativos al validar con SHACL.

```
1 CREATE TABLE IF NOT EXISTS gold.agenda_cultural_publica (  
2     evento_id          BIGINT      PRIMARY KEY,  
3     fecha              DATE        NOT NULL,  
4     municipio          TEXT        NOT NULL,  
5     categoria          TEXT        NOT NULL,  
6     titulo             TEXT        NOT NULL,  
7     asistencia_est     INTEGER     CHECK (asistencia_est >= 0),  
8     actualizado_en     TIMESTAMPTZ DEFAULT now()  
9 );
```

Código 5.1: Fragmento de `sql/opendata_demo_init.sql`: tabla publicable de la capa `gold`.

5.2.2. Doble ingesta técnica

En OpenMetadata, una *ingesta* consiste en conectarse a una fuente de datos y registrar automáticamente sus metadatos técnicos (el servicio, las bases de datos, los esquemas, las tablas y las columnas), sin copiar los datos de negocio. La *doble ingesta* ejecuta ese proceso dos veces sobre la misma base PostgreSQL, registrándola bajo dos servicios lógicos distintos en OM: `postgres_demo_service` y `postgres_validation_service`.

El descubrimiento de activos es automático y no depende de la arquitectura *medallion*: el conector de OM lee el catálogo interno de la propia fuente (en PostgreSQL, las vistas de `information_schema`) y registra cuanto encuentra, es decir, *las tres capas* `bronze`, `silver` y `gold` con todas sus tablas y columnas, no solo `gold`. A cada activo descubierto OM le asigna su FQN (servicio, base de datos, esquema y tabla), que es la nomenclatura con la que el resto del flujo lo identifica sin ambigüedad. La restricción a `gold` no se aplica en el descubrimiento, sino después, como decisión editorial: la hoja de gobierno marca con `publicar` qué activos descubiertos entran en el catálogo publicable (sección 5.1.7.2). Por eso una fuente que no use *medallion* se descubre exactamente igual; lo único que cambia es qué filas se marcan como publicables, sin tocar el núcleo.

El motivo de la doble ingesta no es duplicar datos, sino reproducir de forma controlada un escenario habitual en organizaciones reales: que una misma tabla, por ejemplo `gold`.

`agenda_cultural_publica`, exista con idéntico nombre en dos fuentes diferentes. Con ello se comprueba que la lógica de gobierno identifica cada activo por su nombre completo en OM (FQN, que incluye servicio, base de datos, esquema y tabla) y no por el nombre corto de la tabla. De lo contrario, las dos tablas homónimas se mezclarían y el catálogo resultante sería incorrecto. La ingesta se invoca desde `scripts/infra/ingest_postgres_double.ps1` y su resultado es visible en la UI de OM (figura G.7): cuatro tablas publicables, dos por servicio, cada una con su FQN único.

5.2.3. Núcleo Python: paquete `om_dcat_sync`

El paquete `tfm_ingestor` (alias importable `om_dcat_sync`) concentra toda la lógica de gobierno, exportación y validación. Está estructurado en módulos con responsabilidades acotadas, conforme al principio de núcleo único declarado en la sección 5.1. La tabla 5.2 lista los módulos principales y la figura G.9 del Anexo G resume su composición.

Módulo	Responsabilidad
<code>workflow_service.py</code>	Orquestador canónico del workflow end-to-end.
<code>governance_service.py</code>	Sincroniza gobierno entre hoja CSV y OM.
<code>governance_sheet.py</code>	Lectura, escritura y validación de la hoja CSV.
<code>om_api.py</code>	Cliente REST ligero para la API de OM.
<code>dcat_export.py</code>	Construye el grafo DCAT-AP-ES y lo serializa en JSON-LD.
<code>shacl_validation.py</code>	Aplica las <i>shapes</i> vendorizadas con pySHACL.
<code>config.py</code>	Carga defaults YAML y reglas de mapeo.
<code>operational_profile.py</code>	Une defaults, reglas y caso de validación.
<code>runtime_validation.py</code>	Verifica coherencia del estado vivo de OM.
<code>main.py</code>	CLI basado en <code>argparse</code> .

Tabla 5.2: Módulos principales de `tfm_ingestor`. Elaboración propia.

5.2.3.1. Configuración inmutable y entrypoint

El módulo `workflow_service.py` define una `dataclass` inmutable (`frozen=True`) [32] que fija todos los parámetros del workflow antes de su ejecución, lo que evita estados mutables compartidos entre llamadas. El entrypoint `run_workflow` encadena las cuatro etapas (refresco de hoja, sincronización de gobierno, exportación y validación); su esqueleto completo, que amplía el listado G.1 con la secuencia operativa real, se recoge en el listado G.2 del Anexo G.

5.2.3.2. Sincronización repetible (idempotente) de gobierno

En este contexto, *sincronización idempotente* significa, sencillamente, que la operación es **repetible sin efectos colaterales**: ejecutarla una vez o varias veces seguidas deja

OpenMetadata en el mismo estado. La primera ejecución aplica los cambios necesarios y las siguientes no vuelven a modificar nada. `governance_service.py` mapea cada fila de la hoja CSV a operaciones PATCH o POST sobre la entidad correspondiente en OM usando su FQN. Esa propiedad se materializa por dos vías: las propiedades gestionadas (MANAGED_DCAT_CUSTOM_PROPERTIES) solo se reescriben si cambian, y las etiquetas `dcat_theme.*` se aplican como conjuntos, nunca como acumulación lineal. La consecuencia es que ejecutar el workflow dos veces consecutivas produce cero cambios en la segunda ejecución, una de las métricas de validación de la sección 5.3.

El resultado del gobierno es directamente observable en OM: tras el workflow, cada tabla `gold` publicable conserva sus propiedades personalizadas DCAT (`dcat_publisher_name`, `dcat_hvd_category`, `dcat_access_url`) y sus etiquetas `dcat_theme.*`, que son la materia prima que el exportador transforma en el grafo DCAT-AP-ES (figura G.8, Anexo G).

5.2.3.3. Exportador DCAT-AP-ES

`dcat_export.py` compone el grafo conceptual a partir de las clases activas `dcat:Catalog`, `dcat:Dataset`, `dcat:Distribution`, `dcat:DataService` y `foaf:Agent`. La construcción se hace en Python puro mediante diccionarios anidados que respetan el contexto JSON-LD, y cuando es necesario manipular el grafo (normalización o deduplicación) se delega en RDFLib [33]. El contexto JSON-LD canónico, con prefijos estables, idioma por defecto en español y enlace a los vocabularios temáticos de NTI-RISP [19], se recoge en el listado G.3 del Anexo G.

5.2.3.4. Validación SHACL del catálogo exportado

Esta etapa comprueba formalmente que el catálogo cumple el perfil DCAT-AP-ES. `shacl_validation.py` toma como entrada el grafo JSON-LD exportado en la etapa anterior y un *grafo de shapes* (el conjunto de restricciones del perfil), y los confronta con el motor pySHACL [34], implementación de referencia de SHACL para Python. El concepto de *shape* y un ejemplo mínimo se introdujeron en la sección 3.6. Aquí se aplican las shapes oficiales completas.

El bundle de shapes está vendorizado en `tfm_ingestor/resources/shacl/` y congelado en el commit `f2c8a888` del repositorio oficial `datosgobes/DCAT-AP-ES` [10]. Congelarlo, en lugar de descargarlo en cada ejecución, evita dependencias de red y garantiza que dos validaciones del mismo catálogo den siempre el mismo resultado. El caso `hvd` carga, además de las restricciones generales, las shapes específicas de los conjuntos de datos de alto valor, que exigen propiedades adicionales como la categoría HVD, la legislación aplicable y el servicio de acceso.

El resultado de pySHACL tiene dos partes: un valor booleano `conforms`, que indica si el

grafo cumple todas las restricciones *obligatorias*, y un informe en Turtle que detalla cada incidencia mediante `sh:focusNode` (el recurso afectado), `sh:resultPath` (la propiedad implicada), `sh:severity` (la gravedad) y `sh:resultMessage` (la explicación). La gravedad distingue dos casos con tratamiento distinto. Una `sh:Violation` señala el incumplimiento de una restricción obligatoria y hace que `conforms` sea falso. Un `sh:Warning` indica solo una recomendación no cubierta y no invalida la conformidad. Por eso el módulo `postprocesa` el informe para filtrar los `sh:Warning` cuando se ejecuta con `--allow-warnings`, dejando visibles únicamente las incidencias bloqueantes. El criterio de aceptación es, por tanto, inequívoco: el catálogo se considera conforme cuando no contiene ninguna `sh:Violation`. Este resultado se consolida como la métrica de conformidad SHACL de la sección 5.3 y se contrasta con un validador europeo independiente en la sección 5.3.4.

5.2.4. Hoja funcional de gobierno y defaults globales

La curación funcional se concentra en `tfm_ingestor/config/gold_governance.csv`, cuya estructura columnar se describe en la tabla G.1 y se detalla con criterios de validación en el Anexo C. El uso de CSV con separador `;` no es accidental: facilita la edición en hojas de cálculo locales sin Excel y, sobre todo, hace que el diff en Git sea legible para revisión editorial. Los defaults globales viven en `governance_defaults.yaml`, que centraliza publicador, homepage, idioma, licencia, contacto y la legislación HVD aplicable. La pantalla *Gobierno* de la consola web (figura G.10, Anexo G) actúa sobre la misma hoja y aplica las mismas reglas de validación que el CLI; su uso real se narra en la sección F.5 del Anexo F.

5.2.5. Consola web operativa Next.js

La consola web se construye con Next.js [27], React [26] y TypeScript [28], organizada en rutas que mapean uno a uno con las etapas del workflow: *Infraestructura*, *Ingesta*, *Gobierno*, *Workflow*, *DCAT*, *Estado vivo*, *Validación*, *SHACL*, *Artefactos* y *Ejecuciones*. La arquitectura interna respeta la regla de núcleo único: la UI nunca habla directamente con OM ni reimplementa reglas, sino que invoca una lista cerrada de operaciones de servidor (`operations.ts`) que ejecutan el CLI o los scripts PowerShell. La descripción pantalla a pantalla, con los artefactos producidos en cada paso y sus capturas, se desarrolla en el Anexo F, y los comandos equivalentes desde CLI y `pnpm` en el Anexo E.

Cada ejecución crea un *job* versionado en `state/web_jobs/` con identificador único, log completo, código de salida, duración y artefactos generados. El listado G.6 del Anexo G muestra la estructura de un *job* persistido, que la pantalla *Ejecuciones* consume para renderizar el historial.

5.2.6. Despliegue reproducible en Kubernetes

El despliegue local utiliza Kind como motor de clúster Kubernetes encima de Docker [23, 24, 16]. OM y sus dependencias (MySQL y OpenSearch) se instalan mediante Helm [25] usando los *values* canónicos del repositorio, cuyo fragmento mínimo se recoge en el listado G.4 del Anexo G. El script `scripts/infra/launch_infra.ps1` encadena la creación del clúster, la instalación de los charts y los `port-forward` necesarios para exponer la UI de OM en `localhost:8585` y el PostgreSQL de referencia en `localhost:5432`. La idempotencia se garantiza mediante `helm upgrade --install` y comprobaciones con `kubectl get` previas a cada paso destructivo. Los contenedores Docker y los *Pods* resultantes (figuras G.4 y G.5, Anexo G), así como los *values* Helm canónicos, se detallan en el Anexo B.

5.2.7. Resumen de implementación

La implementación cumple las tres reglas duras declaradas en la sección 5.1: una única lógica de negocio en Python, idempotencia comprobable y separación estricta entre datos técnicos, curación funcional y derivaciones automáticas. La sección siguiente describe cómo se valida formalmente este ensamblaje y qué evidencias produce.

5.3. Validación y resultados de la validación

Esta sección evalúa la corrección de la salida producida por la plataforma. Complementa el recorrido operativo *end-to-end* documentado en el Anexo F, donde se narran las pantallas de la consola web y los artefactos producidos en cada paso. Aquí se mide qué garantiza esa salida. Allí se muestra cómo se obtiene paso a paso.

La validación combina tres niveles:

1. pruebas automatizadas de módulos Python y servidor web;
2. validación estructural del estado vivo de OpenMetadata;
3. validación semántica del catálogo JSON-LD mediante SHACL.

Esta estrategia evita depender de una comprobación visual de la interfaz. Cada resultado debe poder reproducirse mediante comandos versionados y artefactos generados.

5.3.1. Pruebas automatizadas

El repositorio contiene pruebas `pytest` para configuración, reglas de mapeo, hoja funcional, exportación DCAT, validación SHACL, validación runtime y workflow. La consola web

incluye tests (Vitest) para operaciones, lectura y escritura de CSV, resumen de jobs, artefactos y redacción de secretos. La suite se ejecuta con `python -m pytest` en el núcleo y `pnpm test` en `web/`, con los comandos exactos recogidos en el Anexo E. Estas pruebas cubren la lógica interna de forma reproducible, sin necesidad de inspeccionar manualmente cada módulo.

5.3.2. Validación estructural del estado vivo

La validación runtime compara la instancia de OpenMetadata con el contrato esperado. Comprueba que existan servicio, base de datos, esquemas, tablas y columnas definidos por `sql/opendata_demo_init.sql`, y que los datasets `gold` publicados tengan los metadatos de gobierno esperados según la hoja funcional. Se invoca con `om_dcat_sync validate-runtime --strict` y valida la trazabilidad entre la fuente técnica, OpenMetadata y la configuración funcional.

5.3.3. Validación SHACL

La validación SHACL usa el caso `hvd` de DCAT-AP-ES, que incluye las restricciones generales y las formas específicas para conjuntos de datos de alto valor [10]. Esas shapes están vendorizadas en `tfm_ingestor/.../resources/shacl/`, junto con un manifiesto que fija el origen. Puede ejecutarse de forma aislada con `om_dcat_sync validate-dcat --profile-case hvd`, aunque la ruta recomendada es integrarla en el workflow completo (listado 5.2), que ejecuta sincronización, exportación y validación en un único paso repetible.

```
1 python -m om_dcat_sync workflow run --allow-warnings
```

Código 5.2: Workflow canónico end-to-end: sincroniza gobierno, exporta DCAT-AP-ES y valida SHACL HVD.

5.3.4. Validación externa con el validador europeo (SEMIC/ITB)

Además de la validación SHACL local con las *shapes* DCAT-AP-ES congeladas, el catálogo exportado se ha sometido a una validación externa e independiente con el *SEMIC SHACL Validator* del *Interoperability Test Bed* (ITB) de la Comisión Europea [35]. Este servicio comprueba la conformidad de un grafo RDF frente a las especificaciones SEMIC oficiales. Se cargó el catálogo `validation_suite_catalog.jsonld` y se validó contra el grupo DCAT-AP, perfil **DCAT-AP HVD**, versión **3.0.0 Base Zero** (sin conocimiento de fondo), con sintaxis de entrada JSON-LD.

El resultado, recogido en la figura 5.8, fue **SUCCESS** con **0 errores**, **0 advertencias** y **0 mensajes**. Este hallazgo es relevante por dos motivos. Primero, confirma que la salida no so-

lo cumple el perfil nacional DCAT-AP-ES validado localmente, sino también el perfil europeo *DCAT-AP HVD* verificado por una herramienta oficial ajena a la plataforma. Segundo, la modalidad *Base Zero* aplica las restricciones sin resolver vocabularios externos, de modo que una conformidad sin hallazgos evidencia que la estructura, las relaciones y las propiedades del grafo son correctas por sí mismas y no por inferencias del validador.

The screenshot shows the SEMIC SHACL Validator web interface. At the top, it features the European Commission and SEMIC logos. The main heading is 'SEMIC SHACL Validator'. Below this, there are links to GitHub, SEMIC Support Centre, SEMIC SHACL Validator, and SEMIC XML Validator. A descriptive paragraph explains the service's purpose: checking RDF graphs against SEMIC specifications. It mentions that validation is available via REST and SOAP APIs and provides a link to documentation. Below the description, there are input fields for 'File to validate' (set to 'validation_suite_catalog.jsonld'), 'Group' (set to 'DCAT-AP'), 'Validate for' (set to 'DCAT-AP HVD'), 'Release' (set to '3.0.0 Base Zero (no background knowledge)'), and 'Content syntax' (set to 'JSON-LD'). A 'Validate' button is present. The 'Validation result' section shows an 'Overview' table with the following data: Date: 2026-05-31T20:37:29.371Z, File name: validation_suite_catalog.jsonld, Validation type: DCAT-AP HVD 3.0.0 Base Zero validation (no background knowledge), Findings: 0 error(s), 0 warning(s), 0 message(s), and Result: SUCCESS. At the bottom of the result section, there are three buttons: 'Download report', 'Download SHACL shapes', and 'Download validated content'.

Figura 5.8: Validación externa del catálogo en el *SEMIC SHACL Validator* (ITB, Comisión Europea): perfil DCAT-AP HVD 3.0.0 Base Zero, resultado SUCCESS sin errores ni advertencias. Elaboración propia.

5.3.5. Resultados del caso de uso de validación

El caso de uso de validación parte de dos conjuntos de datos de negocio de la capa gold:

- *Agenda cultural pública*, a partir de `gold.agenda_cultural_publica`;
- *Resumen de movilidad por municipio*, a partir de `gold.movilidad_resumen_municipio`.

Como cada una se ingiere bajo los dos servicios de la doble ingesta (`postgres_demo_service` y `postgres_validation_service`), el catálogo exportado contiene **cuatro datasets publicables** (dos por servicio), cada uno identificado por su FQN.

La documentación del repositorio registra una ejecución completa de la suite con los siguientes resultados: conformidad runtime, conformidad técnica, conformidad de gobierno, conformidad SHACL, idempotencia positiva, cuatro datasets exportados, cuatro cambios aplicados en la primera ejecución y cero cambios aplicados en la segunda. Esta última cifra evidencia la idempotencia: una vez sincronizado el estado, repetir el workflow no vuelve a aplicar operaciones innecesarias.

Comprobación	Resultado documentado	Interpretación
Estado runtime	Conforme	OpenMetadata contiene los activos esperados.
Gobierno aplicado	Conforme	La hoja funcional coincide con metadatos vivos.
SHACL HVD (local)	Conforme con warnings permitidos	No hay violaciones bloqueantes.
Validación europea (ITB)	SUCCESS (0/0/0)	Conforme con DCAT-AP HVD 3.0.0 en el validador oficial europeo.
Datasets exportados	4	Dos tablas <code>gold</code> por cada uno de los dos servicios.
Segunda ejecución	0 cambios aplicados	El workflow es idempotente.

Tabla 5.3: Resumen de resultados del caso de uso de validación. Elaboración propia a partir de la documentación del repositorio.

5.3.6. Artefactos generados

Los artefactos esperados son:

- catálogo JSON-LD exportado;
- informe SHACL en Turtle;
- informe runtime en JSON;
- plan de workflow en JSON;
- resumen de suite de validación;
- informe de validación legible en HTML y PDF.

A partir del resumen JSON de la validación, la plataforma genera un *informe legible* en HTML y PDF mediante el comando `om_dcat_sync render-report`, que se apoya en la biblioteca `fpdf2` para el PDF. El informe resalta los indicadores de conformidad (estado runtime, gobierno, idempotencia y SHACL) y aplanar el resto de resultados en un listado consultable. Puede generarse desde la consola web (pantalla *Validación*) o desde el CLI, y queda

disponible como evidencia autocontenida para revisión, defensa o entrega a terceros sin necesidad de inspeccionar el JSON en bruto. La consola web sirve el informe HTML con su tipo Multipurpose Internet Mail Extensions (MIME) real mediante el botón *Abrir*, de modo que se renderiza directamente en el navegador (figura G.14, Anexo G).

Estos ficheros se generan en `tmp_pytest/`. No se versionan porque son reproducibles y dependen de la ejecución local, pero sí se referencian como evidencia regenerable. La cadena de pasos para regenerarlos desde un repositorio limpio se describe en el Anexo A, y el recorrido visual que conduce a cada artefacto en el Anexo F.

5.4. Discusión de los resultados

5.4.1. Cumplimiento del problema planteado

La solución resuelve el problema principal: conectar un catálogo técnico en OpenMetadata con una salida interoperable DCAT-AP-ES validable. La plataforma no se limita a documentar un mapeo, sino que implementa un flujo ejecutable que descubre activos técnicos, permite curación funcional, aplica metadatos, exporta JSON-LD y valida el resultado. El Anexo F narra ese flujo de extremo a extremo desde la consola web, hasta dejar el archivo RDF listo para entregarlo a un harvester compatible.

La decisión de usar PostgreSQL de referencia en lugar de cosechar CKAN refuerza la defendibilidad del trabajo. CKAN habría permitido transformar metadatos ya publicados. PostgreSQL y OpenMetadata permiten demostrar gobierno desde activos técnicos, que es más coherente con las competencias de ingeniería, Big Data y cloud del máster.

5.4.2. Beneficios obtenidos

Los principales beneficios son:

- reproducibilidad del entorno y del caso de uso;
- separación entre metadatos técnicos y curación funcional;
- núcleo Python único para CLI, scripts y web;
- validación formal mediante SHACL;
- trazabilidad entre SQL, OpenMetadata, hoja funcional, JSON-LD e informe de validación;
- facilidad de operación mediante consola web.

5.4.3. Limitaciones

La solución tiene limitaciones deliberadas:

- una tabla SQL no equivale semánticamente a un `dcat:Dataset`; en el caso de uso se usa como aproximación técnica gobernable;
- se genera una única *Distribution* y un único *DataService* por dataset;
- la clasificación HVD es una hipótesis de validación dentro de la plataforma, no una calificación jurídica automática;
- la validación se centra en metadatos, no en profiling de contenido ni calidad fila a fila;
- no se publica automáticamente en `datos.gob.es`, CKAN ni `data.europa.eu`;
- el modelo OpenMetadata se mantiene mínimo y no cubre todos los campos opcionales de DCAT-AP-ES.

La primera limitación merece una precisión. Un `dcat:Dataset` es una abstracción editorial: una colección publicada y mantenida por un agente como unidad coherente, con independencia de su almacenamiento físico [7]. Una tabla SQL es una estructura física, de modo que la correspondencia «una tabla es un dataset» es una decisión de modelado y no una equivalencia natural: un dataset podría corresponder igualmente a una vista, a una agregación o a la unión de varias tablas. En el caso de uso de validación se elige la tabla `gold` como unidad publicable por ser la representación más estable del dato refinado. En un escenario real, la hoja de gobierno permitiría asociar el dataset a la vista o agregación que mejor lo represente, sin alterar el núcleo.

Estas limitaciones no invalidan el trabajo: forman parte del alcance funcional declarado y delimitan la frontera entre lo que la plataforma demuestra de forma reproducible y lo que correspondería a una iniciativa de portal de datos abiertos a escala institucional, con presupuesto, equipo y compromisos de servicio diferenciados.

5.4.4. Amenazas a la validez

Siguiendo la clasificación habitual de amenazas a la validez en ingeniería del software experimental [36], se consideran los cuatro tipos:

- **Validez interna:** el riesgo de que los resultados (conformidad e idempotencia) se deban al diseño de la prueba y no a la plataforma. Se mitiga porque cada resultado se obtiene con comandos versionados, *shapes* congeladas por *commit* y exportación determinista, de modo que lo observado es atribuible al sistema y no a un estado puntual del entorno.

- **Validez externa:** el caso de uso emplea datos sintéticos y un número reducido de datasets de una única fuente PostgreSQL, por lo que la generalización a catálogos institucionales a gran escala queda fuera de lo demostrado. La arquitectura está diseñada para escalar a más tablas gold y a otras fuentes con la misma hoja funcional y el mismo workflow.
- **Validez de constructo:** la conformidad SHACL mide la corrección estructural del grafo frente al perfil, no su calidad editorial ni su utilidad. Por eso un catálogo conforme puede contener `sh:Warning:` el perfil *recomienda* propiedades adicionales no obligatorias [11], cuya ausencia no invalida la conformidad demostrada. La conformidad «con warnings permitidos» es, por tanto, una distinción explícita entre lo obligatorio y lo recomendado, no una amenaza.
- **Validez de conclusión:** los artefactos son reproducibles y deterministas, y la conformidad se corrobora con un validador europeo independiente (sección 5.3.4), lo que reduce el riesgo de que las conclusiones dependan de la implementación local.

5.5. Trazabilidad entre objetivos y evidencias

La tabla 5.4 relaciona cada objetivo con el bloque donde se aborda y con la evidencia versionada que lo respalda.

Objetivo	Bloque	Evidencia del repositorio
Análisis DCAT-AP-ES	Estado del arte	docs/dcat_mapping.md, bibliografía oficial
Mapeo OpenMetadata	Arquitectura e implementación	dcat_export.py, governance_service.py
Configuración de gobierno	Implementación	gold_governance.csv, governance_defaults.yaml
Doble ingesta PostgreSQL	Implementación y validación	ingest_postgres_double.ps1, gold_governance.csv
Workflow reproducible	Implementación y validación	workflow_service.py, run_validation_suite.ps1
Exportación JSON-LD	Validación	dcat_export.py, artefactos *.jsonld
Validación SHACL	Validación	shacl_validation.py, shapes vendorizadas
Consola operativa	Implementación	web/, operations.ts, jobRunner.ts

Tabla 5.4: Trazabilidad entre objetivos, bloques de la memoria y evidencias. Elaboración propia.

6. Conclusiones y trabajos futuros

Este capítulo cierra la memoria revisando el objetivo principal, comprobando el cumplimiento de los objetivos parciales y dónde se cubren, justificando las competencias asociadas y planteando las líneas de evolución.

6.1. Revisión del objetivo principal

El objetivo general, **diseñar y desplegar una configuración de metadatos en OpenMetadata alineada con DCAT-AP** que **mejore la interoperabilidad y la gestión semántica de catálogos de datos**, se ha cumplido. El trabajo no se queda en una configuración conceptual: entrega una Plataforma de Gobierno del Dato reproducible que descubre activos técnicos en OpenMetadata, los enriquece con metadatos funcionales mediante una hoja de gobierno versionada, sincroniza ese gobierno de forma repetible, exporta un catálogo DCAT-AP-ES en JSON-LD y valida su conformidad con SHACL para el caso HVD, contrastada además con el validador europeo oficial.

La aportación central es la separación de tres responsabilidades: OpenMetadata como catálogo técnico, la hoja funcional como punto de curación editorial y un núcleo Python único como mecanismo de sincronización, exportación y validación, reutilizado por la CLI y la consola web sin duplicar reglas de negocio. Esa separación es la que hace el resultado reproducible, trazable y operable también por perfiles no desarrolladores.

6.2. Cumplimiento de los objetivos

Los objetivos específicos propuestos para el TFE se han cubierto en su totalidad, en los bloques que recoge la tabla 6.1.

Objetivo	Cómo se ha cubierto	Dónde
Analizar DCAT-AP y sus extensiones	Análisis de DCAT, DCAT-AP, DCAT-AP-ES y HVD, RDF/JSON-LD y SHACL	Estado del arte
Mapear clases y propiedades de DCAT-AP con OpenMetadata	Mapeo activo de <code>dcat:Catalog</code> , <code>dcat:Dataset</code> , <code>dcat:Distribution</code> , <code>dcat:DataService</code> y <code>foaf:Agent</code>	Resultados (arquitectura) y Anexo de mapeo
Configurar taxonomías y <i>custom metadata</i> en OpenMetadata	Custom properties DCAT y etiquetas <code>dcat_theme.*</code> gobernadas de forma repetible	Resultados (implementación)
Implementar un pipeline de ingestión/sincronización	Doble ingesta PostgreSQL y workflow <code>om_dcat_sync</code> de sincronización	Resultados (implementación)
Validar mediante exportación/federación del catálogo	Exportación JSON-LD y validación SHACL HVD (local y validador europeo)	Resultados (validación)
Evaluar beneficios y limitaciones	Discusión de beneficios, limitaciones y amenazas a la validez	Resultados (discusión)

Tabla 6.1: Cumplimiento de los objetivos del TFE y bloque donde se evidencian. Elaboración propia.

6.3. Justificación de competencias

El trabajo desarrolla las dos competencias oficialmente asociadas al TFE:

- **CN02** (conocer las arquitecturas para el tratamiento masivo de datos y las técnicas de almacenamiento, orquestación y gestión de pipelines): se evidencia en el despliegue reproducible sobre Kubernetes y Helm, en la fuente PostgreSQL organizada en capas *medallion*, en la doble ingesta y en el workflow orquestado de extremo a extremo.
- **CP03** (implementar marcos de trabajo de gobierno de datos y estrategias de aseguramiento de la calidad para gestionar la integridad, la seguridad y la accesibilidad de los datos masivos): se evidencia en el gobierno centralizado y declarativo de metadatos, la curación funcional bajo responsabilidad humana, la conformidad verificable con DCAT-AP-ES y el perfil HVD mediante SHACL, y la trazabilidad entre la fuente técnica, el catálogo técnico, la hoja funcional, el JSON-LD y el informe de validación.

El trabajo asume además limitaciones explícitas (datos sintéticos, un subconjunto controlado de DCAT-AP-ES y ausencia de publicación automática hacia portales externos), diseñadas como frontera de alcance y no como déficit técnico, que no afectan al resultado

principal: una solución reproducible, trazable y defendible que transforma activos técnicos gobernados en un catálogo interoperable validado formalmente.

6.4. Trabajo futuro

Las líneas de evolución más razonables son:

- publicar automáticamente el JSON-LD en un CKAN o portal compatible cuando exista un destino real;
- ampliar la cobertura de metadatos recomendados y opcionales de DCAT-AP-ES;
- incorporar aprobaciones editoriales y roles si la web pasa a uso multiusuario;
- automatizar ejecuciones periódicas mediante Airflow, CI/CD o un scheduler, entendidas como automatización de la reproducibilidad y no como delegación de la decisión de publicar;
- validar el comportamiento con más fuentes técnicas y más datasets `gold`, por ejemplo mediante ingesta desde APIs o bases de datos públicas.

Referencia bibliográfica

- [1] Comisión Europea. Una Estrategia Europea de Datos. <https://eur-lex.europa.eu/legal-content/ES/TXT/?uri=CELEX:52020DC0066>, 2020. Comunicación COM(2020) 66 final; impulsa los espacios europeos de datos (data spaces); última consulta: mayo de 2026.
- [2] Iniciativa Aporta. Guía técnica y modelo DCAT-AP-ES 1.0.0. <https://datosgobes.github.io/DCAT-AP-ES/>, 2025. Última consulta: mayo de 2026.
- [3] Comisión Europea. Reglamento de Ejecución (UE) 2023/138 relativo a conjuntos de datos de alto valor (HVD). <https://eur-lex.europa.eu/legal-content/ES/TXT/?uri=CELEX:32023R0138>, 2023. Diario Oficial de la Unión Europea; última consulta: mayo de 2026.
- [4] SEMIC – European Commission. DCAT Application Profile for Data Portals in Europe (DCAT-AP). <https://semiceu.github.io/DCAT-AP/releases/2.1.1/>, 2023. Versión 2.1.1; última consulta: mayo de 2026.
- [5] W3C. RDF 1.1 Concepts and Abstract Syntax. <https://www.w3.org/TR/rdf11-concepts/>, 2014. W3C Recommendation; última consulta: mayo de 2026.
- [6] W3C. JSON-LD 1.1. <https://www.w3.org/TR/json-ld11/>, 2020. W3C Recommendation.
- [7] W3C. Data Catalog Vocabulary (DCAT) - Version 3. <https://www.w3.org/TR/vocab-dcat-3/>, 2024. W3C Recommendation; última consulta: mayo de 2026.
- [8] DAMA International. DAMA-DMBOK: Data Management Body of Knowledge. <https://dama.org/learning-resources/dama-data-management-body-of-knowledge-dmbok/>, 2026. Última consulta: mayo de 2026.

-
- [9] David Puig and Juan Mañes. El potencial uso de la metodología de DAMA en la gestión de los datos abiertos. <https://datos.gob.es/es/blog/el-potencial-uso-de-la-metodologia-de-dama-en-la-gestion-de-los-datos-abiertos>, 2021. Publicado en datos.gob.es; última consulta: mayo de 2026.
- [10] Iniciativa Aporta. Validación DCAT-AP-ES. <https://datosgobes.github.io/DCAT-AP-ES/validation/>, 2025. Última consulta: mayo de 2026.
- [11] W3C. Shapes Constraint Language (SHACL). <https://www.w3.org/TR/shacl/>, 2017. W3C Recommendation.
- [12] Collate. OpenMetadata Documentation. <https://docs.open-metadata.org/>, 2026. Documentación oficial de OpenMetadata; última consulta: mayo de 2026.
- [13] Anjana Data. Gobierna tus datos abiertos conforme a DCAT-AP-ES: listo para publicar en datos.gob.es. <https://anjanadata.com/gobierna-tus-datos-abiertos-conforme-a-dcat-ap-es-listo-para-publicar-en-datos-gob-es/>, 2026. Referencia de mercado consultada en mayo de 2026.
- [14] DataHub Project. DataHub: A Metadata Platform for the Modern Data Stack. <https://datahubproject.io/docs/>, 2026. Documentación oficial; última consulta: mayo de 2026.
- [15] Amundsen (LF AI & Data Foundation). Amundsen: data discovery and metadata engine. <https://www.amundsen.io/amundsen/>, 2026. Documentación oficial; última consulta: mayo de 2026.
- [16] The Kubernetes Authors. Kubernetes Concepts. <https://kubernetes.io/docs/concepts/>, 2026. Documentación oficial; última consulta: mayo de 2026.
- [17] pytest team. pytest – helps you write better programs. <https://docs.pytest.org/>, 2026. Documentación oficial; última consulta: mayo de 2026.
- [18] Vitest team. Vitest – Next Generation Testing Framework. <https://vitest.dev/>, 2026. Documentación oficial; última consulta: mayo de 2026.
- [19] Gobierno de España. Norma Técnica de Interoperabilidad de Reutilización de Recursos de Información del Sector Público (NTI-RISP). <https://datos.gob.es/es/doc-tags/norma-tecnica-de-interoperabilidad-nti-risp>, 2013. Resolución de 19 de febrero de 2013, BOE 4 de marzo de 2013; última consulta: mayo de 2026.
- [20] David J. Anderson. *Kanban: Successful Evolutionary Change for Your Technology Business*. Blue Hole Press, 2010. ISBN 978-0984521401.
-

- [21] Project Management Institute. A Guide to the Project Management Body of Knowledge (PMBOK Guide), 7th Edition, 2021. ISBN 978-1628256642.
- [22] GitHub, Inc. About Projects (GitHub Projects). <https://docs.github.com/en/issues/planning-and-tracking-with-projects/learning-about-projects/about-projects>, 2026. Documentación oficial; última consulta: mayo de 2026.
- [23] Docker, Inc. Docker overview. <https://docs.docker.com/get-started/docker-overview/>, 2026. Documentación oficial; última consulta: mayo de 2026.
- [24] Kubernetes SIGs. kind – Kubernetes IN Docker. <https://kind.sigs.k8s.io/docs/>, 2026. Documentación oficial; última consulta: mayo de 2026.
- [25] Helm. Using Helm. https://helm.sh/docs/intro/using_helm/, 2026. Última consulta: mayo de 2026.
- [26] Meta Platforms. React – The library for web and native user interfaces. <https://react.dev/>, 2026. Documentación oficial; última consulta: mayo de 2026.
- [27] Vercel. Next.js Documentation. <https://nextjs.org/docs>, 2026. Documentación oficial; última consulta: mayo de 2026.
- [28] Microsoft. TypeScript Handbook. <https://www.typescriptlang.org/docs/handbook/intro.html>, 2026. Documentación oficial; última consulta: mayo de 2026.
- [29] Mermaid Contributors. Mermaid – Diagramming and charting tool. <https://mermaid.js.org/intro/>, 2026. Documentación oficial; última consulta: mayo de 2026.
- [30] Databricks. What is a Medallion Architecture? <https://www.databricks.com/glossary/medallion-architecture>, 2024. Documentación oficial; última consulta: mayo de 2026.
- [31] The PostgreSQL Global Development Group. PostgreSQL Documentation. <https://www.postgresql.org/docs/>, 2026. Documentación oficial; última consulta: mayo de 2026.
- [32] Python Software Foundation. dataclasses – Data Classes (Python Standard Library). <https://docs.python.org/3/library/dataclasses.html>, 2026. Documentación oficial; última consulta: mayo de 2026.
- [33] RDFLib Team. RDFLib – pure Python package for working with RDF. <https://rdflib.readthedocs.io/>, 2026. Documentación oficial; última consulta: mayo de 2026.

-
- [34] Ashley Sommer. pySHACL – Python validator for SHACL. <https://github.com/RDFLib/pySHACL>, 2026. Repositorio oficial; última consulta: mayo de 2026.
- [35] Comisión Europea – DG DIGIT, Interoperability Test Bed. SEMIC SHACL Validator. <https://www.itb.ec.europa.eu/shacl/semic-shacl/upload>, 2026. Servicio oficial de validación SHACL del Interoperability Test Bed (ITB); última consulta: mayo de 2026.
- [36] Claes Wohlin, Per Runeson, Martin Höst, Magnus C. Ohlsson, Björn Regnell, and Anders Wesslén. *Experimentation in Software Engineering*. Springer, 2012. ISBN 978-3-642-29043-5.
- [37] OpenMetadata. Kubernetes Deployment. <https://docs.open-metadata.org/v1.11.x/deployment/kubernetes>, 2026. Última consulta: mayo de 2026.

A. Reproducción del caso de uso de validación

Este anexo concentra los comandos exactos para reproducir el caso de uso de validación desde un repositorio limpio, clonado del repositorio público del TFM en GitHub.¹ Se asume Windows 10 u 11 con PowerShell, Docker Desktop, Node.js 20 o superior, Python 3.11 o superior y MiKTeX para compilar la memoria. Los comandos están extraídos de `scripts/infra/` y de la documentación canónica del repositorio.

A.1. Preparación del entorno

```
1 python -m pip install --upgrade pip
2 python -m pip install -e .\tfm_ingestor
```

Código A.1: Instalación de dependencias Python del núcleo `om_dcat_sync`.

```
1 cd .\web
2 pnpm install
```

Código A.2: Instalación de dependencias de la consola web con pnpm.

A.2. Despliegue de infraestructura

```
1 powershell -ExecutionPolicy Bypass -File .\scripts\infra\launch_infra.ps1
2 powershell -ExecutionPolicy Bypass -File .\scripts\infra\ingest_postgres_double.
   ↪ ps1
```

¹https://github.com/AlonsoMarcosM/TFM_Alonso_Marcos_Mu-oz

Código A.3: Lanzamiento de Kind, Helm e ingesta doble.

A.3. Ejecución del workflow canónico

```
1 python -m om_dcat_sync workflow run --dry-run
2 python -m om_dcat_sync workflow run --allow-warnings
```

Código A.4: Ejecución del workflow canónico end-to-end.

A.4. Suite reproducible

```
1 powershell -ExecutionPolicy Bypass -File .\scripts\infra\run_validation_suite.ps1
  ↵
```

Código A.5: Suite reproducible documentada.

La salida de la suite incluye `runtime_validation_report.json`, `dcat_catalog.jsonld` y `dcat_validation_report.ttl` en `tmp_pytest/`.

B. Detalle de infraestructura

Este anexo amplía la sección 5.2.6 con los detalles de despliegue que conviene tener accesibles sin entrar al repositorio.

B.1. Versión de OpenMetadata

El despliegue de referencia del caso de uso de validación utiliza **OpenMetadata 1.12.9**, instalado mediante los *charts* oficiales de Helm [37]. La versión exacta puede consultarse en cualquier momento contra la instancia viva mediante el endpoint `/api/v1/system/version` de la API REST. Las dependencias gestionadas por el *chart* `openmetadata-dependencies` son MySQL como almacén de metadatos y OpenSearch como motor de búsqueda. Las entidades de la API empleadas por la plataforma (servicios, bases de datos, esquemas, tablas, columnas, *custom properties* y *tags*) son estables en la rama 1.12.x, por lo que la lógica de gobierno no depende de detalles internos de una versión concreta.

B.2. Matriz de versiones del sistema

La reproducibilidad exige que todas las versiones del sistema estén fijadas y documentadas. La tabla B.1 recoge las versiones empleadas en el caso de uso de validación y el punto del repositorio donde quedan ancladas. El detalle operativo se mantiene actualizado en `docs/versiones.md`.

Componente	Versión	Dónde se fija
OpenMetadata (app y charts)	1.12.9	--version en launch_infra.ps1
Helm (CLI)	3.14.4	.tools/helm-v3.14.4/
Kind	0.31.0	prerrequisito (check_prereqs.ps1)
kubectl	1.34.1	prerrequisito
Docker Engine	28.5.1	prerrequisito
Python	3.12 (>= 3.10)	tfm_ingestor/pyproject.toml
Node.js	22.11	prerrequisito de web/
pnpm	11.x	gestor Node canónico
Next.js / React / TypeScript	16.2 / 19.2 / 6.0	web/package.json
pyshacl	>= 0.28	extra validation (pyproject.toml)
PyJWT / cryptography	>= 2.8 / 42	extra infra (pyproject.toml)
fpdf2	>= 2.7	extra report (pyproject.toml)
Bundle SHACL DCAT-AP-ES	1.0.0 (f2c8a888)	resources/shacl/manifest.json
Perfiles DCAT	DCAT-AP-ES 1.0.0 DCAT-AP 2.1.1 HVD 2.2.0	documentación oficial [2]

Tabla B.1: Matriz de versiones fijadas del sistema. Elaboración propia.

B.3. Valores Helm de OpenMetadata

Los valores canónicos viven en `k8s/openmetadata.values.yaml` y `k8s/openmetadata-dependencies.values.yaml`. Documentan: tamaño de pods, configuración de Elasticsearch (OpenSearch), credenciales por defecto del JWT de servicio, namespace y puertos expuestos.

B.4. Manifiesto de PostgreSQL de referencia

El manifiesto `k8s/postgres-demo.yaml` crea un Deployment, un Service y un ConfigMap con el contenido de `sql/opendata_demo_init.sql`. La inicialización es automática: el contenedor monta el SQL como volumen y lo ejecuta en el arranque.

B.5. Diagrama de despliegue

El diagrama de la figura G.3 resume el despliegue. En operación real, el `kubectl port-forward` expone OpenMetadata en `localhost : 8585` y PostgreSQL en `localhost : 5432`.

B.6. Notas operativas

- El clúster Kind se borra con `kind delete cluster --name tfm` si se requiere partir de cero.
- Las credenciales por defecto son las de la documentación oficial de OpenMetadata [12]; en producción deberían rotarse.
- No se han activado *políticas de red* ni RBAC avanzado por estar fuera del alcance académico. Las *políticas de red* (recurso `NetworkPolicy` de Kubernetes) restringen qué *pods* pueden comunicarse entre sí y hacia el exterior; el RBAC (*Role-Based Access Control*) limita qué acciones puede ejecutar cada usuario o cuenta de servicio sobre los recursos del clúster. Ambos son controles de *hardening* habituales en producción que no aportan valor al caso de uso de validación, centrado en gobierno de metadatos y reproducibilidad.

B.7. Instalación portable y organismos con OpenMetadata propio

La plataforma es portable y no exige desplegar su propio OpenMetadata. El despliegue con Kind, Helm y PostgreSQL de referencia descrito en este anexo solo construye un entorno autocontenido y reproducible para el caso de uso de validación. La lógica de gobierno reside íntegramente en el núcleo Python `om_dcat_sync`, que únicamente necesita acceso a la API REST de un OpenMetadata y un token JWT válido.

En un organismo o empresa que ya disponga de OpenMetadata, la integración no requiere recrear infraestructura: basta con apuntar la plataforma a la instancia existente.

- Configurar `OPENMETADATA_BASE_URL` hacia la API del OpenMetadata del cliente (por ejemplo, `https://catalogo.organismo.es/api/v1`).
- Proporcionar un token de servicio en `OPENMETADATA_TOKEN` o `OPENMETADATA_JWT_TOKEN`, obtenido de un *bot* de ingesta del propio OpenMetadata.

-
- Omitir los pasos de creación de clúster (`launch_infra.ps1`) y ejecutar directamente el workflow de gobierno, exportación y validación, ya sea desde el CLI o desde la consola web.

De este modo, el mismo flujo (descubrimiento, curación funcional, sincronización repetible, exportación DCAT-AP-ES y validación SHACL) opera sobre el catálogo técnico que el cliente ya mantiene, sin imponer su motor de despliegue ni duplicar su gobierno. La compatibilidad se ha verificado contra OpenMetadata 1.12.x (sección B.1). Las versiones próximas que conserven las mismas entidades de la API REST son igualmente compatibles. Esta separación entre lógica portable y despliegue de referencia es la que permite ofrecer la plataforma como capa de conformidad DCAT-AP-ES sobre un OpenMetadata preexistente.

C. Esquema de gobierno funcional

C.1. Estructura de la hoja `gold_governance.csv`

La hoja `tfm_ingestor/config/gold_governance.csv` usa separador `;` y las siguientes columnas:

- `publicar`: si o no; controla la inclusión en el catálogo exportado.
- `schema_name`: esquema PostgreSQL (siempre `gold` en el caso de uso de validación).
- `table_name`: nombre de tabla en PostgreSQL.
- `table_fqn`: identificador completo en OM (`service.database.schema.table`).
- `titulo_dataset`: título funcional del dataset.
- `descripcion_dataset`: descripción de la fila desde la perspectiva editorial.
- `publicador`: nombre del organismo publicador (UCLM en el caso de uso).
- `tematica_dcat`: tema NTI-RISP del dataset (`cultura_ocio`, `transporte`, etc.).
- `categoria_hvd`: categoría HVD del Reglamento (UE) 2023/138.
- `access_url_distribucion`: URL pública o estable de la distribución.

C.2. Defaults globales

El archivo `tfm_ingestor/config/governance_defaults.yaml` centraliza los valores constantes del catálogo. Su edición altera el comportamiento de todos los datasets simultáneamente, principio clave de la sección 3.7.1.

D. Mapeo DCAT-AP-ES y entregables

Este anexo resume el mapeo entre el catálogo técnico de OM y el perfil DCAT-AP-ES, y enumera los entregables del caso de uso de validación. La referencia completa y canónica del mapeo, con la matriz íntegra de propiedades, cardinalidades y elevaciones de obligatoriedad respecto a DCAT-AP, se mantiene en `docs/dcat_mapping.md`.

D.1. Correspondencia OpenMetadata → DCAT-AP-ES

El exportador traduce un subconjunto deliberadamente mínimo de elementos de OM a clases y propiedades DCAT-AP-ES, completando el resto por configuración (tabla D.1).

Elemento en OpenMetadata	Destino en DCAT-AP-ES
Table/View (capa gold)	<code>dcat:Dataset</code>
<i>custom property</i> <code>dcat_access_url</code>	<code>dcat:Distribution / dcat:accessURL</code>
<i>custom property</i> <code>dcat_hvd_category</code>	<code>dcatap:hvdCategory</code>
<i>custom property</i> <code>dcat_publisher_name</code>	<code>foaf:name</code> del publicador
<i>tags</i> <code>dcat_theme.*</code>	<code>dcat:theme</code>
<i>defaults</i> del catálogo (YAML)	<code>dcat:Catalog</code>
<i>defaults</i> HVD (YAML)	<code>dcat:DataService</code> y restricciones HVD derivadas

Tabla D.1: Correspondencia mínima OpenMetadata → DCAT-AP-ES. Elaboración propia a partir de `docs/dcat_mapping.md`.

D.2. Qué gobierna OpenMetadata y qué deriva el sistema

En OM solo se mantienen los metadatos personalizados imprescindibles para no alargar artificialmente el modelo: `displayName`, `description`, `dcat_publisher_name`, `dcat_`

`hvd_category`, `dcat_access_url` y los *tags* `dcat_theme.*`. El resto del perfil activo se deriva durante la exportación: el `dcat:Catalog` completo desde `governance_defaults.yaml`; la legislación aplicable, las licencias HVD y los derechos de acceso; el `dcat:DataService` con su punto de acceso, descripción, contacto y página de documentación; y los enlaces `dcat:accessService` y `dcat:servesDataset` entre distribución, servicio y dataset.

D.3. Qué aporta activar el caso HVD

Activar el caso HVD no se limita a añadir una clasificación. En el perfil activo implica `dcatap:hvdCategory` en dataset y servicio, `dcatap:applicableLegislation` derivada para dataset, distribución y servicio, `dct:license` en distribución y servicio, un `dcat:DataService` explícito, `dcat:contactPoint` y `dct:accessRights` derivados de la configuración global, y la validación SHACL específica del caso `hvd`.

D.4. Entregables del caso de uso

El flujo produce un conjunto cerrado de entregables reproducibles, todos regenerables en `tmp_pytest/` y visualizados a lo largo de la memoria (tabla D.2).

Entregable	Formato	Dónde se visualiza
Catálogo gobernado	JSON-LD	Listados 3.1 y G.3; pantalla <i>DCAT</i> (fig. F.6)
Informe de validación SHACL	Turtle	Listado G.8; pantalla <i>SHACL</i> (fig. F.8)
Informe del estado vivo	JSON	Sección 5.3, validación <i>runtime</i>
Resumen de la suite	JSON	Tabla 5.3; pantalla <i>Validación</i> (fig. F.7)
Informe legible de validación	HTML y PDF	Generado por <code>render-report</code> ; pantalla <i>Artefactos</i> (fig. F.9)

Tabla D.2: Entregables del caso de uso de validación y su visualización en la memoria. Elaboración propia.

E. Validación y consola web

E.1. Comandos de validación

```
1 python -m om_dcat_sync validate-runtime --strict `
2     --output tmp_pytest/runtime_validation_report.json
3 python -m om_dcat_sync validate-dcat --profile-case hvd --allow-warnings `
4     --report-output tmp_pytest/dcat_validation_report.ttl
```

Código E.1: Validación independiente y por suite.

E.2. Estructura de la consola web

La consola web en `web/` es una aplicación Next.js con rutas independientes por etapa. El listado G.6 muestra cómo persisten los jobs. Documentación operativa completa: `docs/app_web.md`.

E.3. Pruebas de la consola web

```
1 cd .\web
2 pnpm test
```

Código E.2: Pruebas Vitest de la consola web.

F. Demostración guiada de la plataforma

Este anexo describe el recorrido completo que sigue un operario para gobernar metadatos en la plataforma desde el navegador, sin invocar comandos directamente sobre el núcleo Python. Su objetivo es triple: servir de referencia para reproducir la solución, soportar una defensa visual del trabajo y dejar trazado qué artefacto se obtiene en cada paso, hasta el archivo final RDF que un harvester externo puede consumir.

F.1. Propósito y supuestos iniciales

La demostración asume que el entorno se ha preparado siguiendo el Anexo A: el clúster Kind está activo con OM y sus dependencias desplegados mediante Helm, el PostgreSQL de referencia está inicializado con los esquemas *bronze*, *silver* y *gold*, los dos servicios de ingesta están registrados en OM y la consola web se sirve localmente con `pnpm dev` desde `web/`. La verificación rápida de prerequisites se realiza con el listado F.1.

```
1 kubectl get pods                                # OpenMetadata, MySQL, OpenSearch,  
    ↪ Postgres  
2 curl http://localhost:8585/api/v1/health-check  
3 cd .\web; pnpm dev                               # consola web en http://localhost:3000
```

Código F.1: Comprobación rápida del entorno antes de iniciar la demostración.

A partir de aquí toda la operación discurre por el navegador. La consola web no implementa lógica de gobierno: invoca los mismos servicios Python que el CLI (sección 5.2.3) y los scripts de `scripts/infra/`. La equivalencia entre acción visual y comando subyacente queda recogida en la tabla resumen al cierre del anexo (tabla F.1).

F.2. Apertura de la consola

La página inicial muestra el resumen de la plataforma y un *sidebar* con las once secciones de operación. El orden recomendado de uso reproduce el flujo lógico: *Infraestructura* → *Ingesta* → *Gobierno* → *Workflow* → *DCAT* → *Validación* → *SHACL* → *Artefactos*. Las dos secciones complementarias, *Estado vivo* y *Ejecuciones*, permiten consultar respectivamente la situación actual de OM y el historial de *jobs* versionados.

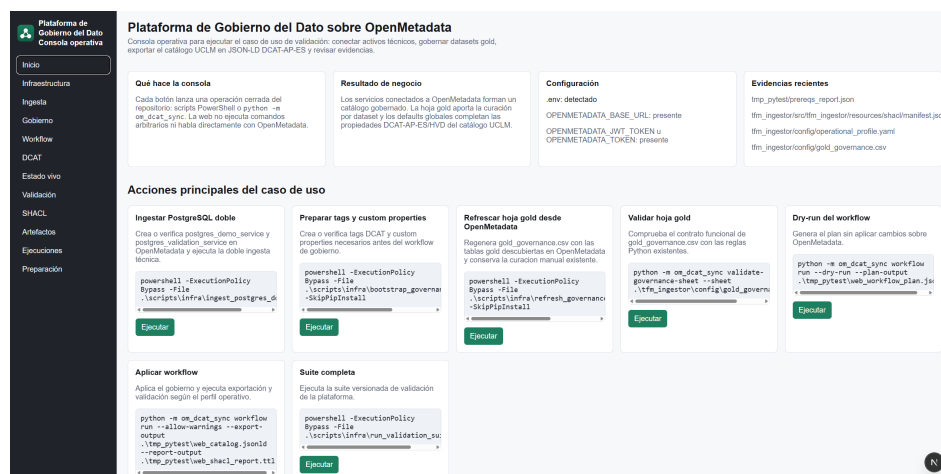


Figura F.1: Pantalla inicial de la consola web y *sidebar* con las secciones operativas. Elaboración propia.

F.3. Estado de la infraestructura

La pantalla *Infraestructura* expone dos acciones: comprobar prerequisites y, opcionalmente, ejecutar un reset limpio. La comprobación lanza un *job* que verifica la presencia de los binarios (docker, kubectl, helm, kind) y la accesibilidad de los servicios desplegados. Tras unos segundos se muestra el resultado con código de salida, log completo y duración. En la demostración solo se ejecuta la comprobación, ya que el entorno está listo.

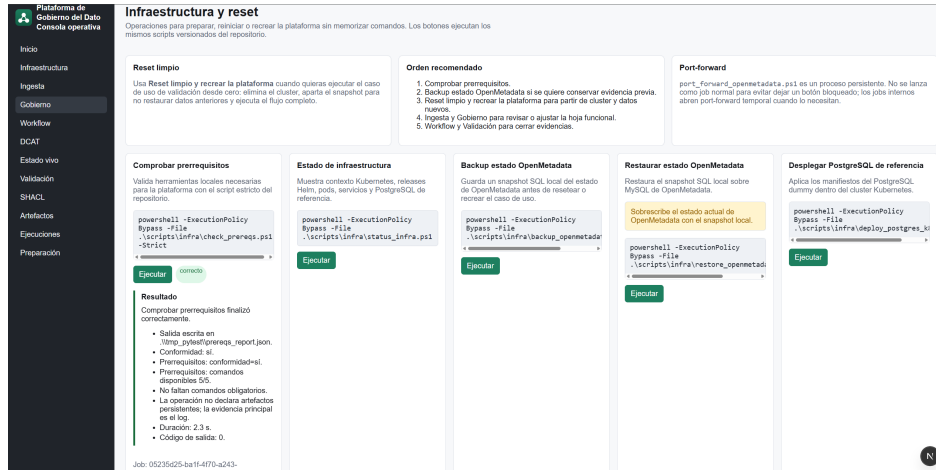


Figura F.2: Pantalla *Infraestructura*: comprobación de prerequisites y opciones de reset limpio. Elaboración propia.

F.4. Servicios y datasets ingeridos

En la pantalla *Ingesta* se observa el resultado de la doble ingesta técnica. Aparecen los dos DatabaseService en OM, postgres_demo_service y postgres_validation_service, cada uno con la misma estructura interna: la base de datos opendata_demo, los esquemas bronze, silver y gold, y las tablas descubiertas en cada esquema. La demostración confirma que las dos tablas gold se han registrado en ambos servicios y, por tanto, generan cuatro datasets publicables identificados por su FQN.

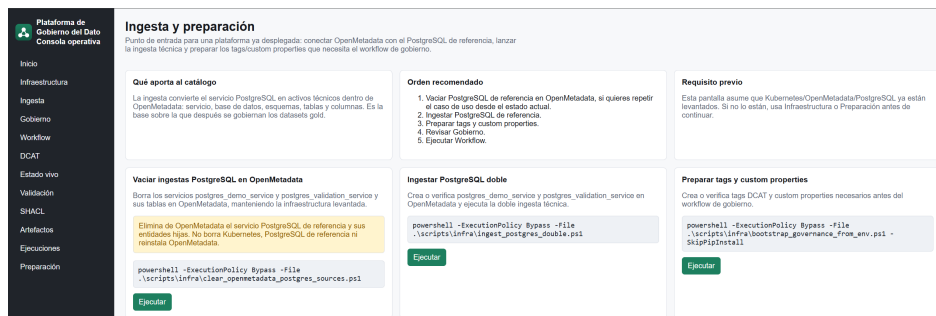


Figura F.3: Pantalla *Ingesta*: servicios PostgreSQL y datasets gold descubiertos por OpenMetadata. Elaboración propia.

F.5. Curación funcional en la pantalla *Gobierno*

Esta pantalla es el centro del recorrido. Concentra toda la curación funcional sobre la hoja gold_governance.csv (tabla G.1) y se utiliza en cuatro pasos.

1. Refrescar hoja desde OpenMetadata. La acción descubre las tablas gold de los dos servicios y añade las filas correspondientes a la hoja, preservando la curación previa. El operario ve cómo aparece la lista de datasets con su `table_fqn` único.

2. Editar metadatos DCAT-AP-ES. Por cada dataset publicable se completa título funcional, descripción, publicador, temática NTI-RISP, categoría HVD y Uniform Resource Locator (URL) de la distribución. La pantalla ofrece listas desplegables para las taxonomías controladas y validación inmediata de los campos obligatorios.

3. Validar la hoja. Una segunda acción ejecuta la validación funcional y marca cualquier fila incompleta o inconsistente; el operario puede corregir y volver a validar antes de aplicar nada en OM.

4. Guardar. El guardado persiste la hoja CSV en disco. Hasta este punto no se ha modificado el estado de OM: solo se han preparado los datos editoriales que el *workflow* aplicará en el siguiente paso.

Figura F.4: Pantalla *Gobierno*: edición de la hoja `gold_governance.csv` con listas controladas para temática y categoría HVD. Elaboración propia.

F.6. Aplicación del workflow

La pantalla *Workflow* ofrece dos botones equivalentes al listado 5.2. La demostración los ejecuta en secuencia para mostrar el comportamiento idempotente.

Dry-run. El primer botón ejecuta `workflow run --dry-run`. El núcleo Python descubre las tablas, fusiona los *defaults* YAML con la hoja editada y genera un plan en JSON con la

lista exacta de operaciones que se aplicarían sobre OM (PATCH, POST, etiquetas a añadir). El operario revisa el plan sin que nada haya cambiado todavía.

Apply. El segundo botón ejecuta `workflow run --allow-warnings` y aplica el plan. Al completarse, el resumen muestra el número de cambios realmente aplicados y los artefactos producidos: el catálogo JSON-LD y el informe SHACL.

Idempotencia. Repetir el botón *Apply* una segunda vez es ilustrativo: el resumen indica *0 cambios aplicados*, lo que evidencia visualmente la propiedad de idempotencia que la sección 5.3 valida formalmente.

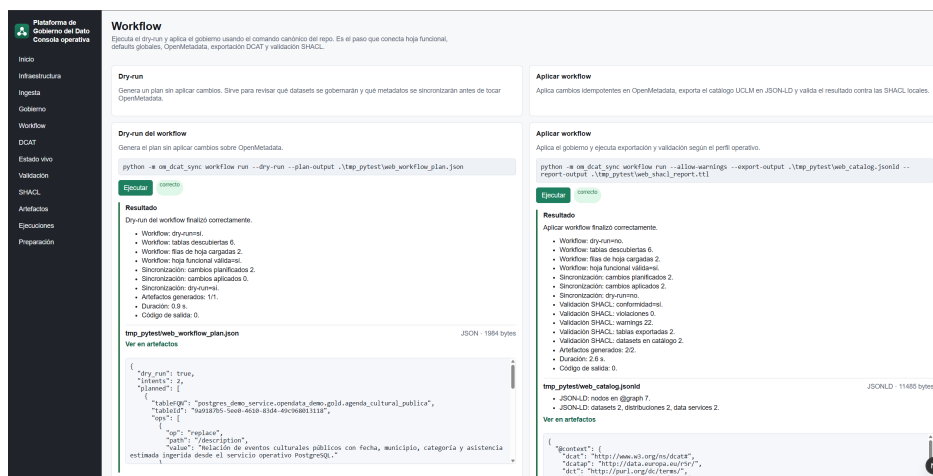


Figura F.5: Pantalla *Workflow*: dry-run y aplicación con resumen de cambios. Elaboración propia.

F.7. Exportación DCAT-AP-ES

La pantalla *DCAT* dispara la exportación independiente del catálogo. Aunque el *workflow* ya produce el mismo artefacto, esta pantalla permite regenerar el JSON-LD sin reaplicar el gobierno: útil si se ajusta únicamente algún *default* global. El resultado es el archivo `tmp_pytest/dcat_catalog.jsonld` con las cinco clases DCAT activas (Catalog, Dataset, Distribution, DataService, Agent).

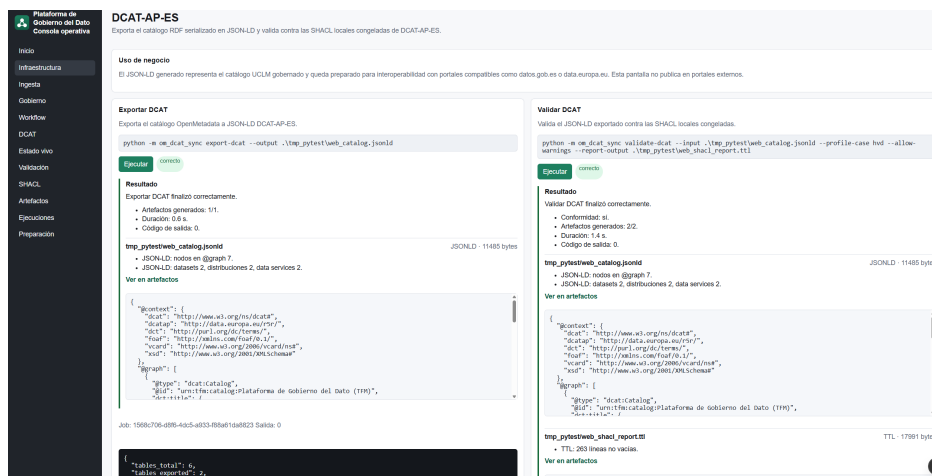


Figura F.6: Pantalla *DCAT*: exportación del catálogo JSON-LD y previusualización. Elaboración propia.

F.8. Validación SHACL y suite reproducible

La pantalla *Validación* ejecuta la suite reproducible completa (`run_validation_suite.ps1`): comprueba el estado vivo de OM, lanza el *workflow* dos veces para verificar la idempotencia y valida el catálogo exportado contra las *shapes* SHACL HVD vendorizadas. El resumen final consolida los códigos de salida de cada etapa en un único informe JSON.

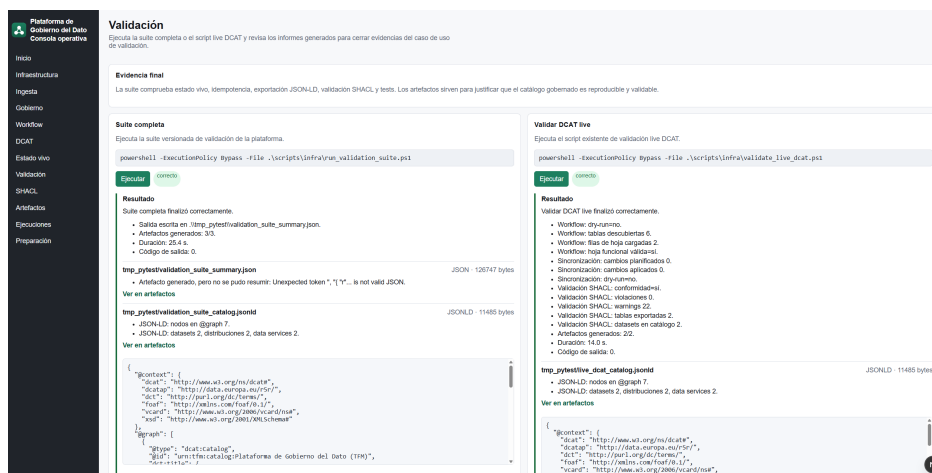


Figura F.7: Pantalla *Validación*: ejecución de la suite reproducible y resumen consolidado. Elaboración propia.

La pantalla *SHACL*, complementaria, muestra el *manifiesto de las shapes DCAT-AP-ES congeladas* en el repositorio (`resources/shacl/manifest.json`): origen, versión y *commit* de las *shapes*. Su función es evidenciar que la validación no descarga *shapes* en tiempo de ejecución, sino que usa exclusivamente las locales. El informe SHACL propiamente dicho, el

Turtle con severidades (sh:Violation, sh:Warning, sh:Info), nodo afectado y mensaje, se genera como artefacto *_shacl_report.ttl y se consulta desde la pantalla *Artefactos*. En la demostración no presenta violaciones bloqueantes, en línea con la sección 5.3.



Figura F.8: Pantalla *SHACL*: manifiesto de las *shapes* DCAT-AP-ES congeladas (sin descarga en tiempo de ejecución). Elaboración propia.

F.9. Artefactos y descarga del catálogo RDF

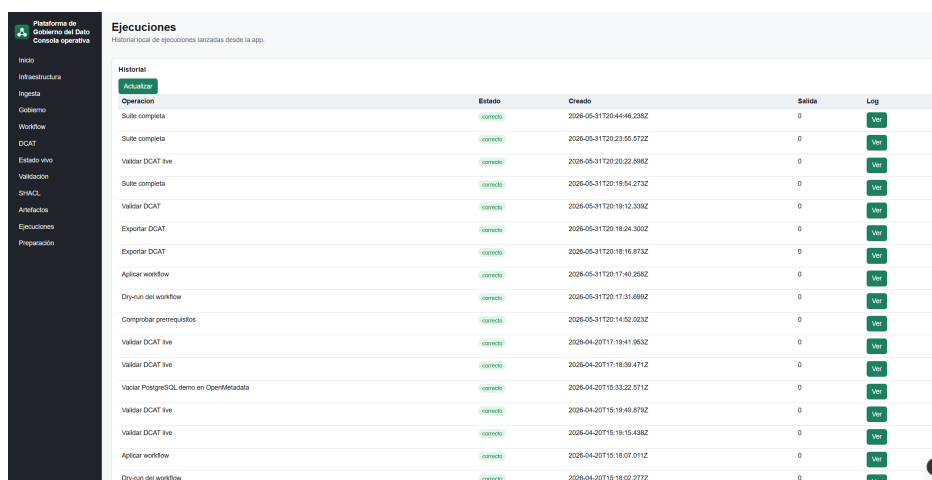
La pantalla *Artefactos* expone todos los ficheros generados en `tmp_pytest/` con dos acciones por fila: *Ver* (previsualización de texto en la misma página) y *Abrir* (sirve el fichero con su tipo MIME real en una pestaña nueva, de modo que el HTML se renderiza como página y el PDF se abre en el visor del navegador). El operario puede abrir o descargar directamente `dcat_catalog.jsonld`, el artefacto final del recorrido, el informe SHACL en Turtle o el informe HTML/PDF de validación.

Artefactos		Ficheros reproducidos generados por los comandos y scripts de la plataforma				
Artefactos	Ruta	Estado	Tamaño	Actualizado	Acción	
Infraestructura						
Ingesta	tmp_pytestgreengrass_report.json	existe	1973	2026-05-31T20:14:54.301Z	Ver	Abrir
Gobierno						
Workflow	tmp_pytestvalidation_suite_summary.json	existe	126747	2026-05-31T20:20:19.796Z	Ver	Abrir
DCAT	tmp_pytestvalidation_report.html	no existe	0	-	Ver	Abrir
Estado vivo	tmp_pytestvalidation_report.pdf	no existe	0	-	Ver	Abrir
Validación						
SHACL	tmp_pytestruntime_validation_report.json	existe	8502	2026-05-31T20:20:12.807Z	Ver	Abrir
Artefactos	tmp_pytestvalidation_suite_catalog.jsonld	existe	11485	2026-05-31T20:20:13.594Z	Ver	Abrir
Ejecuciones						
Preparación	tmp_pytestvalidation_suite_shacl_report.ttl	existe	17932	2026-05-31T20:20:14.418Z	Ver	Abrir
	tmp_pytestweb_catalog.jsonld	existe	11485	2026-05-31T20:18:24.853Z	Ver	Abrir
	tmp_pytestweb_shacl_report.ttl	existe	17981	2026-05-31T20:19:13.680Z	Ver	Abrir
	tmp_pytestweb_runtime_report.json	no existe	0	-	Ver	Abrir
	tmp_pytestweb_workflow_plan.json	existe	1984	2026-05-31T20:17:32.584Z	Ver	Abrir
	tmp_pytestlive_dcat_catalog.jsonld	existe	11485	2026-05-31T20:20:35.596Z	Ver	Abrir
	tmp_pytestlive_dcat_validation_report.ttl	existe	18004	2026-05-31T20:20:36.497Z	Ver	Abrir
	tmp_pytestworkflow_first_plan.json	existe	76	2026-05-31T20:20:11.023Z	Ver	Abrir
	tmp_pytestworkflow_second_plan.json	existe	76	2026-05-31T20:20:13.431Z	Ver	Abrir
	itm_ingestorconfigm_ingestorresourceshaclmanifest.json	existe	5161	2026-04-13T17:28:20.061Z	Ver	Abrir
	itm_ingestorconfigoperational_profile.yaml	existe	344	2026-05-12T17:17:41.431Z	Ver	Abrir

Figura F.9: Pantalla *Artefactos*: listado de evidencias con vista previa (*Ver*) y apertura nativa (*Abrir*) para HTML renderizado y PDF en el visor del navegador. Elaboración propia.

F.10. Historial de ejecuciones

La pantalla *Ejecuciones* lista todos los *jobs* generados por la consola en orden cronológico inverso. Cada entrada muestra el identificador único del *job*, la operación ejecutada, el estado (*ok* o *error*), la duración en segundos y el código de salida. El operario puede abrir cualquier *job* para ver el log completo, el resumen del resultado y los artefactos que produjo, lo que permite reconstruir la trazabilidad de cualquier ejecución pasada sin necesidad de volver a ejecutarla.



Operación	Estado	Creado	Salida	Log
Suite completa	correcto	2026-05-31T20:44:46.236Z	0	Ver
Suite completa	correcto	2026-05-31T20:23:55.572Z	0	Ver
Validar DCAT live	correcto	2026-05-31T20:20:22.586Z	0	Ver
Suite completa	correcto	2026-05-31T20:19:54.273Z	0	Ver
Validar DCAT	correcto	2026-05-31T20:19:12.336Z	0	Ver
Exportar DCAT	correcto	2026-05-31T20:18:24.300Z	0	Ver
Exportar DCAT	correcto	2026-05-31T20:18:16.873Z	0	Ver
Aplicar workflow	correcto	2026-05-31T20:17:40.206Z	0	Ver
Run del workflow	correcto	2026-05-31T20:17:31.699Z	0	Ver
Comprobar prerrequisitos	correcto	2026-05-31T20:14:52.023Z	0	Ver
Validar DCAT live	correcto	2026-04-20T17:19:41.953Z	0	Ver
Validar DCAT live	correcto	2026-04-20T17:18:38.471Z	0	Ver
Validar PostgreSQL demo en OpenMetadata	correcto	2026-04-20T17:16:33.22.571Z	0	Ver
Validar DCAT live	correcto	2026-04-20T15:19:49.879Z	0	Ver
Validar DCAT live	correcto	2026-04-20T15:19:15.438Z	0	Ver
Aplicar workflow	correcto	2026-04-20T15:18:07.011Z	0	Ver
Run del workflow	correcto	2026-04-20T15:18:02.277Z	0	Ver

Figura F.10: Pantalla *Ejecuciones*: historial de *jobs* con estado, duración, código de salida y acceso al log y artefactos. Elaboración propia.

F.11. Federación del catálogo

El archivo `dcatalog.jsonld` es RDF válido conforme a DCAT-AP-ES 1.0.0 con perfil HVD, comprobado por SHACL. A partir de ese punto el operario puede llevarlo a cualquiera de los destinos compatibles del ecosistema español o europeo de datos abiertos.

- **Harvester CKAN con extensión `ckanext-dcat`:** subir el JSON-LD a través del endpoint `/dataset/import` del portal destino. CKAN reconoce las clases DCAT y crea los datasets correspondientes.
- **Portal nacional `datos.gob.es`:** el archivo cumple los requisitos publicados de DCAT-AP-ES y puede ofrecerse al equipo de federación como entrada del proceso editorial del portal.
- **Triplestore RDF (Apache Jena Fuseki o equivalente):** cargar el catálogo en un *dataset* del *store* y consultarlo con SPARQL para auditoría o integración con otras fuentes RDF.

- **rdflib en Python:** cargar localmente el grafo con `rdflib.Graph().parse('dcat_catalog.jsonld', format='json-ld')` para análisis programático.

El listado F.2 ilustra cómo se entregaría el archivo a un harvester compatible.

```
1 curl -X POST https://harvester.example.org/api/dataset/import \
2   -H "Authorization: Bearer $TOKEN" \
3   -H "Content-Type: application/ld+json" \
4   --data-binary @tmp_pytest/dcat_catalog.jsonld
```

Código F.2: Envío del catálogo a un harvester CKAN con extensión DCAT.

La plataforma, por tanto, no se limita a producir un JSON-LD formalmente válido: lo entrega listo para integrarse con la cadena editorial real que se utiliza hoy en datos.gob.es, data.europa.eu y los portales basados en CKAN.

F.12. Resumen del recorrido

Paso	Pantalla / acción	Artefacto producido	Sección que lo formaliza
1	Infraestructura: comprobar	Estado de pods	§ 5.2.6
2	Ingesta: revisar	Tablas gold en OM	§ 5.2.2
3	Gobierno: editar y guardar	gold_governance.csv	§ 5.2.4
4	Workflow: dry-run	Plan JSON	§ 5.2.3
5	Workflow: apply	Cambios aplicados en OM	§ 5.2.3
6	DCAT: exportar	dcat_catalog.jsonld	§ 3.3
7	Validación: suite	Informe consolidado JSON	secc. 5.3
8	SHACL: manifiesto congelado	manifest.json (informe .ttl en Artefactos)	§ 3.6
9	Artefactos: descargar / Abrir	Catálogo RDF, informe HTML/PDF	§ F.9
10	Ejecuciones: revisar log	Historial de jobs trazables	§ F.10

Tabla F.1: Mapa del recorrido demo: acción del operario, artefacto producido y sección de la memoria donde se justifica formalmente. Elaboración propia.

Este recorrido es la demostración mínima defendible de la plataforma: parte de un entorno reproducible, opera íntegramente desde la consola web sin abandonar el navegador, recorre las nueve etapas con artefactos versionados y termina con un archivo RDF listo para entrar en la cadena editorial real de datos abiertos.

G. Capturas y listados complementarios

Este anexo reúne las capturas de pantalla, los listados de código y configuración y la tabla extensa que evidencian la implementación descrita en los capítulos de metodología y de resultados. Se han trasladado aquí, fuera del cuerpo principal, para respetar el límite de extensión, pero cada elemento se referencia desde la sección que lo discute. El material se organiza siguiendo el mismo orden que la memoria: primero la organización del trabajo, después la infraestructura, la ingesta y el gobierno en OpenMetadata, el núcleo Python, la consola web, y por último los listados y la tabla de gobierno.

G.1. Organización del repositorio y planificación

El árbol de la figura G.1 detalla la estructura canónica del repositorio descrita en la sección 4.6, y el diagrama de Gantt de la figura G.2 corresponde a la vista temporal del roadmap del capítulo de metodología.

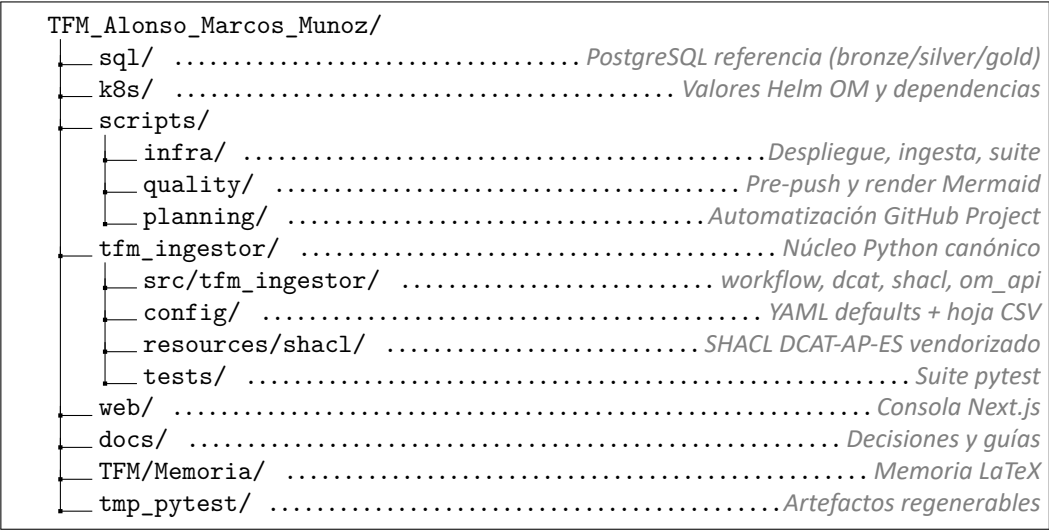


Figura G.1: Árbol canónico del repositorio del TFM. Elaboración propia.

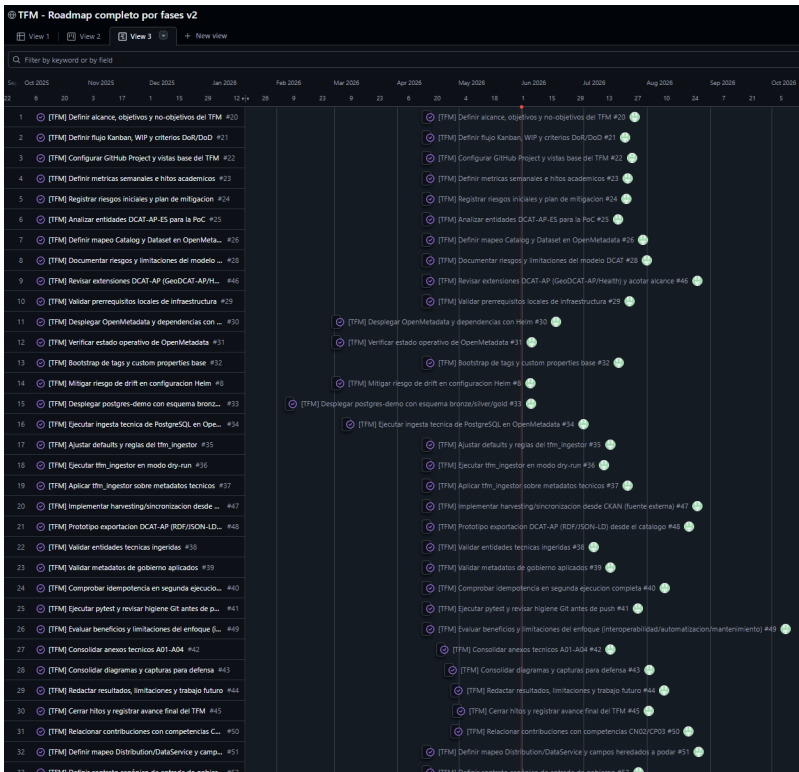


Figura G.2: Diagrama de Gantt del roadmap del TFM, derivado del archivo declarativo scripts/planning/github_project_planificacion.json. Elaboración propia.

G.2. Infraestructura y despliegue

Las tres figuras siguientes evidencian el despliegue reproducible de la sección 5.2.6: el esquema lógico del clúster, los contenedores Docker que lo materializan y los *pods* resultantes en Kubernetes.

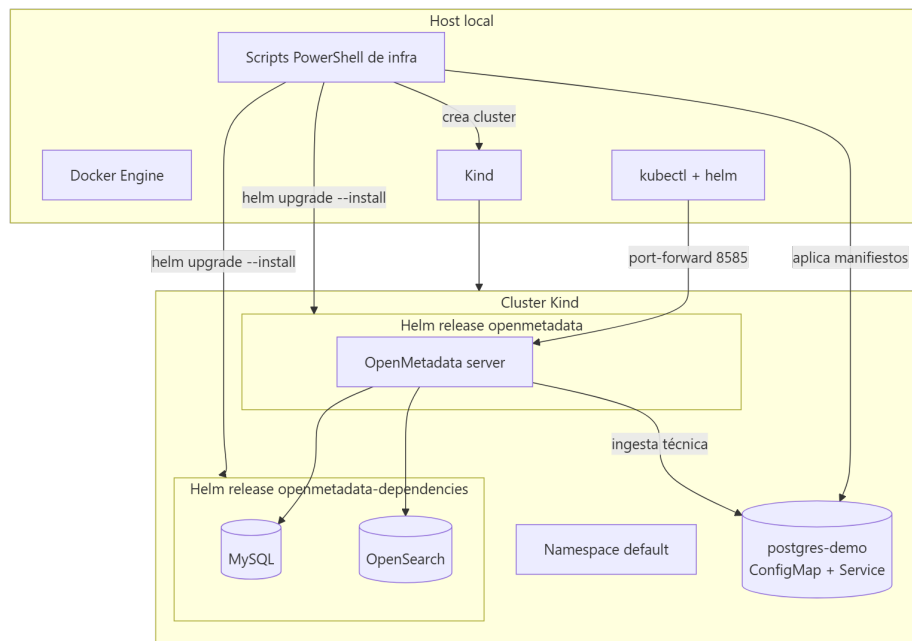


Figura G.3: Despliegue Kubernetes reproducible usado como base del caso de uso de validación. Elaboración propia.

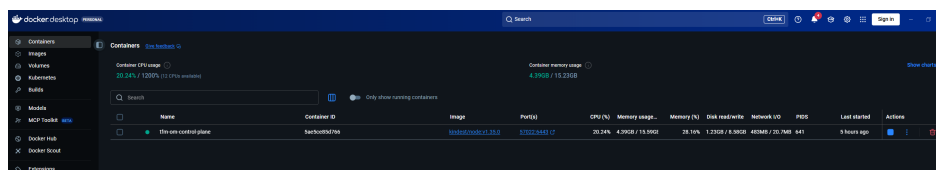


Figura G.4: Contenedores Docker que materializan el clúster Kind y los nodos auxiliares. Elaboración propia.

```
Microsoft Windows [Versión 10.0.26280.8457]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\Alonso>kubectl get pods -o wide
NAME                READY   STATUS    RESTARTS   AGE   IP            NODE                NOMINATED NODE   READINESS GATES
mysql-0              1/1     Running   6 (4h53m ago)  19d   10.244.0.6    tfm-om-control-plane   <none>           <none>
openmetadata-864f4874f5-v6xd9  1/1     Running   0          4h52m  10.244.0.9    tfm-om-control-plane   <none>           <none>
opensearch-0         1/1     Running   6 (4h53m ago)  19d   10.244.0.8    tfm-om-control-plane   <none>           <none>
postgres-demo-549fd8f8bc-7vghx  1/1     Running   6 (4h53m ago)  19d   10.244.0.5    tfm-om-control-plane   <none>           <none>
```

Figura G.5: Pods desplegados en el clúster Kind: OpenMetadata, MySQL, OpenSearch y PostgreSQL de referencia. Elaboración propia.

G.3. Ingesta y gobierno en OpenMetadata

Estas capturas evidencian la fuente técnica y el gobierno aplicado. La figura G.6 muestra las capas del PostgreSQL de referencia (sección 5.2.1); la figura G.7, los dos servicios registrados tras la doble ingesta (sección 5.2.2); y la figura G.8, las propiedades DCAT y las etiquetas aplicadas a una tabla gold tras ejecutar el workflow.

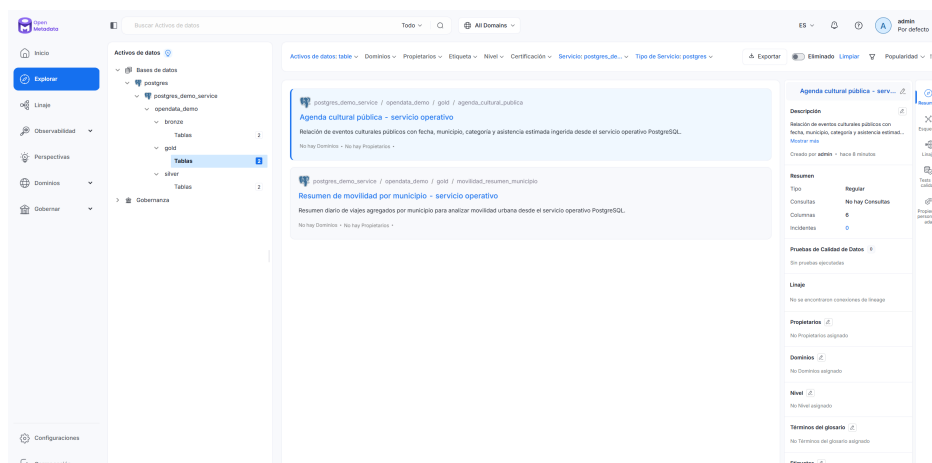


Figura G.6: Esquemas bronze, silver y gold del PostgreSQL de referencia, vistos desde la UI de OM. Elaboración propia.

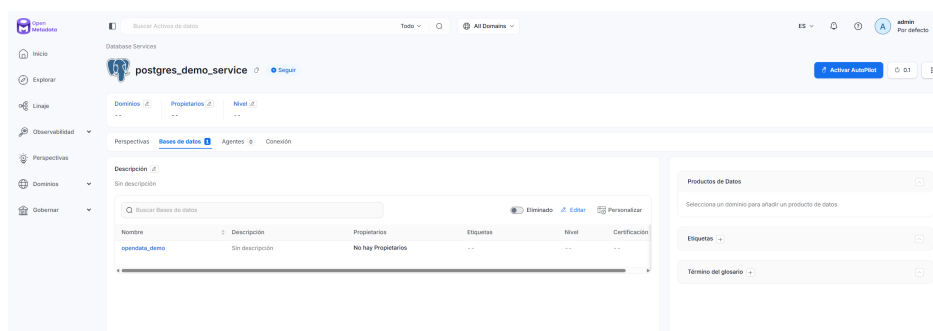


Figura G.7: Servicios de base de datos ingeridos en OpenMetadata tras la doble ingesta. Elaboración propia.

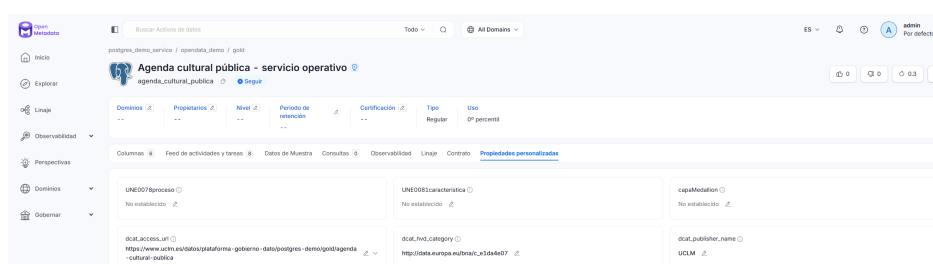


Figura G.8: Propiedades personalizadas DCAT y etiquetas dcat_theme.* aplicadas a una tabla gold en OpenMetadata tras ejecutar el workflow. Elaboración propia.

G.4. Núcleo Python y pipeline de metadatos

La figura G.9 resume la estructura interna del paquete `om_dcat_sync` (sección 5.2.3). El flujo operativo paso a paso y la transformación conceptual de los metadatos se ilustran en las figuras 5.4 y 5.7 del capítulo de Resultados (capítulo 5).

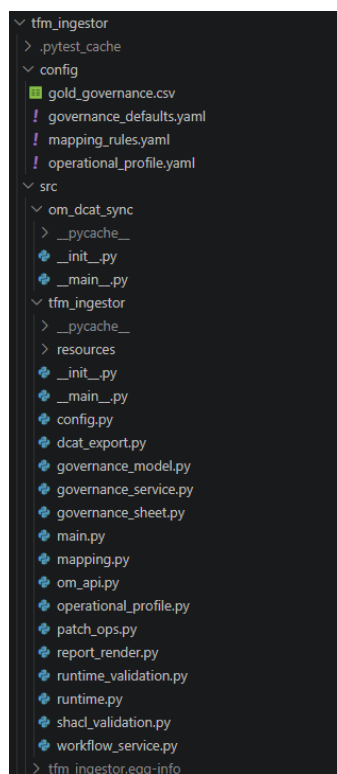


Figura G.9: Estructura interna del paquete `om_dcat_sync`. Elaboración propia.

G.5. Consola web operativa

Las pantallas de la consola web (sección 5.2.5) documentan la operación guiada del flujo: la edición de la hoja de gobierno, el lanzamiento del workflow, la exportación DCAT y la validación, además del informe de validación renderizado en el navegador.


Plataforma de Gobierno del Datos
Controla Operativa

Inicio
Infraestructura
Ingenta
Gobierno
Workflows
DCAT
Estado vivo
Validación
SHACL
Afectados
Ejecuciones
Preparación

Gobierno funcional gold

Esta es la plataforma funcional de los datasets gold concretos de la plataforma. La hoja cubre los campos por dataset que decide una persona responsable del catálogo; los demás obligatorios HVD se derivan desde la configuración global y el exportador DCAT.

¿Qué significa publicar

Al significar que la tabla gold entra en el catálogo DCAT exportado y debe tener título, descripción, publicador, temática, categoría HVD y URL de acceso, no deja la tabla fuera del contrato publicable de la plataforma.

Cobertura HVD

La hoja no contiene todos DCAT-AP-EVS HVD. Contiene la curación funcional por dataset. Catalog, legislation, tematicas, DatosInternos, controlParticip and paginas de informacion se generan desde `governance_defaults.yaml` y `dcats_export.py`.

Listas controladas

Inserta i.e. el `dcats` usa las secciones NTU, controladas, `category_hvd` usa las secciones NTU, controladas, `category_hvd` usa las secciones NTU, controladas, `category_hvd` usa las secciones NTU, controladas.

Hoja gold

Los identificadores técnicos son solo lectura. La validación autoritativa la hace Python; la web loquiza los errores de vocabulario y formato más frecuentes antes de guardar.

publicar

Usa o para determinar que se va a declarar y validar en la plataforma. Usa o no si se va a insertar o no en la base de datos. No se inserta en la base de datos si no se va a insertar.

título_dataset

Nombre formal, claro y legible. Evita códigos internos y nombres técnicos de base de datos.

descripcion_dataset

Obligatorio si publicar=1. Debe explicar el contenido de dataset, garantizar el uso y finalidad de modificación.

publicador

Nombre de la organización responsable. Si está vacío se usará el publicador configurado.

tematica_dataset

Siendo NTU (EVS) usado en `dcats` theme. La lista provendrá de la `SHACL` sección `controladas` de `governance_defaults.yaml`.

categoría_hvd

Categoría superior del vocabulario europeo HVD. En la plataforma se usan `Modicidad` y `Estadística`, pero la ley obliga los sea categorías oficiales.

access_urls

URLs, info de acceso, etc. para el dataset. No se inserta en la base de datos si no se va a insertar.

publicar	schema_name	table_name	table_type	título_dataset	descripcion_dataset	publicador	tematica_dataset	categoría_hvd	access_urls
si - publicar en el catálogo	gold	agenda_cultural_publica	portugues_demo_service_agenda_publica_demo gold agenda_cultural_publica	Agenda cultural pública	Relación de eventos culturales públicos con fecha, municipio, categoría y asistencia estimada. Agenda desde el	UCLM	Cultura y ocio	Estadística	...
si - publicar en el catálogo	gold	movilidad_resumen_municipio	portugues_demo_service_agenda_publica_demo gold movilidad_resumen_municipio	Resumen de movilidad	Relación de eventos culturales públicos con fecha, municipio, categoría y asistencia estimada. Agenda desde el	UCLM	Transporte	Modicidad	...
si - publicar en el catálogo	gold	agenda_cultural_publica	portugues_validation_service_agenda_publica_demo gold agenda_cultural_publica	Agenda cultural pública	Relación de eventos culturales públicos con fecha, municipio, categoría y asistencia estimada. Agenda desde el	UCLM	Cultura y ocio	Estadística	...
si - publicar en el catálogo	gold	movilidad_resumen_municipio	portugues_validation_service_agenda_publica_demo gold movilidad_resumen_municipio	Resumen de movilidad	Relación de eventos culturales públicos con fecha, municipio, categoría y asistencia estimada. Agenda desde el	UCLM	Transporte	Modicidad	...

Autogenerar vistas

Consultar hoja

Recargar

Refrescar hoja gold desde OpenMetadata

Validar hoja gold

Figura G.10: Pantalla *Gobierno* de la consola web: edición de la hoja `gold_governance.csv`. Elaboración propia.

**Plataforma de Gobierno del Data**
Consola operativa

Inicio

Infraestructura

Ingesta

Gobierno

Workflow

DCAT

Estado vivo

Visualizaci3n

SHACL

Afectaci3n

Ejecuciones

Preparaci3n

Workflow

Ejecuta el dry-run y aplica el gobierno usando el comando `catálogo del repo`. Es el paso que conecta hoja funcional, catálogo global, OpenMetadata, exportaci3n DCAT y validaci3n SHACL.

Dry-run

Genera un plan sin aplicar cambios. Sirve para revisar que datasets se gobiernan y que metadatos se sincronizan antes de tocar OpenMetadata.

```
python -m dcatalog_sync workflow run --dry-run --plan-output .\tmp_testyaml_workflow_plan.json
```

Ejecutar

Cancelar

Resultado

Dry-run del workflow finalizado correctamente.

- Workflow: dry-run.
- Workflow: tablas descubiertas 6.
- Workflow: filas de hoja cargadas 2.
- Workflow: hoja funcional validada.
- Sincronizaci3n: cambios aplicados 2.
- Sincronizaci3n: cambios aplicados 0.
- Sincronizaci3n: dry-run.
- Metadatos generados: 191.
- Duration: 5.9 s.
- C3digo de salida: 0.

tmp_testyaml_workflow_plan.json JSON - 1094 bytes

```
{
  "dry_run": true,
  "status": "2",
  "planned": [
    {
      "table_id": "testings_demo_service.opendata_demo_gobierno_cultural_publica",
      "table_id": "testings_demo_data_demo_agencia",
      "ops": [
        {
          "op": "replace",
          "path": "/description",
          "value": "Relaci3n de eventos culturales p3blicos con fecha, municipio, categora y asistencia estimada registrada desde el servicio operaci3n testings."
        }
      ]
    }
  ]
}
```

Aplicar workflow

Aplica cambios implementados en OpenMetadata, exporta el cat3logo UCLM en JSON-LD y valida el resultado contra las SHACL locales.

```
python -m dcatalog_sync workflow run --allow-warnings --export-output .\tmp_testyaml_catalog_jsonld --repo-output .\tmp_testyaml_workflow_repo.txt
```

Ejecutar

Cancelar

Resultado

Aplicar workflow finalizado correctamente.

- Workflow: dry-run.
- Workflow: tablas descubiertas 6.
- Workflow: filas de hoja cargadas 2.
- Workflow: hoja funcional validada.
- Sincronizaci3n: cambios planificados 2.
- Sincronizaci3n: cambios aplicados 2.
- Sincronizaci3n: dry-run.
- Validaci3n SHACL: conformada.
- Validaci3n SHACL: violaciones 0.
- Validaci3n SHACL: warnings 22.
- Validaci3n SHACL: tablas exportadas 2.
- Validaci3n SHACL: datasets en cat3logo 2.
- Metadatos generados: 22.
- Duration: 2.6 s.
- C3digo de salida: 0.

tmp_testyaml_catalog_jsonld JSONLD - 11460 bytes

```
{
  "@context": {
    "dc": "http://www.w3.org/ns/dcat#",
    "data": "http://data.europa.eu/mi/",
    "test": "http://test.org/flow/"
  },
  "@id": "dataset",
  "dcat:dataset": 2,
  "dcat:distributions": 2,
  "data:servers": 2
}
```

Ejecutar

Cancelar

Ver en afectados

```
{
  "context": {
    "dc": "http://www.w3.org/ns/dcat#",
    "data": "http://data.europa.eu/mi/",
    "test": "http://test.org/flow/"
  },
  "@id": "dataset",
  "dcat:dataset": 2,
  "dcat:distributions": 2,
  "data:servers": 2
}
```

Figura G.11: Pantalla *Workflow*: lanzamiento de dry-run y apply con resumen de cambios. Elaboración propia.

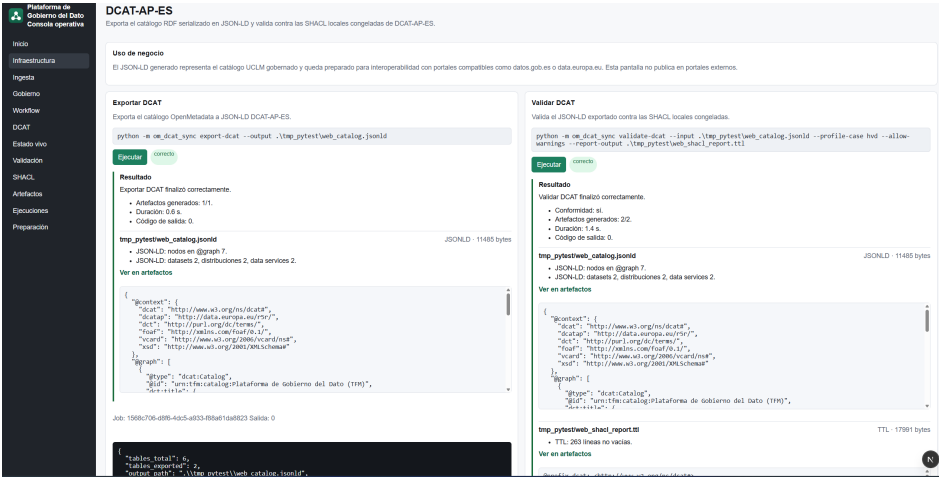


Figura G.12: Pantalla *DCAT*: exportación y previsualización del catálogo JSON-LD. Elaboración propia.

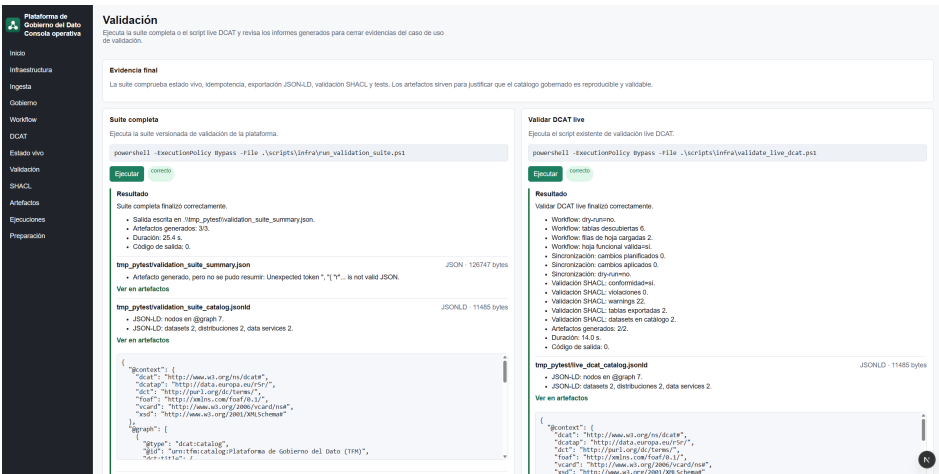


Figura G.13: Pantalla *Validación*: ejecución de la suite reproducible y resumen de resultados. Elaboración propia.

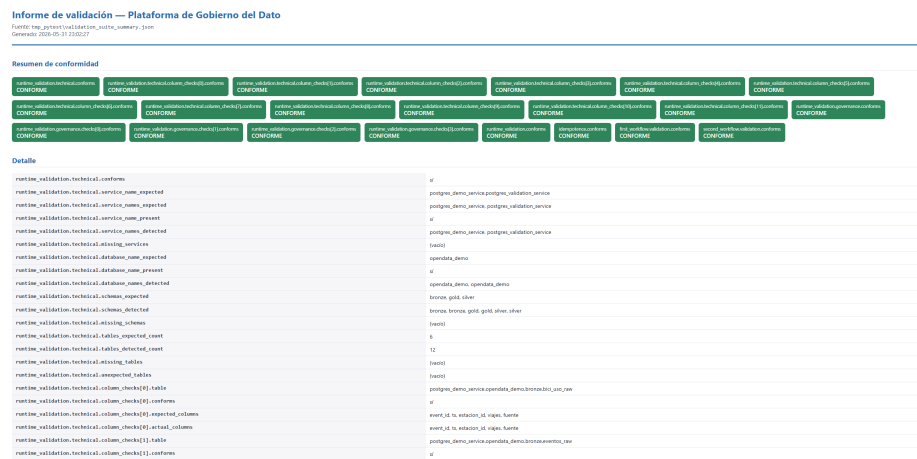


Figura G.14: Informe de validación HTML renderizado en el navegador desde la consola web. Elaboración propia.

G.6. Listados de código y configuración

G.6.1. Núcleo Python

El entriypoint del workflow (sección 5.2.3) se configura con una dataclass inmutable (listado G.1) y se orquesta en `run_workflow`, cuya secuencia operativa real se recoge en el listado G.2.

```

1 from dataclasses import dataclass
2
3 @dataclass(frozen=True)
4 class WorkflowRunConfig:
5     defaults_path: str
6     rules_path: str
7     sheet_path: str
8     base_url: str
9     token: str | None
10    limit: int = 1000
11    dry_run: bool = False
12    refresh_sheet: bool = True
13    export_output: str = "dcat_catalog.jsonld"
14    report_output: str | None = None
15    allow_warnings: bool = False
16    profile_case: str = "hvd"
17    plan_output: str | None = None
18    tables_input_path: str | None = None
19    skip_export: bool = False
20    skip_validate: bool = False

```

Código G.1: Configuración inmutable del workflow canónico en `workflow_service.py`.

```
1 def run_workflow(config: WorkflowRunConfig) -> dict[str, Any]:
2     defaults = load_defaults(config.defaults_path)
3     rules    = load_rules(config.rules_path)
4     api      = OpenMetadataApi(base_url=config.base_url, jwt_token=config.token)
5     tables   = discover_tables(api=api, limit=config.limit)
6
7     # 1) Refrescar la hoja desde activos descubiertos sin perder curación previa
8     if config.refresh_sheet:
9         existing = load_governance_sheet(config.sheet_path) if Path(config.
10 ↪ sheet_path).exists() else []
11         generate_governance_sheet(tables, defaults, config.sheet_path,
12 ↪ existing_rows=existing,
13                                   tags_by_prefix=rules.table_tags_by_prefix)
14
15     # 2) Sincronizar gobierno hacia OpenMetadata (idempotente)
16     plan = run_governance_sync(api=api, sheet_path=config.sheet_path,
17                               defaults=defaults, rules=rules, dry_run=config.
18 ↪ dry_run)
19
20     # 3) Exportar el catálogo DCAT-AP-ES y 4) validar con SHACL
21     if not config.skip_export:
22         export_catalog(api=api, defaults=defaults, output=config.export_output)
23     if not config.skip_validate:
24         export_and_validate_catalog(profile_case=config.profile_case,
25                                     report_output=config.report_output,
26                                     allow_warnings=config.allow_warnings)
27     return {"plan": plan, "sheet_refreshed": config.refresh_sheet}
```

Código G.2: Esqueleto de `run_workflow` en `workflow_service.py`.

G.6.2. Exportación, configuración e infraestructura

El contexto JSON-LD canónico del exportador (listado G.3), el fragmento de *values* Helm del despliegue (listado G.4), el perfil operativo que enlaza defaults, reglas y caso de validación (listado G.5) y la estructura de un *job* persistido por la consola web (listado G.6) completan la configuración descrita en los resultados.

```
1 "@context": {
2     "@language": "es",
3     "dcat": "http://www.w3.org/ns/dcat#",
```

```

4  "dct":    "http://purl.org/dc/terms/",
5  "foaf":   "http://xmlns.com/foaf/0.1/",
6  "skos":   "http://www.w3.org/2004/02/skos/core#",
7  "vcard":  "http://www.w3.org/2006/vcard/ns#",
8  "hvd":    "http://data.europa.eu/r5r/applicableLegislation/",
9  "tema":   "http://datos.gob.es/kos/sector-publico/sector/"
10 }

```

Código G.3: Contexto JSON-LD canónico generado por `dcat_export.py`.

```

1  openmetadata:
2    config:
3      elasticsearch:
4        host: opensearch
5        port: 9200
6      database:
7        host: mysql
8        port: 3306
9        driverClass: com.mysql.cj.jdbc.Driver

```

Código G.4: Fragmento de `k8s/openmetadata.values.yaml`: configuración mínima del release Helm.

```

1  defaults_path: "tfm_ingestor/config/governance_defaults.yaml"
2  rules_path:    "tfm_ingestor/config/mapping_rules.yaml"
3  sheet_path:    "tfm_ingestor/config/gold_governance.csv"
4
5  workflow:
6    profile_case:  "hvd"
7    allow_warnings: false
8    refresh_sheet: true
9    export_output: "dcat_catalog.jsonld"
10   report_output: "tmp_pytest/dcat_validation_report.ttl"

```

Código G.5: Perfil operativo `operational_profile.yaml` que enlaza defaults, reglas, hoja y caso de validación.

```

1  {
2    "id":          "job-2026-05-18T10-43-12-workflow-run",
3    "operation":   "workflow.run",
4    "status":      "ok",
5    "started_at":  "2026-05-18T10:43:12Z",
6    "duration_s":  47.8,
7    "exit_code":   0,
8    "summary":     { "tables": 4, "changes_applied": 0, "shacl_conformant": true },
9
10   ↵

```

```

9   "artifacts":  ["tmp_pytest/dcat_catalog.jsonld",
10                  "tmp_pytest/dcat_validation_report.ttl"]
11 }

```

Código G.6: Estructura de un *job* persistido en state/web_jobs/.

G.6.3. Ejemplos de RDF y SHACL

Como apoyo al estado del arte (sección 3.3), el listado G.7 muestra en Turtle el mismo grafo que el ejemplo JSON-LD del cuerpo, y el listado G.8 ilustra una restricción SHACL mínima.

```

1  @prefix dcat: <http://www.w3.org/ns/dcat#> .
2  @prefix dct:  <http://purl.org/dc/terms/> .
3  @prefix ex:   <https://www.uclm.es/datos/plataforma-gobierno-dato/> .
4
5  ex:agenda-cultural-publica a dcat:Dataset ;
6      dct:title             "Agenda cultural pública - servicio operativo"@es ;
7      dct:description       "Relación de eventos culturales públicos."@es ;
8      dct:publisher         ex:UCLM ;
9      dcat:theme             <http://datos.gob.es/kos/sector-publico/sector/cultura-ocio>
10     ↪ .

```

Código G.7: Tripletas RDF que describen un dataset DCAT en sintaxis Turtle (mismo grafo que el listado 3.1).

```

1  [] a sh:NodeShape ;
2      sh:targetClass dcat:Dataset ;
3      sh:property [
4          sh:path      dct:title ;
5          sh:minCount  1 ;
6          sh:severity   sh:Violation ;
7          sh:message    "Todo dcat:Dataset debe declarar al menos un dct:title."@es ;
8      ] ;
9      sh:property [
10         sh:path      dct:description ;
11         sh:minCount  1 ;
12         sh:severity   sh:Violation ;
13         sh:message    "Todo dcat:Dataset debe declarar al menos una dct:description."
14         ↪ "@es ;

```

Código G.8: Fragmento ilustrativo de SHACL que exige título y descripción a cada Dataset DCAT.

G.7. Columnas de la hoja de gobierno

La tabla G.1 detalla las columnas de la hoja funcional `gold_governance.csv` (sección 5.2.4); los criterios de validación completos se recogen en el Anexo C.

Columna	Tipo	Propósito
publicar	si/no	Inclusión del dataset en el catálogo exportado.
schema_name	texto	Esquema PostgreSQL (gold en el caso de uso).
table_name	texto	Nombre técnico de la tabla.
table_fqn	texto	Identificador completo en OM, clave única.
titulo_dataset	texto	Título funcional editorial.
descripcion_dataset	texto	Descripción editorial del dataset.
publicador	texto	Nombre del organismo publicador.
tematica_dcat	enum NTI-RISP	Temática NTI-RISP del dataset.
categoria_hvd	enum HVD	Categoría del Reglamento (UE) 2023/138.
access_url_distribucion	URL	URL pública de la distribución.

Tabla G.1: Columnas de la hoja funcional `gold_governance.csv`. Elaboración propia.

H. Posicionamiento Big Data y gobierno supervisado

Este anexo recoge dos discusiones complementarias que, por espacio, se han trasladado fuera del cuerpo de la memoria: el posicionamiento del trabajo frente a los entornos Big Data y la responsabilidad humana en el gobierno supervisado.

H.1. Posicionamiento frente a entornos Big Data

La descripción oficial propuesta para el TFE sitúa el trabajo en “entornos Big Data y de computación en la nube”, expresión que conviene matizar para evitar una lectura equívoca. La plataforma no opera en la capa de procesamiento masivo de datos, sino en la capa de gobierno de metadatos e interoperabilidad. El caso de uso de validación trabaja con un volumen reducido de datos sintéticos y, por tanto, no constituye Big Data en el sentido estricto de volumen, velocidad y variedad. Esta frontera es coherente con la distinción establecida en el capítulo 2: el objeto gobernado son los metadatos, no los datos de negocio.

La relevancia para entornos Big Data es, en consecuencia, indirecta pero sustantiva. Los ecosistemas de datos a gran escala se caracterizan por activos heterogéneos, distribuidos y de múltiples fuentes, cuya utilidad depende de que permanezcan descubribles, descritos y reutilizables. Esa función (catalogación, descripción semántica e interoperabilidad) es precisamente la que DAMA encuadra como gestión de metadatos dentro del gobierno del dato [8]. La plataforma demuestra dicho mecanismo de forma reproducible: descubre activos técnicos, los describe, los exporta como catálogo DCAT-AP-ES y valida su conformidad.

En un escenario Big Data real, el proyecto se utilizaría sin cambios en su lógica de negocio, dado que el flujo opera sobre metadatos y es independiente del volumen del dato subyacente. El crecimiento se absorbería por tres vías: (i) la incorporación de nuevas fuentes y servicios en OpenMetadata, catálogo concebido para inventarios técnicos amplios y

heterogéneos [12]; (ii) la ampliación del contrato funcional añadiendo más tablas `gold` a la misma hoja de gobierno y al mismo workflow idempotente, sin modificar el núcleo; y (iii) el despliegue nativo en la nube, que habilita el escalado de la capa de catálogo y gobierno conforme a las primitivas de orquestación de Kubernetes [16]. La salida interoperable en DCAT-AP-ES es, además, el vehículo por el que un patrimonio de datos extenso se hace federable y localizable entre portales nacionales y europeos [2, 7]. En síntesis, el trabajo no escala procesando más datos, sino gobernando más metadatos, que es la dimensión en la que el gobierno aporta valor a un entorno Big Data.

H.2. Gobierno supervisado y responsabilidad del operador

Aunque el flujo es ampliamente automatizable y repetible, una parte esencial del gobierno reside en decisiones que deben permanecer bajo autoridad humana. Determinar qué activos son publicables, asignar la temática NTI-RISP [19], clasificar un dataset como de alto valor o redactar su título, descripción y publicador no son transformaciones deterministas, sino juicios editoriales y de responsabilidad. El carácter manual y dinámico de esta curación no es una carencia del trabajo, sino una propiedad deseada del gobierno del dato.

El marco DAMA es explícito en este punto: el gobierno del dato asigna responsabilidad (*accountability*) a roles definidos como el propietario y el *steward* del dato, y la automatización debe asistir esa función, no sustituirla [8, 9]. Por consiguiente, la persona que opera o gobierna la plataforma debe conocer y aplicar las buenas prácticas del dominio, y responder de aquello que publica. La interoperabilidad técnica no exime de la responsabilidad sobre el contenido publicado.

Este principio tiene una implicación normativa concreta. La condición de conjunto de datos de alto valor está determinada por el Reglamento de Ejecución (UE) 2023/138 [3]; la plataforma trata la clasificación HVD como hipótesis de validación y no como una calificación jurídica automática, como ya se reconoció al discutir las limitaciones de los resultados. Por ello, la categorización debe ser revisada y autorizada por una persona responsable antes de cualquier publicación efectiva, y nunca derivarse de forma ciega por mera configuración.

El diseño de la plataforma materializa esta filosofía mediante salvaguardas explícitas. La asistencia de autorrellenado solo completa campos vacíos con valores de validación y nunca sobrescribe la curación introducida por el operador, de modo que la automatización ayuda sin desplazar el criterio humano. Del mismo modo, las líneas de trabajo futuro orientadas a la ejecución periódica mediante un planificador o CI/CD deben entenderse como automatización de la reproducibilidad, no como delegación de la decisión de publicar. La recomendación operativa es inequívoca: que el sistema sea técnicamente capaz de automatizar el flujo de extremo a extremo no implica que deba ejecutarse sin supervisión; la publicación de metadatos gobernados, por su visibilidad externa y por sus consecuencias jurídicas

y reputacionales, debe ser un acto autorizado y supervisado.